VECTOR

# Flash Bootloader VolvoCar

## Technical Reference

Version 2.02.00


| | |
|---|---|
| **Authors:** | A. Caspari |
| **Version:** | 2.02.00 |
| **Status:** | Released |

# 1 Document Information

## 1.1 History

| Author | Date | Version | Remarks |
|---|---|---|---|
| A. Caspari | 2017-02-16 | 2.00.00 | ▶ Updated document for SLP2 |
| A. Caspari | 2017-04-27 | 2.01.00 | ▶ Added requirements for diag buffer size |
| A. Caspari | 2018-06-20 | 2.02.00 | ▶ Added information about reprogrammable ESS |

Table 1-1 Document History

## 1.2 Reference Documents

| No. | Source | Title | Document No. | Version |
|---|---|---|---|---|
| [3] | VCC | VCC General Software Download Specification | 31835914 | 010 |
| [4] | VCC | Versatile Binary Format Specification 2.6 | 31808832 | 011 |
| [5] | VCC | VCC - Security Access Algorithm | 31839109 | 003 |
| [6] | ISO | Diagnostics on CAN | 15765-3 | |
| [7] | ISO | Unified Diagnostic Services (UDS) | 14229-1 | |
| [8] | HIS | Functional Specification of a Flash Driver | - | 1.3 |
| [9] | Vector | Flash Bootloader User Manual | - | 2.7 |

Table 1-2 Reference Documents

## Contents

**1 Document Information ..... 2**
1.1 History ..... 2
1.2 Reference Documents ..... 2

**2 Introduction ..... 10**
2.1 Bootloader Integration Guide ..... 11
2.2 User Specific Requirements ..... 14

**3 FBL Software Architecture ..... 17**
3.1 [#oem_files] Components ..... 17
3.1.1 Bootloader Core Files ..... 17
3.1.2 Bootloader Application Files ..... 18
3.1.3 Generated Files ..... 19
3.1.4 Decompression Module Files ..... 19
3.1.5 Security Related Files ..... 19
3.1.6 Flash Driver Files ..... 20
1.1.1 MICROSAR Communication Stack Files ..... 20
1.1.2 MICROSAR Stub Files ..... 21
3.2 Startup ..... 21
3.3 Download Flow ..... 22
3.3.1 DiagnosticSessionControl ..... 22
3.3.2 ReadDataByIdentifier ..... 22
3.3.3 Security Access ..... 22
3.3.4 Download Secondary Bootloader ..... 22
3.3.5 Activate Secondary Bootloader ..... 22
3.3.6 Erase Flash Memory ..... 22
3.3.7 Download Application/Calibration Files ..... 22
3.3.8 Check Memory Routine ..... 23
3.3.9 Check Complete & Compatible Routine ..... 23
3.3.10 EcuReset ..... 23
3.4 User Callback Functions ..... 23
3.4.1 ApplFblWriteDataByIdentifier ..... 23
3.4.2 ApplFblReadDataByIdentifier ..... 24

**4 Configuration of the VolvoCar Flash Bootloader ..... 25**
4.1 General Configuration Setup ..... 25
4.2 Module Fbl | FblGeneral ..... 31
4.2.1 Application Task ..... 33
4.2.2 Common Data ..... 33

4.2.3 Compression Mode ..... 33
4.2.4 Data Processing Buffer Size ..... 33
4.2.5 Diagnostic Buffer Size ..... 33
4.2.6 Encryption Mode ..... 34
4.2.7 Fill Code ..... 34
4.2.8 Header Address ..... 34
4.2.9 Internal Memory Copy ..... 35
4.2.10 Maximum Amount Of Logical Blocks ..... 35
4.2.11 Maximum Number Of Segments ..... 35
4.2.12 Output Verification ..... 35
4.2.13 P2 Server ..... 35
4.2.14 P2* Server ..... 36
4.2.15 Pipelined Programming ..... 36
4.2.16 Pipelined Verification ..... 36
4.2.17 Presence Pattern ..... 36
4.2.18 Project State ..... 37
4.2.19 Reprogrammable LBT ..... 38
4.2.20 S3 Server Programming Session ..... 38
4.2.21 Start Function ..... 38
4.2.22 Stay in Boot ..... 39
4.2.23 Unaligned Data Transfer ..... 39
4.2.24 User Config File ..... 39
4.2.25 User Routine ..... 41
4.2.26 User Service ..... 41
4.2.27 User Subfunction ..... 42
4.2.28 Verification Segmentation ..... 42
4.2.29 Watchdog Service ..... 42
4.2.30 Watchdog Trigger Cycle ..... 42
4.2.31 Write Segmentation ..... 42
4.3 Module Fbl | FblMemoryDeviceTable ..... 42
4.4 Module Fbl | FblFlashBlockTable ..... 44
4.5 Module Fbl | FblLogicalBlockTable ..... 45
4.6 Module Fbl | VolvoCar ..... 47
4.6.1 Complete & Compatible Block Address ..... 47
4.6.2 Flash Erased Detection ..... 47
4.6.3 Maximum Readable DIDs ..... 48
4.6.4 RequestUpload Service ..... 48
4.6.5 Secret Key ..... 48
4.6.6 Seed/Key Module ..... 48
4.6.7 WriteDataByIdentifier Service ..... 48
4.7 Module FblCw ..... 48

4.8 Module Smh | SmhGeneral ... 52
4.9 Module Smh | SmhCryptographicKeys ... 52
4.10 Module Smh | SmhEncryption ... 53
4.11 Module Smh | SmhVerification ... 54
4.12 Module Smh | Crc ... 54
4.13 Module Smh | SmhSeedKey ... 54

**5 Validation / Invalidation of Flashed Application ... 55**
5.1 [#oem_valid] Introduction to VolvoCar Validation ... 55
5.2 [#oem_valfunc] The Validation Functions ... 56
5.3 Handling of Validity Information ... 57
5.4 Files, Blocks and Software Units ... 57
5.5 Logical Block Table ... 57
5.6 Programming Sequence and User Callback Functions ... 58
5.7 Presence Pattern ... 61
5.8 Validation Structure ... 62
5.8.1 General Concept ... 62
5.8.2 Validation Structure Format ... 64
5.8.3 Batch Scripts for Validation Structure Generation ... 65
5.8.3.1 Validation Structure Generation for SBL/Flash Driver ... 65
5.8.3.2 Validation Structure Generation for Application Software/Data ... 65
5.9 [#oem_time] Validation OK – Application Faulty ... 66

**6 Interaction with the Application ... 68**
6.1 FBL Header Files ... 68
6.2 FBL Version Information ... 68
6.3 Common Data Structure ... 68
6.4 [#oem_trans] Transitioning from Application to Bootloader ... 69
6.4.1 Method 1: Call FblStart() ... 70
6.4.2 Method 2: Start over Reset ... 72
6.4.3 Conclusion ... 73
6.5 Starting the Application from the Bootloader ... 74

**7 Flash Driver ... 75**
7.1 Flash Driver Functions ... 75
7.2 Memory Driver Interface ... 75

**8 Security Module ... 77**
8.1 Components ... 77
8.1.1 [#oem_sec] Security Access Diagnostic Service ... 77
8.2 Integration and Configuration ... 79

**9 API Description ..... 80**
9.1 Flash Driver Interface ..... 80
9.1.1 Overview ..... 80
9.1.2 Function Description ..... 80
9.1.2.1 FlashDriver_InitSync ..... 80
9.1.2.2 FlashDriver_DeinitSync ..... 80
9.1.2.3 FlashDriver_RWriteSync ..... 81
9.1.2.4 FlashDriver_RReadSync ..... 81
9.1.2.5 FlashDriver_REraseSync ..... 82
9.2 Callback-Function API ..... 83
9.2.1 Overview ..... 83
9.2.2 Function Description ..... 85
9.2.2.1 ApplFblInit ..... 85
9.2.2.2 ApplFblStartup ..... 85
9.2.2.3 ApplTrcvrNormalMode ..... 85
9.2.2.4 ApplFblSetVfp ..... 86
9.2.2.5 ApplFblResetVfp ..... 86
9.2.2.6 ApplFblReset ..... 87
9.2.2.7 ApplFblTask ..... 87
9.2.2.8 ApplFblCanBusOff ..... 87
9.2.2.9 ApplFblSecuritySeedInit ..... 88
9.2.2.10 ApplFblSecuritySeed ..... 88
9.2.2.11 ApplFblSecurityKey ..... 88
9.2.2.12 ApplFblSecurityInit ..... 89
9.2.2.13 ApplDiagUserSubFunction ..... 89
9.2.2.14 ApplDiagUserService ..... 90
9.2.2.15 ApplDiagRoutineControl ..... 90
9.2.2.16 ApplFblReadDataByIdentifier ..... 91
9.2.2.17 ApplFblWriteDataByIdentifier ..... 92
9.2.2.18 ApplFblExtProgRequest ..... 93
9.2.2.19 ApplFblIsValidApp ..... 93
9.2.2.20 ApplFblValidateBlock ..... 94
9.2.2.21 ApplFblInvalidateBlock ..... 94
9.2.2.22 ApplFblValidateApp ..... 95
9.2.2.23 ApplFblGetBlockErased ..... 96
9.2.2.24 ApplFblReadSecurityBytes ..... 96
9.2.2.25 ApplFblWriteSecurityBytes ..... 97
9.2.2.26 ApplFblReadPublicKey ..... 97
9.2.2.27 ApplFblReadPublicKeyChecksum ..... 98
9.2.2.28 ApplFblWritePublicKey ..... 99
9.2.2.29 ApplFblVerifyPublicKeyChecksum ..... 99

9.2.2.30 ApplFblOtpGetMemAreaErased ..... 100
9.2.2.31 ApplFblOtpReadEntry ..... 100
9.2.2.32 ApplFblOtpWriteEntry ..... 101
9.2.2.33 ApplFblGetCcBlockInfo ..... 101
9.2.2.34 ApplFblCheckCompatibility ..... 102
9.2.2.35 ApplFblSetCcReturnValue ..... 103
9.2.2.36 ApplFblWDInit ..... 103
9.2.2.37 ApplFblWDShort ..... 103
9.2.2.38 ApplFblWDLong ..... 104
9.2.2.39 ApplFblWDTrigger ..... 104
9.2.2.40 ApplFblFatalError ..... 104
9.2.2.41 ApplFblInitDataProcessing ..... 105
9.2.2.42 ApplFblDataProcessing ..... 105
9.2.2.43 ApplFblDeinitDataProcessing ..... 106
9.2.2.44 ApplFblStartApplication ..... 107
9.2.2.45 ApplFblGetPatternLocation ..... 107
9.2.2.46 ApplFblAdjustLbtBlockData ..... 108
9.2.2.47 ApplFblGetPatternState ..... 108
9.2.2.48 ApplFblClrPatternState ..... 109
9.2.2.49 ApplFblSetPatternState ..... 109
9.2.2.50 ApplFblDiagVerification ..... 110
9.2.2.51 ApplFblSetSecAccessDelayFlag ..... 110
9.2.2.52 ApplFblGetSecAccessDelayFlag ..... 111
9.2.2.53 ApplFblClrSecAccessDelayFlag ..... 111
9.2.2.54 ApplFblReadSecAccessInvalidCount ..... 112
9.2.2.55 ApplFblWriteSecAccessInvalidCount ..... 112
9.3 Data API ..... 114
9.3.1 Types ..... 114
9.3.1.1 tFblResult ..... 114
9.3.1.2 tFblAddress ..... 114
9.3.1.3 tFblLength ..... 114
9.3.1.4 tApplStatus ..... 114
9.3.1.5 tFblProgStatus ..... 114
9.3.1.6 tFlashStatus ..... 114
9.3.1.7 tBlockDescriptor ..... 115
9.3.1.8 tDiagSegmentList ..... 115
9.3.1.9 tDiagSegmentInfo ..... 115
9.3.1.10 tProcParam ..... 115
9.3.1.11 SecM_VerifyParamType ..... 116
9.3.1.12 SecM_StatusType ..... 116
9.4 Implementation Details ..... 116

9.4.1 Bootloader Assertions ..... 116
9.4.2 The Watchdog ..... 117
9.4.2.1 Watchdog Synchronization Support ..... 118
9.4.3 ApplFblReadDataByIdentifier ..... 119
9.4.4 ApplFblWriteDataByIdentifier ..... 119
9.4.5 One-Time Programmable Area ..... 120
9.4.6 Flash Erased Detection ..... 122
9.4.7 Reprogrammable LBT/ESS ..... 123
9.4.7.1 Constraints ..... 124
9.4.7.2 Configuration ..... 125
9.4.7.3 Build and Post-Processing ..... 125
9.4.7.4 LBT Update Process ..... 125
9.4.7.5 LBT Access from User Code ..... 126

**10 Miscellaneous ..... 128**
10.1 Functional vs. Physical ..... 128

**11 Deviations ..... 129**

**12 Terminology ..... 131**

**13 Contact ..... 132**

## Illustrations

Figure 2-1 Manuals and References for the Flash Bootloader ..... 10
Figure 3-1 Bootloader Module Dependencies ..... 17
Figure 3-2 Interaction of FBL and Application ..... 21
Figure 4-1 DaVinci Configurator: General Settings ..... 25
Figure 4-2 DaVinci Configurator: Target Settings ..... 26
Figure 4-3 DaVinci Configurator: Project Folder Structure ..... 26
Figure 4-4 DaVinci Configurator: Input Files ..... 27
Figure 4-5 DaVinci Configurator: Selection of ECU Instance ..... 27
Figure 4-6 DaVinci Configurator: Update Configuration ..... 28
Figure 4-7 DaVinci Configurator: Add FBL Modules (1) ..... 29
Figure 4-8 DaVinci Configurator: Add FBL Modules (2) ..... 29
Figure 4-9 DaVinci Configurator: Module Short Names ..... 30
Figure 4-10 DaVinci Configurator: Add Dummy Modules (1) ..... 31
Figure 4-11 DaVinci Configurator: Add Dummy Modules (2) ..... 31
Figure 4-12 DaVinci Configurator: FblGeneral Configuration ..... 32
Figure 4-13 DaVinci Configurator: FblGeneral User Config File ..... 40
Figure 4-14 DaVinci Configurator: FblMemoryDevices ..... 43
Figure 4-15 DaVinci Configurator: FblFlashBlockTable ..... 44
Figure 4-16 DaVinci Configurator: Block Protection ..... 45
Figure 4-17 DaVinci Configurator: FblLogicalBlockTable ..... 46
Figure 4-18 DaVinci Configurator: Module FblJlr ..... 47
Figure 4-19 DaVinci Configurator: PduR BSW Modules ..... 49
Figure 4-20 DaVinci Configurator: CDD ComStack Contribution ..... 49
Figure 4-21 DaVinci Configurator: Configuration of RSA key ..... 53
Figure 4-22 DaVinci Configurator: Configuration of Symmetric Key ..... 53
Figure 5-1 Detailed Validation Flow with User Callback Functions ..... 59
Figure 5-2 Illustration of the Presence Pattern Mechanism ..... 61
Figure 5-3 Definition: Software Units, Logical Blocks, Data Blocks ..... 63
Figure 6-1 Transition from Application to Bootloader via FblStart() ..... 70
Figure 6-2 Transition from Application to Bootloader via Reset ..... 72
Figure 8-1 Security Module Dependencies ..... 77
Figure 8-2 Seed Generation and Key Verification with the Security Access Procedure ..... 78
Figure 9-1 Memory Map of the Watchdog Code ..... 118
Figure 9-2 LBT Configuration with Option “Reprogrammable LBT” ..... 124
Figure 9-3 Initial Download Sequence with Option “Reprogrammable LBT” ..... 126

# 2 Introduction

This document covers the OEM-specific particularities of the Flash Bootloader. It complements the explanations started in the user manual with VolvoCar-specific details. All references there are resumed here in this document again and explained in detail. The connection between a reference in the user manual and its specific description in this document is the headline. Both the reference and its explanation can be found below the same headline.

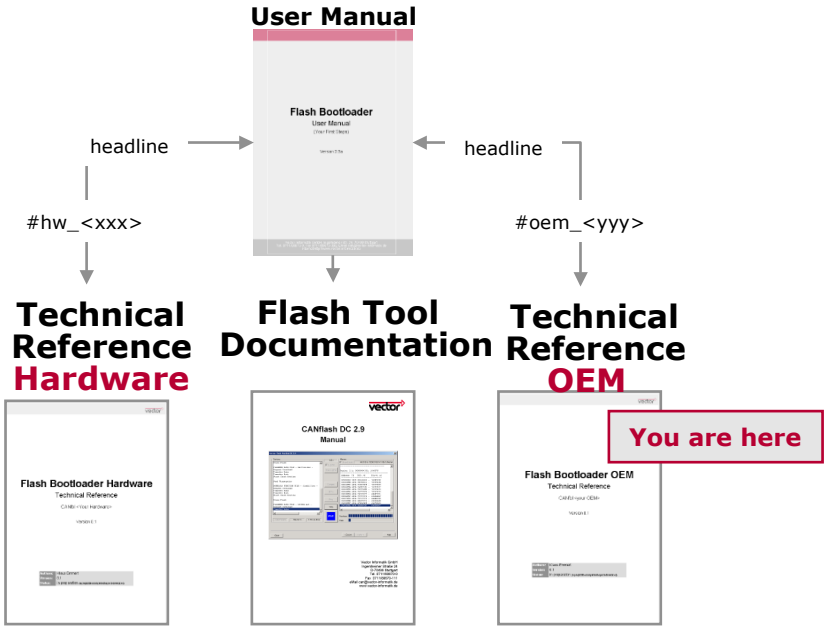



Figure 2-1 Manuals and References for the Flash Bootloader

Additionally, this headline is marked with the ID of the reference from the User Manual. This ID looks like: **[#oem_<yyy>]**.

## 2.1 Bootloader Integration Guide

This chapter is intended to provide a step-by-step guide how to integrate the VolvoCar flash bootloader into your project.

1. If you are not familiar with (Vector) bootloaders, read document [9] Flash Bootloader User Manual (UserManual_FlashBootloader.pdf) first. Note that this document provides a general overview, but might not be accurate in every detail. When it comes to details, always refer to the VolvoCar specific technical reference document at hand.
2. Install the bootloader delivery and take a look at the content (see chapter 3.1, Components).
3. The delivery always comes with a fully functional demo bootloader (folder Demo\DemoFbl). Take a look at the demo to get an impression which parts are required and how they work together. The included make environment allows you a hassle-free re-build of the demo bootloader.
4. Read the hardware specific technical reference document of the bootloader (TechnicalReference_FBL_<hardware>.pdf) to learn about the hardware specific particularities of the bootloader.
5. Think about the memory layout of your ECU. Decide which memory area(s) shall be used by the bootloader and which by the application software. You will need this information during bootloader configuration and adaptation of the make environment (e.g. linker command file). Consider OEM requirements REQPROD 53839, REQPROD 53975, REQPROD 53835 and REQPROD 53936.
6. Start your new project by collecting all files that are required to build the bootloader (see chapter 3.1, Components):
   - Core files
   - Application files (remove leading underscore)
   - Decompression module files
   - Security related files
   - Flash driver header file
   - Generated files
   - MICROSAR communication stack files and stubs
7. Configure the bootloader with DaVinci Configurator (see chapter 4, Configuration of the VolvoCar Flash Bootloader).
8. Adapt the hardware specific parts of the bootloader to the requirements of your ECU. At least, you should review, and – if necessary – adapt the following items:
   - Startup code:
     - Remember that there are two startup codes executed during ECU startup: the one of the bootloader and those of the application software – see chapter 3.2, Startup.

- The bootloader does NOT require initialization of the data/bss sections in RAM – in fact, this can be harmful if the pattern for entering the bootloader from application is located in RAM.
- The bootloader requires that some code parts are loaded to RAM during startup. The copy algorithm has to be provided by the startup code.

- fbl_ap.c::ApplFblInit(): This function usually initializes watchdog, clocks/PLL, I/O ports, etc.
- fbl_ap.c::ApplTrcvrNormalMode(): This function can be used to initialize the CAN transceiver (if required).
- fbl_apwd.c::ApplFblWDTrigger(): The code to serve an activated watchdog should be placed here. However, it's recommended to deactivate any watchdog circuit during the first bootloader setup.

9. Adapt the make environment: care for the correct memory location of the bootloader. Additionally, some code sections of the bootloader have to be loaded to RAM on startup. For this purpose, you may use the "RAMCODE" sections which are configured in file MemMap.h.

   Note: If option Pipelined Programming is disabled, it's sufficient to load the functions `FblLookForWatchdog()` and `ApplFblWDTrigger()` into RAM (including any functions or libraries which are called from within these functions).

| Module | Function |
|---|---|
| fbl_wd.c | |
| | FblLookForWatchdog |
| | FblLookForWatchdogVoid |
| fbl_apwd.c | |
| | ApplFblWDTrigger |
| fbl_hw.c | |
| | <whole module: all ROM parts (code + const)> |
| fbl_cw.c | |
| | <whole module: all ROM parts (code + const)> |
| fbl_diag.c | |
| | DiagResetResponseHandling |
| | DiagExRCRResponsePending |
| | FblDiagTxErrorIndication |
| | FblDiagRxStartIndication |
| | FblDiagRxErrorIndication |
| | FblDiagRxIndication |
| | FblDiagTxConfirmation |

| Module | Function |
|---|---|
| | FblDiagRxGetPhysBuffer |
| | FblDiagRxGetFuncBuffer |
| fbl_mem.c | |
| | FblMemRxNotification |
| fbl_ap.c | |
| | ApplFblFatalError (Project State = Integration) |
| fbl_apdi.c | |
| | ApplFblSecuritySeedInit |
| library code | |
| | <all (compiler specific) library code which is used by the above listed modules/functions> |
| MICROSAR communication stack modules (depending on bus type, e.g. CAN, FlexRay) | |
| | <all modules and generated files> |

Table 2-1 Bootloader modules/functions which have to be loaded to RAM

10. Build the bootloader and program it onto the ECU. Test the bootloader by sending a diagnostic request or simply start the download tool. For the first, you can use a download file that just contains dummy data to verify the correct functionality of the bootloader. Such a file can be created by using the tool Hexview which is part of the delivery.
11. After the bootloader has been successfully integrated to perform downloads, the rest of the user functions in the application files (fbl_ap*) have to be reviewed and/or extended as required (see chapter 9.2 Callback Function API for details).
    Special care should be taken for the following items:
    - The ECU specific data identifiers (DIDs) have to be adapted. See chapter 3.4.2 ApplFblReadDataByIdentifier for details.
    - The validation mechanism has to be implemented according to the OEM requirements. See chapter 5 Validation/Invalidation of Flashed Application for details. Special attention should be paid to REQPROD 53957 and REQPROD 53970.
    - Handling of the ECU's I/O ports in the bootloader (fbl_ap.c::ApplFblInit/ApplFblStartup). See REQPROD 53840, REQPROD 53802.
    - Implementation of cold/warm start detection (fbl_apnv.c::ApplFblExtProgRequest). See REQPROD 52254, REQPROD 52255, REQPROD 52252.
    - Implementation of hardware reset (fbl_ap.c::ApplFblReset). See REQPROD 52290, REQPROD 52251.
    - Choose an application start mechanism (fbl_ap.c::ApplFblStartApplication). See REQPROD 53954.

- Verify bus-off implementation (fbl_ap.c::ApplFblCanBusOff). See REQPROD 392111.

12. Adapt your application software to interact with the bootloader. This includes:
    - Adaptation of the application software's memory location
    - Implementation of the complete & compatible function and the corresponding meta data block (including the correct entry point address of the application software).
    - Start of the bootloader from the application software (chapter 6.4 Transitioning from Application to Bootloader)
    - Conversion of the executable file into a VBF container
13. Disable debug support and re-build the bootloader before going to production:
    - Project State = Production (chapter **4.2.18**)

## 2.2 User Specific Requirements

The following table lists all requirements which are not fully implemented for the delivered bootloader. This is usually the case for requirements that are hardware or project (ECU) specific. Please revise these requirements and take the appropriate actions if required.

| User Specific Requirements |
|---|
| General Software Download Specification |
| REQPROD 53838/-;1 : PBL can only write data to volatile memory |
| REQPROD 400961/MAIN;4 : Validity Status Information - Properties |
| REQPROD 366329/MAIN;1 : Public Key storage |
| REQPROD 366327/MAIN;1 : Allocation of the Authenticity Verification function |
| REQPROD 419731/MAIN;1 : Authenticity verification at every start-up |
| REQPROD 53802/-;1 : Power Consumption |
| REQPROD 53816/-;2 : Internal data transfer speed |
| REQPROD 53834/-;4 : Handling of distribution relays |
| REQPROD 53835/-;1 : Calibration and Customer data |
| REQPROD 53837/-;1 : PBL in non-volatile memory |
| REQPROD 53839/-;1 : Protected boot sector |
| REQPROD 53840/-;1 : Pre-condition state of the ECUs IO |
| REQPROD 397292/MAIN;2 : Software parts per logical block |
| REQPROD 397291/MAIN;3 : Logical Block - Properties |
| REQPROD 398372/MAIN;1 : Allocation of the Verification Block Table |
| REQPROD 402292/MAIN;1 : Verification Block Table - Reserve space at application build process |
| REQPROD 400878/MAIN;3 : Erase Identifier – restrictions |

| |
|---|
| REQPROD 400894/MAIN;3 : Request Download - restrictions on number of addressed logical blocks |
| REQPROD 419132/MAIN;0 : Debug tools and ports |
| REQPROD 53935/-;1 : Validation of ECU Software(s) |
| REQPROD 403711/MAIN;2 : Complete and Compatible - Definition of being Complete |
| REQPROD 401157/MAIN;3 : Complete and Compatible - Complete function |
| REQPROD 401140/MAIN;3 : Complete&Compatible - Definition of being Compatible |
| REQPROD 401156/MAIN;2 : Complete and Compatible - Compatible function |
| REQPROD 53957/-;3 : CompleteCompatibleFunction() |
| REQPROD 53958/-;6 : CompleteCompatibleFunction() return value |
| REQPROD 53970/-;3 : Maximum time to perform the Complete&Compatible |
| REQPROD 405051/MAIN;1 : CompleteCompatibleFunction - error handling |
| REQPROD 53975/-;1 : Separate memory sectors |
| General Software Authentication |
| REQPROD 398371 : 1 : Verification Block Table per Software Part |
| REQPROD 420232 : 2 : Software Authentication Test and Development |
| REQPROD 410732 : 0 : Default Signing Method |
| UDS Services |
| REQPROD 409100 : 1 : RequestDownload - dataFormatIdentifier (80) |
| REQPROD 391192 : 1 : ECU start up time |
| REQPROD 74109 : 1 : Allowed diagnostic services |
| REQPROD 74135 : 0 : Response time on the diagnostic service |
| REQPROD 74141 : 0 : If a diagnostic service makes a reset, the response shall be sent before... |
| REQPROD 74147 : 0 : P4Server_max response time for readDataByIdentifier (22) |
| REQPROD 74153 : 0 : P4Server_max response time for writeDataByIdentifier (2E) |
| REQPROD 74159 : 0 : P4Server_max response time for routineControl (31) startRoutine (1), rou... |
| REQPROD 74462 : 0 : Entry conditions for diagnostic service WriteDataByIdentifier (2E) |
| UDS Data |
| REQPROD 67763 : 2 : Development specific data records - implementer specified |
| REQPROD 67945 : 0 : Specification of supported services for each data record |
| REQPROD 67946 : 0 : Data types and classifications |
| REQPROD 68182 : 1 : No read of the Security Constant data record |
| REQPROD 68183 : 1 : One time write of the Security Constant data |
| REQPROD 72161 : 0 : Development Specific routines |
| REQPROD 72166 : 1 : System Supplier Specific routines |
| REQPROD 72214 : 0 : Check Complete & Compatible response time |
| UDS Session |

| |
|---|
| REQPROD 56204/-;1 : P4Server_max equal to P2Server_max |
| UDSonCAN |
| REQPROD 52235 : 1 : RequestDownload Service |
| REQPROD 83301 : 1 : Physical Layer |
| REQPROD 52251 : 1 : Complete hardware initiation when starting the ECU |
| REQPROD 52252 : 1 : Cold or warm start detected |
| REQPROD 52254 : 1 : Cold start detected |
| REQPROD 52255 : 1 : Warm start detected |
| REQPROD 52290 : 1 : Reception of the ECUReset service |
| REQPROD 52293 : 1 : Trig a reset in ECU application mode |
| CAN Data Link |
| REQPROD 53853 : 1 : PLL bypass |
| REQPROD 53861 : 1 : PLL locking |
| REQPROD 53862 : 1 : Resychronisation strategy |
| REQPROD 53863 : 1 : Sample mode |
| REQPROD 53864 : 1 : Bit time set up for 500 kbps |
| REQPROD 53865 : 1 : Bit time set up for 250 kbps |
| REQPROD 53866 : 1 : Bit time set up for 125 kbps |
| REQPROD 53883 : 1 : Voltage hysteresis |
| REQPROD 53884 : 1 : CAN bus level during reset |
| UDSonFR |
| REQPROD 52776/-;1 : DiagnosticSessionControl Service |
| REQPROD 52780/-;1 : RequestDownload Service |
| REQPROD 342637/MAIN;0 : TransferData Service |
| REQPROD 52801/-;1 : Slot IDs for diagnostic requests - receiver |
| REQPROD 52841/-;1 : Complete hardware initiation when starting the ECU |
| REQPROD 52870/-;2 : Complete & Compatible function |
| REQPROD 52875/-;1 : Start the controller with boot schedule, SlotMode=SINGLE |
| REQPROD 52880/-;1 : Reception of the ECUReset service |
| REQPROD 52881/-;1 : S3server timeout |
| REQPROD 52882/-;1 : Reception of DiagnosticSessionControl(defaultSession) |
| REQPROD 52908/-;1 : FlexRay ECU - Stop normal communication |
| REQPROD 52909/-;1 : FlexRay ECU - Enter the bootloader |

Table 2-2 User Specific Requirements

# 3 FBL Software Architecture

## 3.1 [#oem_files] Components

The FBL is a complete application that consists of several modules. The following figure depicts the hierarchy and the dependencies of the modules.

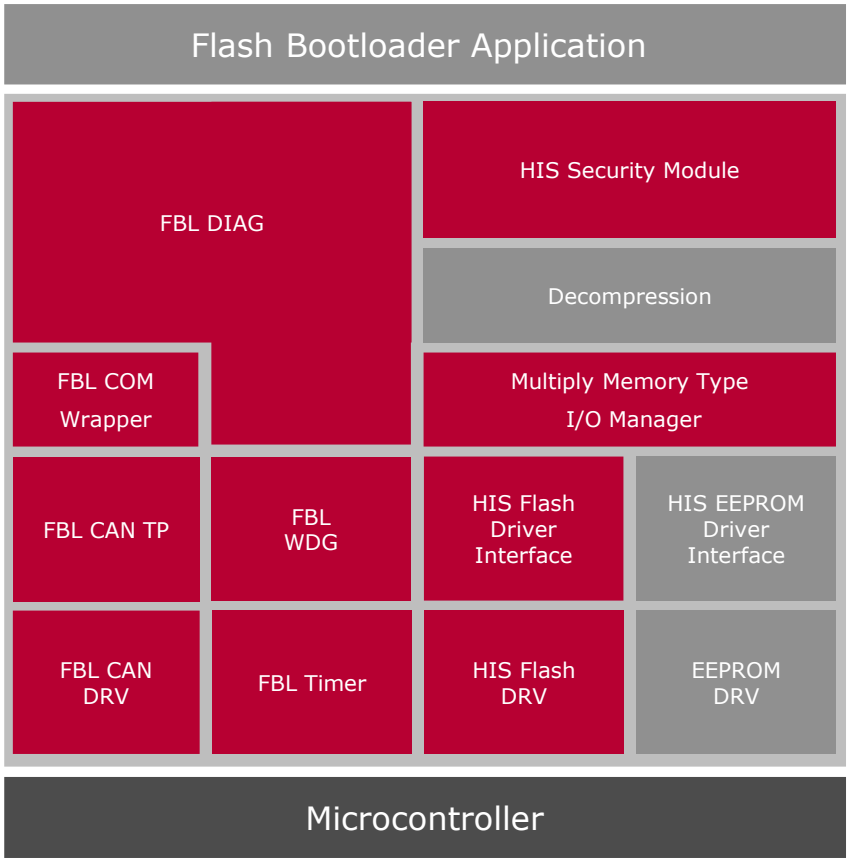



Figure 3-1 Bootloader Module Dependencies

A high-level description of each component follows. The components are grouped in the following categories.

### 3.1.1 Bootloader Core Files

The first group contains components found in the BSW\Fbl folder of your delivery. Do not modify the contents of the files in this group without prior written permission from Vector (modification of these files will void your warranty).

| Component/File | Description |
|---|---|
| applvect.h | Defines, types and declarations which are used by the vector table modules and the hardware module |
| fbl_applvect.c | Instance of the application vector table which is compiled with the bootloader (on certain hardware platforms, this is an assembler file) |
| fbl_can.h | CAN parameter macros |

| Component/File | Description |
|---|---|
| fbl_cw | Communication wrapper (bus system specific) |
| fbl_def.h | Common data types and structure definitions used by the FBL. |
| fbl_diag | Module implements diagnostic service functions. The implementation is specific to requirements defined by VolvoCar (see reference [3]). |
| fbl_flio | Flash I/O routines which provide an interface to the flash driver |
| fbl_hw | Hardware dependent code (CAN communication and hardware timer) |
| fbl_lbt_access | Abstraction layer for accessing the Logical Block Table (LBT) |
| fbl_main | Main module with main loop and task calls |
| fbl_mem | Library module for memory specific tasks |
| fbl_mio | General purpose API to device-drivers (MIO == Memory Input/Output) |
| fbl_nvpattern | Library module which is used for block validity and erased state handling |
| fbl_ramio | RAM driver which is used for downloading the SBL |
| fbl_tp | Transport Layer – Combines (and splits) diagnostics requests (and responses) across multiple CAN message frames. |
| fbl_valstruct | Library module for validation structure handling |
| fbl_wd | Watchdog support module |
| iotypes.h | Type definitions used by the memory Input/Output component |
| v_def.h | ECU and compiler-specific type-definitions and macros used to abstract fundamental data types, pointers, and subroutine declarations. |
| v_ver.h | Version information of all delivered/licensed components. |

Table 3-1 Bootloader Core Files

### 3.1.2 Bootloader Application Files

The second group consists of components that must be customized for your hardware and application. The files may be found in the BSW\Fbl_Template folder. You should copy these files to your bootloader project folder, and rename them, removing the leading underscore from the filename.

| Component/File | Description |
|---|---|
| _applvect.c | Instance of the application vector table which is compiler with the application software |
| _fbl_vect | FBL vector table – defines all interrupt service vectors (on certain hardware platforms, this is an assembler file). Usually, you can use this module without further adaptations. |
| _fbl_ap | Hardware specific callback routines |
| _fbl_apdi | Application specific diagnostic routines |
| _fbl_apnv | Nonvolatile memory access routines, e.g. for presence-pattern handling. |
| _fbl_apwd | Application specific watchdog routines |
| _fbl_inc.h | Include file for including all FBL related include files |
| _comdat.h | Common data structures which are used by bootloader and application |

| Component/File | Description |
|---|---|
| | (e.g. diagnostic data identifier values). |
| _fbl_ramio_cfg.h | Configuration file of RAM driver (usually no changes required) |

Table 3-2 Bootloader Application Files

### 3.1.3 Generated Files

The third set of components is generated by the configuration tool, DaVinci Configurator. You should not modify these files by hand.

| Component/File | Description |
|---|---|
| Fbl_Fbt.c | Contains the flash block table. The table defines the regions of memory that may be programmed by the bootloader. |
| Fbl_Fbt.h | Flash block table structure definitions |
| Fbl_Cfg.h | Bootloader feature switches and parameters. |
| Fbl_Lbt.c | Contains the module table. This table defines the logical partitioning of the available memory as defined in the flash block table. |
| Fbl_Lbt.h | Module table definitions, types and macros |
| Fbl_Lbt_PBcfg.c | Reprogrammable instance of the Logical Block Table |
| SecM_cfg.h | Security module configuration file |
| SecMPar.c | Configuration parameters of security module |
| SecMPar.h | Configuration parameters of security module |
| v_cfg.h | Hardware and compiler specific switches and parameters. |
| *_Cfg.h<br>*_Lcfg.c/*_Lcfg.h<br>*_PBcfg.c/*_PBcfg.h | Configuration files of the MICROSAR communication components (e.g. Can, CanIf, CanTp, PduR, etc.). For further information, please consult the respective TechnicalReference document. |

Table 3-3 Generated Files

### 3.1.4 Decompression Module Files

The decompression module is a mandatory add-on for the VolvoCar bootloader. All decompression related files are stored in delivery folder ‘BSW\Cmpr’.

| Component/File | Description |
|---|---|
| cmpr | Dedicated decompression module (bootloader add-on) |

Table 3-4 Decompression Module Files

### 3.1.5 Security Related Files

The security related files are located in folder ‘BSW\SecMod’. The bootloader uses the security module for security access (seed/key) and verification (hash calculation and signature support).

### 3.1.6 Flash Driver Files

The flash driver (see chapter 7) which is located in folder 'BSW\Flash' is built as a separate project. The resulting executable has to be converted to the VBF format so it can be downloaded during bootloader runtime.

| Component/File | Description |
|---|---|
| flashdrv.c | Flash driver main module. Depending on the supported hardware platform, there might be additional source files which have to be compiled (refer to the hardware documentation and the delivered make files in folder 'Build'). |
| flashdrv.h | Flash driver header with hardware/device specific (type) definitions. This file is included by the flash driver main module as well as the bootloader itself. |
| FlashDrv.vbf | Pre-built flash driver container which has been used during the bootloader tests. Note that it's very likely that you have to re-build the container to adapt the VBF attributes to your project (e.g. ecu_address). |
| \Build (folder) | This folder contains the make environment for the flash driver. Usually, you won't have to re-build the driver – but if you do, please use the provided make environment. |

Table 3-5 Flash Driver Files

### 1.1.1 MICROSAR Communication Stack Files

Usually, the bootloader is delivered in combination with a Vector MICROSAR communication stack, so there will be additional static and generated files. Please add these files to the build environment, too. For a detailed description of these files, please refer to the respective technical reference documentation.



**Example**
The following table contains a list of subfolders beneath folder BSW, which might be present if you receive a bootloader with MICROSAR components.

| Folder | Description |
|---|---|
| _Common | Common type definitions |
| Can | MICROSAR CAN driver |
| CanIf | MICROSAR CAN interface |
| CanTp | MICROSAR transport protocol implementation |
| Cmpr | Bootloader Decompression module |
| ComM | MICROSAR Communication Manager |
| Det | MICROSAR Development Error Tracer |
| Fbl | Bootloader core files |
| FblAsrStubs | Bootloader specific stub files for the MICROSAR communication stack |

| Folder | Description |
|---|---|
| FblFrTrcv | Bootloader specific FlexRay transceiver driver stub |
| Flash | Bootloader memory driver for reprogramming of the code flash memory (this component is built separately, see chapter 7) |
| Fr | MICROSAR FlexRay driver |
| FrIf | MICROSAR FlexRay interface |
| FrSM | MICROSAR FlexRay State Manager |
| FrTrcv_* | MICROSAR FlexRay transceiver driver |
| PduR | MICROSAR PDU Router |
| SecMod | Bootloader Security Module |
| Tp_Iso10681 | MICROSAR FlexRay transport protocol implementation |
| VStdLib | Vector specific library with commonly used functions |

Table 3-6 FlexRay Bootloader with MICROSAR Components (Example)

### 1.1.2 MICROSAR Stub Files

As the bootloader does not include certain MICROSAR components which are referenced by other components, e.g. SchM, some functionality has to be stubbed.

These stub files can be found in BSW\FblAsrStubs. Please include them into your build environment.

## 3.2 Startup

The FBL is operated in the context shown in the diagram. After reset/power-on, the startup code of the FBL is executed. The startup code calls the FBL after the basic (hardware-dependent) initializations have been performed. After startup, the bootloader is in the programming session. The FBL determines if a valid, executable application is available. If this is true, the application is started, otherwise, the software stays in boot mode. When receiving a DiagnosticSessionControl request to enter programming session, the ECU shall change diagnostic sessions and send back a positive response. Being in programming session is necessary before any programming services may be requested. It's possible to enter programming session from the application through a reset or by jumping directly to the bootloader. Either approach may be implemented based on the configuration of the "FblStart Function" option. For more details, please refer to section 6.4.

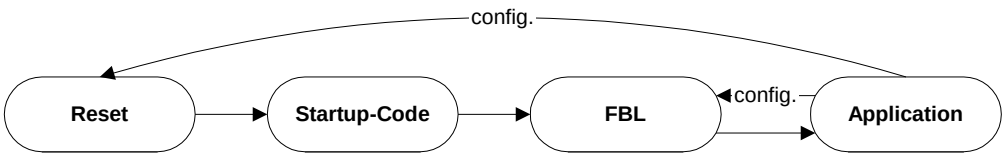



Figure 3-2 Interaction of FBL and Application

## 3.3 Download Flow

Details about the download flow of the VolvoCar FBL can be found in section 2.3 of [3].

During a typical download process, the flash bootloader will call diagnostic service handlers for each respective diagnostic service. Consequently, a diagnostic service function may call a user-callback function to allow the user to handle an ECU specific operation.

Below are the basic diagnostic functions called during a typical download process:

### 3.3.1 DiagnosticSessionControl

A tester sends DiagnosticSessionControl ($10) with the diagnosticSessionType set to $02 to put the ECU in programming session. When received in the application, the application will handle the response and transition to programming mode. For all other cases, handling of this service is handled internally by the flash bootloader.

### 3.3.2 ReadDataByIdentifier

After entering programming mode, the bootloader is expected to respond to ReadDataByIdentifier ($22) requests to read specific DIDs. Handling of these DIDs is done through the user callback function: ApplFblReadDataByIdentifier().

### 3.3.3 Security Access

Following the read of the DIDs, the tester will attempt to unlock the ECU to prepare for flash operations. SecurityAccess ($27) is supported by default in the VolvoCar Flash bootloader. The user can modify how the seed and key are calculated through the user callback functions: ApplFblSecuritySeed() and ApplFblSecurityKey().

For more info, please refer to chapter 8 The Security Module for details on the use and configuration of the security module.

### 3.3.4 Download Secondary Bootloader

Following the unlocking of the flash bootloader, the Tester downloads the SBL via services: Request Download ($34), TransferData ($36), and TransferDataExit ($37). The download of the SBL to RAM is handled internally by the flash bootloader and no user-callbacks are available at this stage.

### 3.3.5 Activate Secondary Bootloader

Following the download of the SBL, the Tester will try to activate the SBL, via a RoutineControl ($31) subroutine, called ActivateSBL ($0301). The Diagnostic Layer calls the initialization function of the SBL to activate it. If the SBL is downloaded correctly, then the activation is usually successful.

### 3.3.6 Erase Flash Memory

Following the Activation of the SBL, the Tester will attempt to erase regions defined in the .vbf file, via the RoutineControl ($31) subroutine, called EraseRoutine ($FF00). As long as the requested regions are defined in the bootloader's logical block table, the erase should succeed.

### 3.3.7 Download Application/Calibration Files

Following the erase, the Tester will start downloading data using the services ($34, $36, and $37). This is handled internally in the flash bootloader and no user-callbacks are available at this stage.

### 3.3.8 Check Memory Routine

At the end of each download block (i.e. logical block), the tester initiates the integrity/authenticity verification by sending a RoutineControl/CheckMemory request ($0212). This request includes a signature which is used by the bootloader to perform the verification. If the verification is performed successfully, the validity information for this block will be persisted in non-volatile memory.

### 3.3.9 Check Complete & Compatible Routine

After the download of all data blocks is complete, the tester attempts to validate the downloaded data via a RoutineControl subroutine, called Check Complete and Compatible ($0205).

The validation handling of download modules is necessary for the FBL to determine if a valid application is programmed so that it can be started after reset. For this purpose, four user adaptable callback functions are provided:

- ApplFblIsValidApp
- ApplFblInvalidateBlock
- ApplFblValidateBlock
- ApplFblValidateApp

See chapter 5 Validation / Invalidation of Flashed Application and chapter 9.2 Callback-Function API for more details.

### 3.3.10 EcuReset

Upon validating the downloaded files, the tester will send an EcuReset ($11) request to reset the ECU. After the reset, the bootloader can detect that a valid application exists and will then start running the application.

## 3.4 User Callback Functions

The following is a list of user callback functions that require a VolvoCar-specific implementation. The API description can be found in chapter 9 API Description. For details on all the user callback functions, refer to [9].



**Caution**
All user callback functions provided in the bootloader demo are for demonstration purposes only. You MUST review the default implementation and adapt it based on your requirements and hardware setup.

### 3.4.1 ApplFblWriteDataByIdentifier

If the ECU is required to support the diagnostic service $2E, then the callback function ApplFblWriteDataByIdentifier shall be used to handle the incoming DID requests. Prior to using this service, option “WriteDataByIdentifier Service” (chapter 4.6.7) has to be enabled in the configuration tool.

It is part of the user configurable code, where the user checks if the DID is supported, if the message length of the corresponding DID is correct, and if the SBL is activated prior to

programming a DID. In case of detecting an error in any of these checks, the user shall issue the corresponding error using the DiagNRCxx macros. A list of these macros along with the corresponding NRCs can be found in fbl_diag.h. In addition to that the user shall return kFblFailed in case of an error and kFblOk in case of a successful operation.

### 3.4.2 ApplFblReadDataByIdentifier

This function can be used to implement so-called “complex” DIDs. “Complex” means the data which shall be reported by the DID is not a constant value. Constant values can be added or modified by adapting the `fblDidTable[]` which can be found at the top of fbl_apdi.c. Each table entry consists of three members:

- DID number (e.g. 0xF121 for the Primary Bootloader Diagnostic Database Part Number)
- DID data size in byte (without service ID and DID number; e.g. 7 for 0xF121)
- Pointer to the constant DID data. If the data shall be shared between bootloader and application, it’s recommended to point to the Common Data Structure.

If a complex DID shall be reported, you still have to add DID number and data size to the table. However, if you enter the define `kDiagComplexDidData` instead of a pointer to the data, the diagnostic core implementation will call user function fbl_apdi.c::ApplFblReadDataByIdentifier.

The current implementation of this callback function illustrates how to report a complex DID with the bootloader. The user needs to check for which DID the function has been called. If it’s called for a supported complex DID, the pbDiagData buffer has to be loaded with the response data and the returnCode is set to kFblOk. The data length will be set automatically based on the value in the table.



**Caution**
You shouldn’t set any NRC (using the provided DiagNRC* macros) in the callback function. Returning kFblFailed in case of failure is sufficient.

# 4 Configuration of the VolvoCar Flash Bootloader

The following section describes how the bootloader is configured through the configuration tool DaVinci Configurator. A list of generated files and the respective description can be found in chapter 3.1.3 Generated Files.

To configure the bootloader, a DaVinci configuration is needed. The bootloader delivery contains an example of such a configuration in folder Demo\DemoFbl (usually named DemoFbl.dpa). It's recommended to start a new configuration for your bootloader using the latest system description that is available.

The following chapters provide a list of all configurable bootloader options, together with a short description.

## 4.1 General Configuration Setup

This chapter describes the general setup of a DaVinci configuration.

After DaVinci Configurator has been started, create a new project by selecting **File | New…**. The following dialog allows you to specify project name and location.

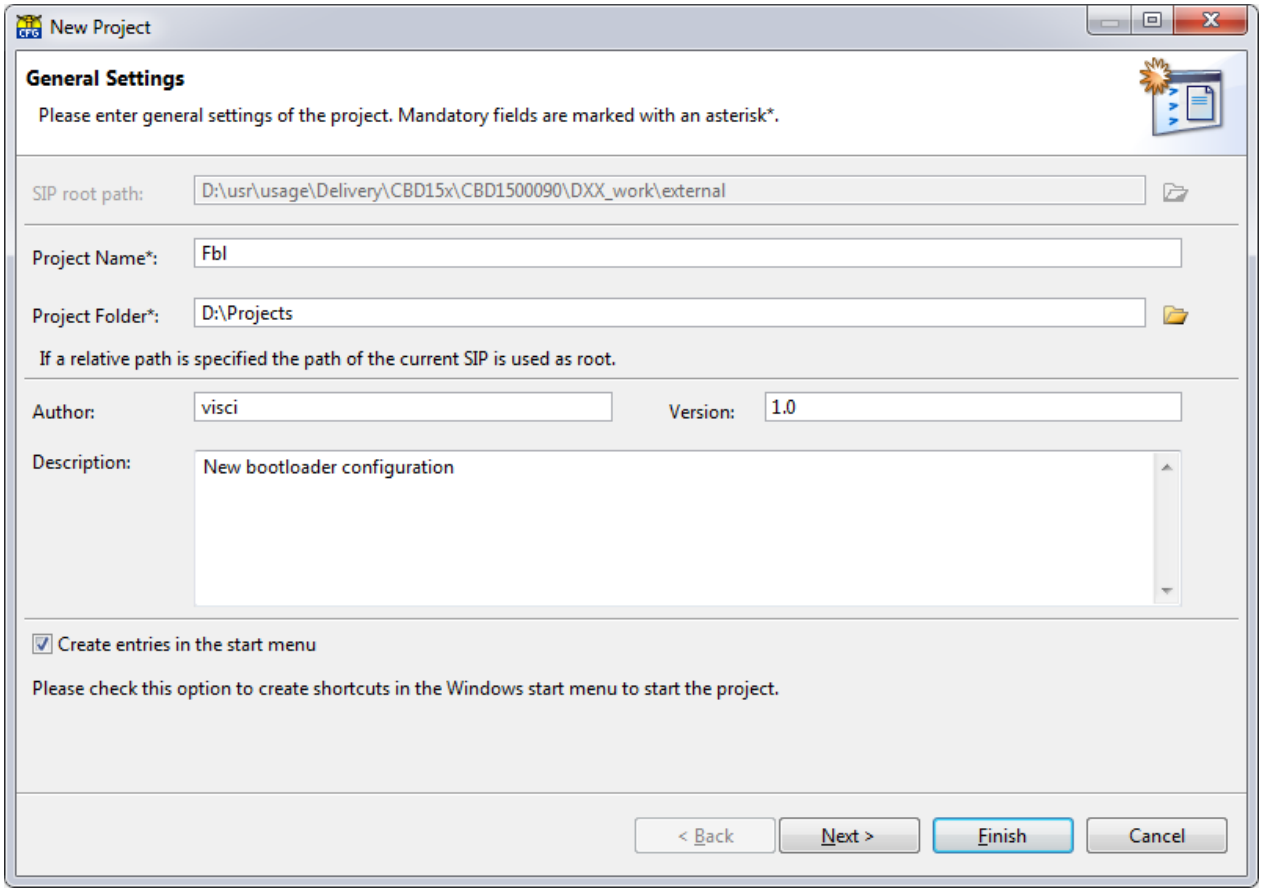



Figure 4-1 DaVinci Configurator: General Settings

The next dialog expects the selection of target type (always "Real Target" for bootloader), derivative, used compiler and pin layout. Please select the applicable values based on

your development environment. The available options may be pre-filtered according to the acquired bootloader license.

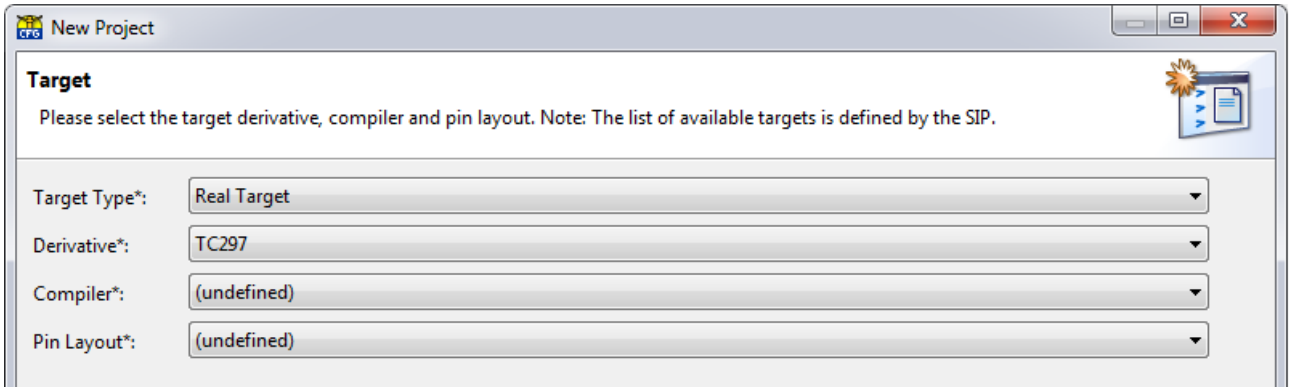



Figure 4-2 DaVinci Configurator: Target Settings

After clicking on the **[Next]** button, the dialog **Project Folder Structure** appears that shows the folder paths which will be used for input and output data. You can choose to store all configuration settings in a single file (default) or in one file per module (sometimes required by 3rd party generators).

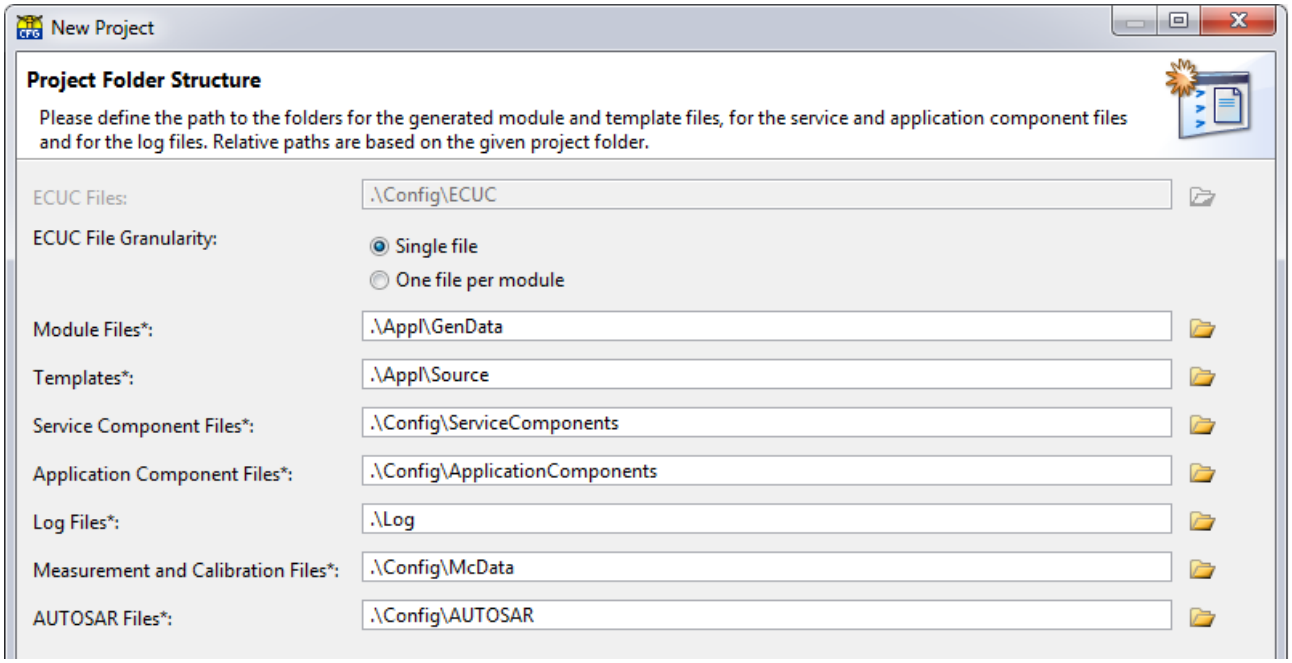



Figure 4-3 DaVinci Configurator: Project Folder Structure

The following dialog provides the option to create a DaVinci Developer workspace – this is not required for bootloader projects. Select **[Finish]** to finally create the new project.

Before you can start the actual configuration process, you have to add at least one system description file in dialog **Input Files**. To open the dialog, please choose menu option **Project | Input Files**. Besides AUTOSAR system description files, you can also add

legacy database files (DBC, LDF, etc.). To open the **File selection** dialog, click on one of the available buttons.

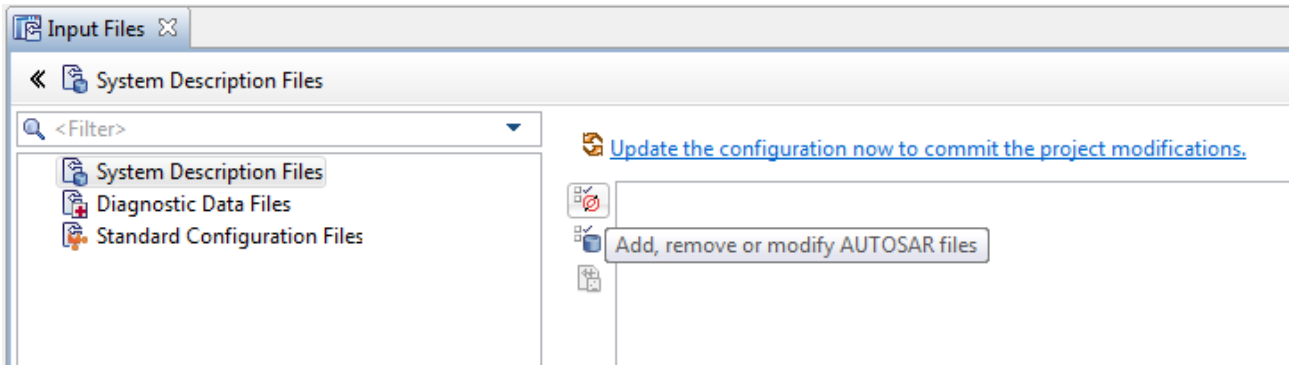



Figure 4-4 DaVinci Configurator: Input Files

After adding one or more input files, the next step allows you to choose the ECU instance.

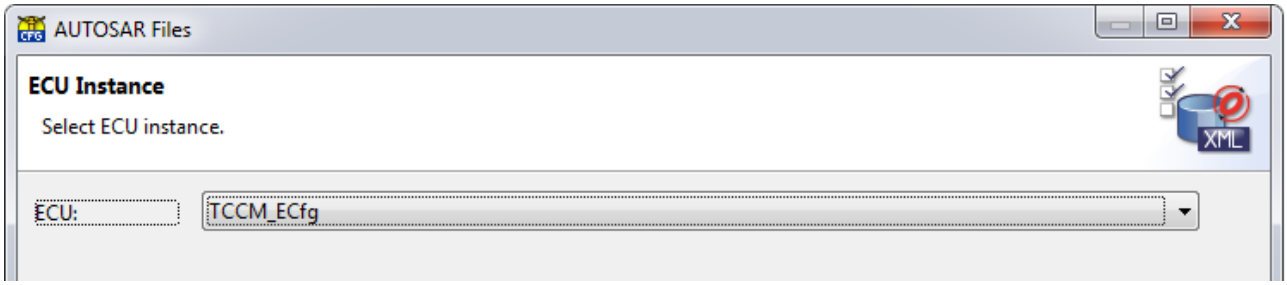



Figure 4-5 DaVinci Configurator: Selection of ECU Instance

After closing the dialog with button **[Finish]**, you have to apply the input data changes by clicking on the “Update Configuration” link:

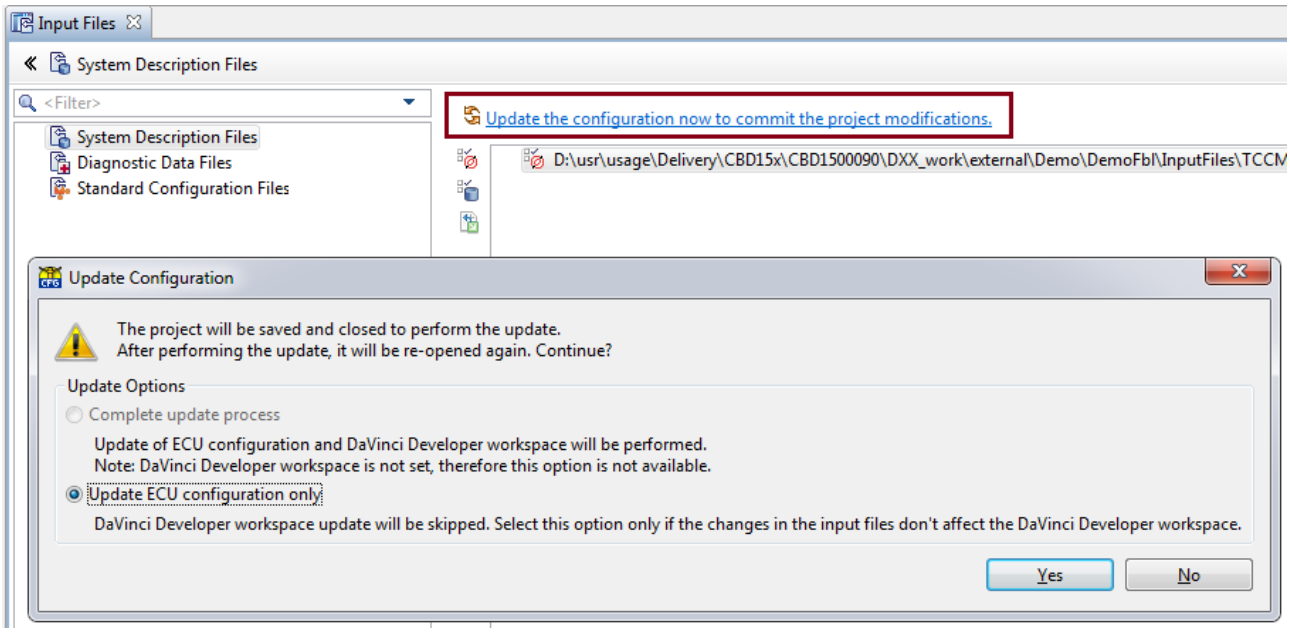



Figure 4-6 DaVinci Configurator: Update Configuration

After the project has been saved and re-opened, please open the Project Settings Editor (menu item **Project | Project Settings**) and add the modules which are required to configure the bootloader.

The following communication modules are relevant for bootloaders which are using the MICROSAR 4 communication Stack:

- Common:
  - Det
  - EcuC
  - PduR
- CAN:
  - Can
  - CanIf
  - CanTp
- FlexRay:
  - ComM
  - Fr
  - FrIf
  - FrSM
  - FrTp

- FrTrcv

In addition to the communication modules, the bootloader specific modules have to be added by using the **[+]** Button.

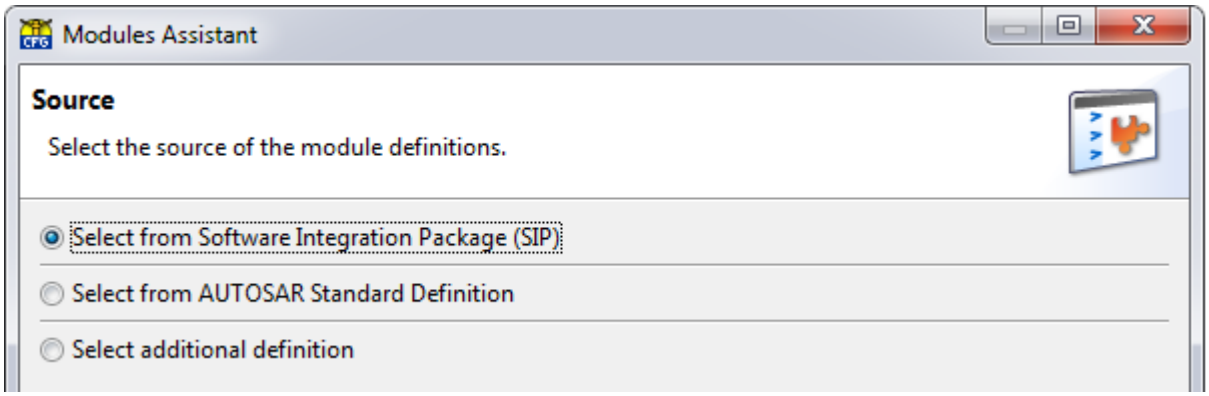



Figure 4-7 DaVinci Configurator: Add FBL Modules (1)

Add the following module definitions:

- Cdd (Communication Wrapper)
- Fbl (Flash Bootloader)
- Smh (Security Module HIS)

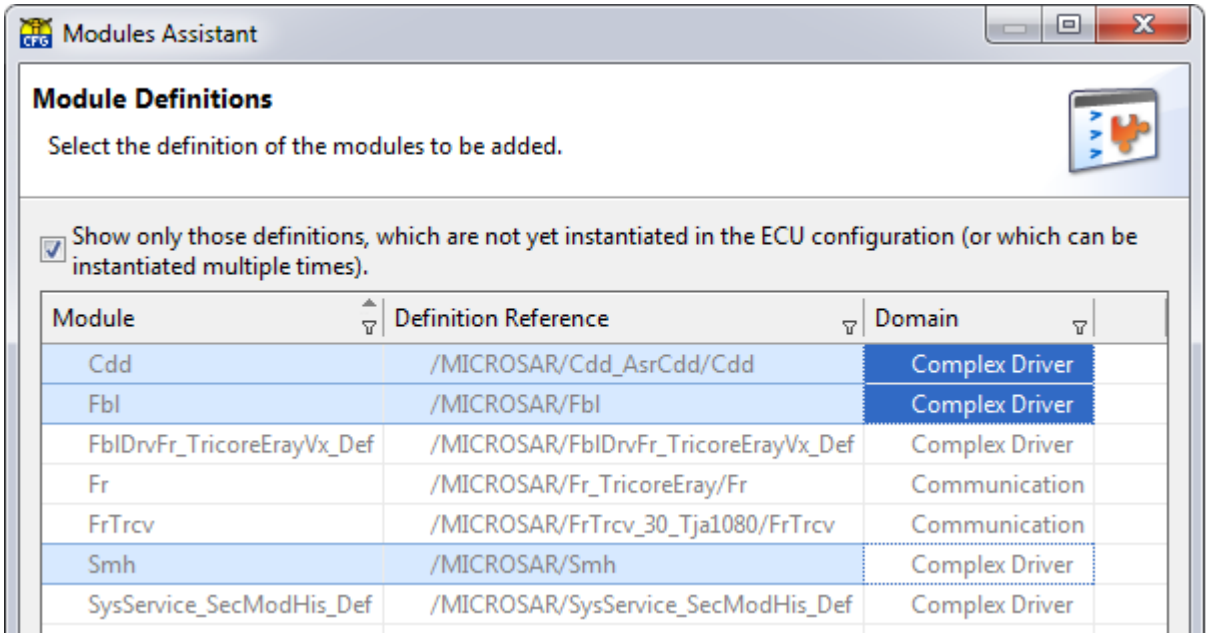



Figure 4-8 DaVinci Configurator: Add FBL Modules (2)

In the next step, please change the module short name of the Cdd module to „FblCw“.

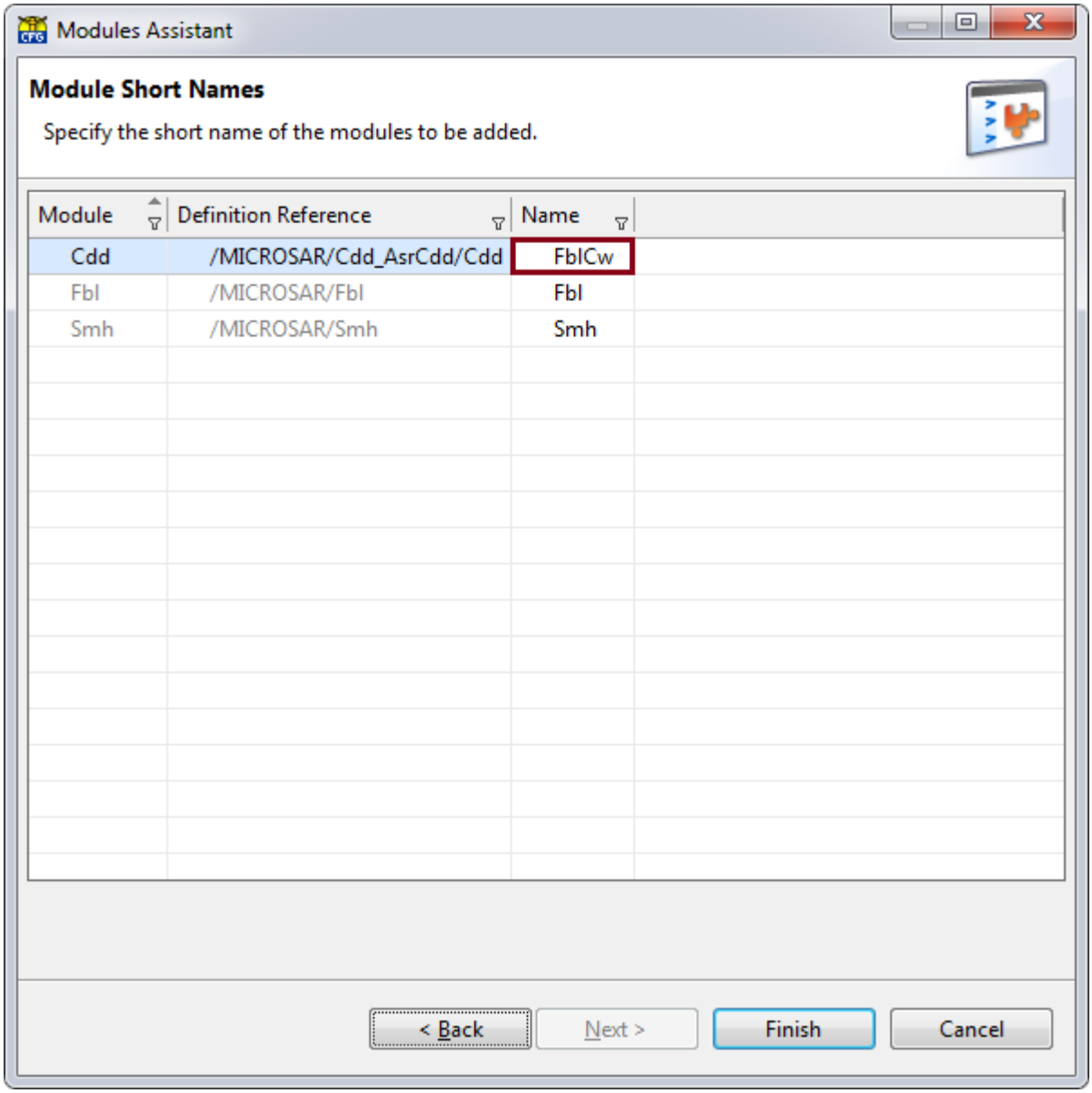



Figure 4-9 DaVinci Configurator: Module Short Names

The communication modules have dependencies to AUTOSAR modules which are not included in the bootloader delivery. To resolve these dependencies, those dummy modules have to be added to the project.

This time the **Select from AUTOSAR Standard Definition** is used in the assistant (**Project | Project Settings | Modules | Add**) to add the **Mcu** and **Os** module definitions.

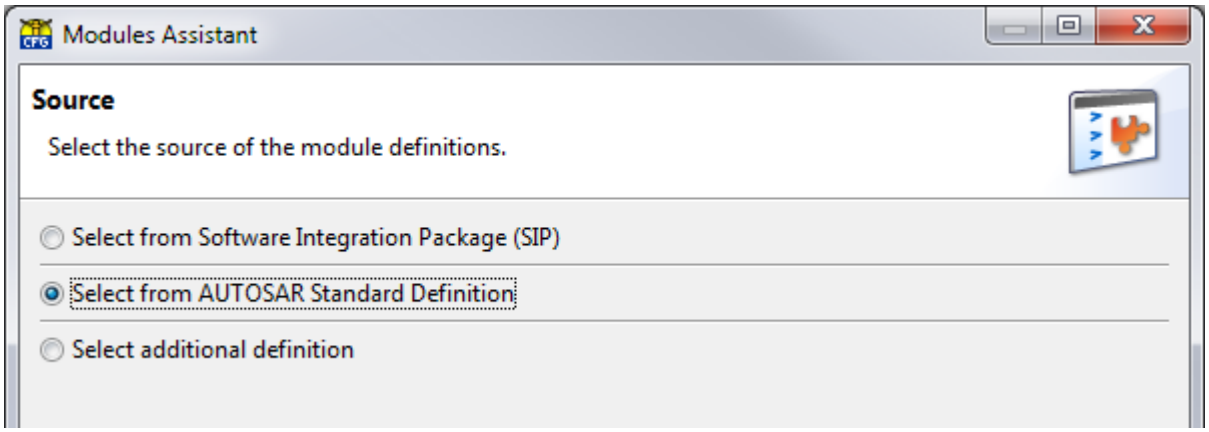



Figure 4-10 DaVinci Configurator: Add Dummy Modules (1)

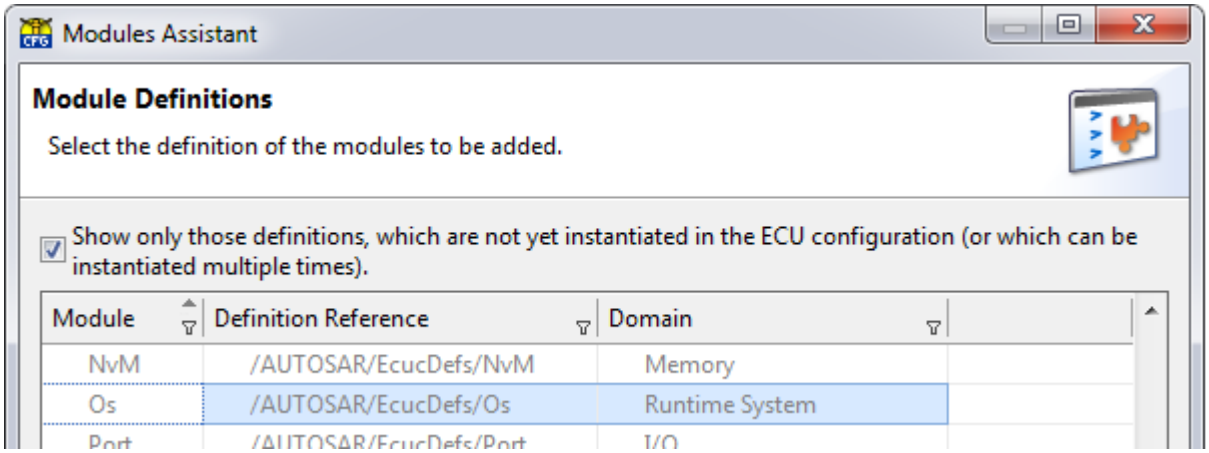



Figure 4-11 DaVinci Configurator: Add Dummy Modules (2)

Now you can start with the actual bootloader configuration. All configuration is performed in the “Basic Editor” which can be opened by using menu option **Project | Basic Editor**.

**Reference**
The following chapters focus on the configuration of the bootloader specific modules. The configuration of the MICROSAR communication modules is described within the module specific user documentation.

## 4.2 Module Fbl | FblGeneral

The general bootloader configuration is located in module **Fbl | FblGeneral**.

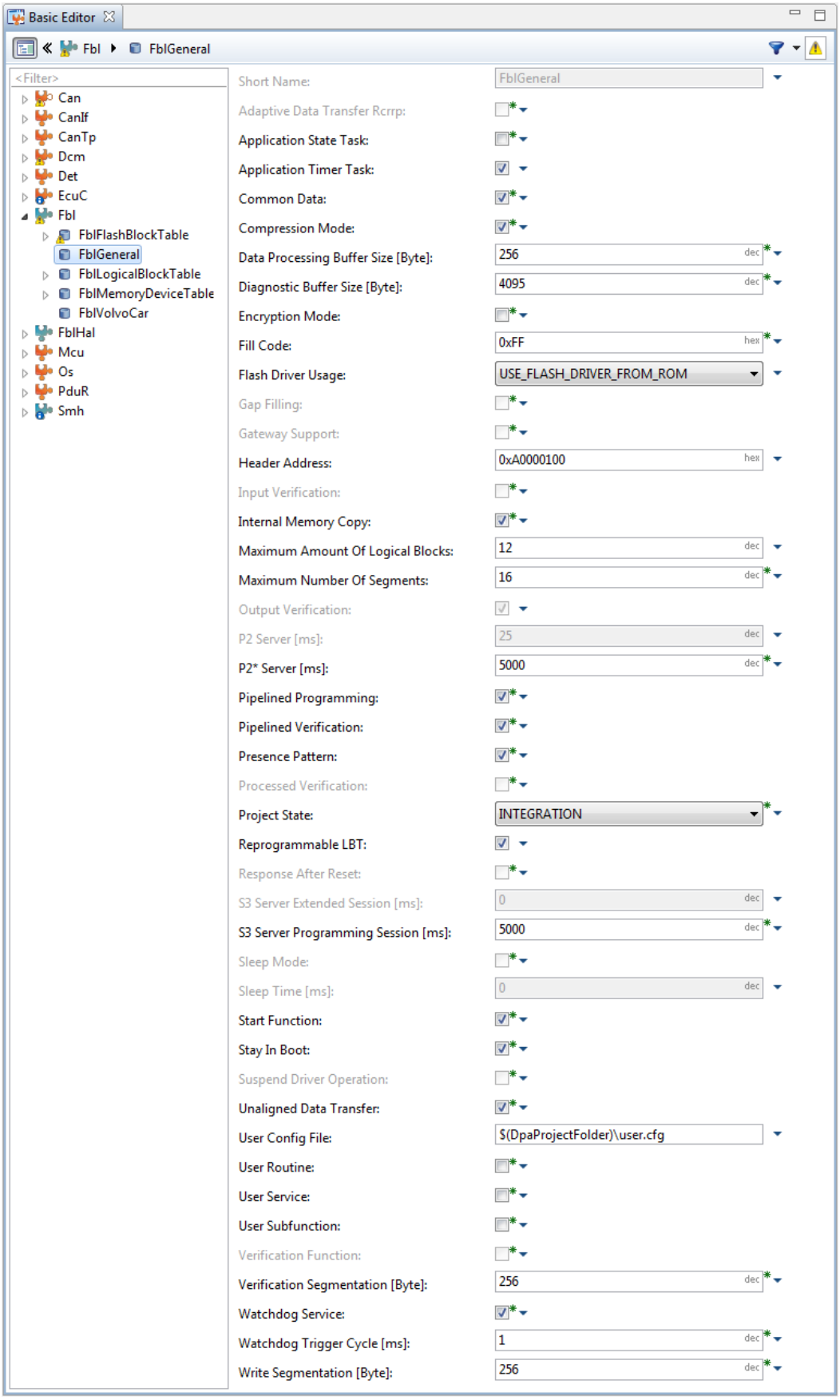



Figure 4-12 DaVinci Configurator: FblGeneral Configuration

Most of the values are already pre-configured with reasonable default values or disabled. The latter is either because they are not relevant for the VolvoCar bootloader at all or not available in the current delivery.

A description of the available configuration items is provided in window **Properties | Description**. For additional information, please refer to the description of the respective configuration items in the sub chapters below.

### 4.2.1 Application Task

Enable application callback function fbl_ap.c::`ApplFblTask()` which is cyclically called by the bootloader. The function is executed in the context of the diagnostic task which runs every millisecond.



**Caution**
As function `ApplFblTask()` is executed from the diagnostic task context, it will not be called while a diagnostic service is processed. I.e. there might be large gaps between individual calls e.g. during flash erasure or checksum calculation.

Consequently, this function is not suitable for any kind of watchdog handling. Use fbl_apwd.c::`ApplFblWDTrigger()` instead.

### 4.2.2 Common Data

By enabling this option, the bootloader provides an interface to access constant data which is linked with the bootloader. This enables the application software to read e.g. DID values from the same memory location as the bootloader. In particular, this is useful for DIDs that report hardware specific information which doesn't change if the application software is updated.

More information about the usage of common data can be found in chapter 6.3 Common Data Structure.

### 4.2.3 Compression Mode

Enabling this option allows for reception of compressed data. The data which is received by the bootloader will be decompressed by means of a dedicated compression module.



**Note**
For VolvoCar bootloaders, the decompression module is usually a mandatory part of the delivery and support for compressed downloads has to be enabled to meet the requirements.

### 4.2.4 Data Processing Buffer Size

This option defines the maximum output buffer size available for handling of preprocessed data, e.g. decryption or decompression.

### 4.2.5 Diagnostic Buffer Size

This is the size of the RAM buffer (in bytes) which is used for the reception of diagnostic request messages. Usually, a bigger buffer provides a better flash programming

performance because transport protocol overhead is reduced and more data can be programmed in a single step.

Volvo specifies the following minimum buffer sizes:

- CAN (REQPROD 52235): 2 kB
- FlexRay (REQPROD 52780): 16 kB



**Note**
If option Unaligned Data Transfer is disabled, the actually used buffer size which is reported in the RequestDownload/-Upload response might differ from the configured value because of the required alignment at the memory specific write segment size.

### 4.2.6 Encryption Mode

Enabling this option allows for reception of encrypted data. The data which is received by the bootloader will be decrypted by means of a dedicated decryption module.



**Note**
Decryption is an optional feature which has to be ordered explicitly.

### 4.2.7 Fill Code

The defined byte value will be used to pad buffer content which is not aligned to the flash segment size. It is recommended to choose the flash erased value for this option (hardware dependent, in most cases: 0xFF).

### 4.2.8 Header Address

Start address of the FBL header structure which is linked with the bootloader. The FBL header contains the following information:

> Address of the `FblStart()` function (if option “FblStart Function” is enabled)

> FBL Version Information

> Pointer to Common Data Structure

The location of the FBL header structure can be influenced via the named section that is provided by the bootloader (the actual name is defined in MemMap.h, in most cases it will be something like “FBLHEADER”).



**Caution**
Remember to update the copy of Fbl_Cfg.h which is used in the application software if the address of FblHeader changes!

### 4.2.9 Internal Memory Copy

The availability of this option depends on the used hardware platform. Activating this configuration option will use an internal MEMCPY function of the bootloader instead of using the library memcpy function of the compiler vendor.

Usually, this option will be pre-configured as enabled (if available). If this option is disabled, there might be issues with the accuracy of the watchdog trigger cycle.

### 4.2.10 Maximum Amount Of Logical Blocks

This is the maximum number of logical blocks which can be handled by the bootloader. For the configuration of this value, please consider potential growth of the LBT in the future. After the bootloader has been programmed into the ECU, it's not possible to download an LBT where the number of blocks exceeds the configured value.

The parameter is only considered if option Reprogrammable LBT is enabled.

### 4.2.11 Maximum Number Of Segments

This value defines the size of the RAM table which is used by the bootloader to keep track of the download segments which go into a single logical block. Each download can be separated into several segments caused by data fragmentation (e.g. alignment gaps).

The absolute minimum number of download segments for a single block is 2. This corresponds to a single data segment plus the validation structure.



**Caution**
If the actual amount of download/erase regions exceeds the configured value, the bootloader will abort the download/erase operation with NRC 0x31. Therefore, you should always choose a value with a certain reserve.

### 4.2.12 Output Verification

If enabled, the verification (i.e. the calculation of the block hash) will take the data written to non-volatile memory (e.g. flash) as input.

The processing will be performed on the written contents, after the complete data was transferred and programmed.

### 4.2.13 P2 Server

Specify the P2 diagnostic timeout here. This value is pre-configured and should not be changed unless explicit approval by VolvoCar.

The FBL will always send a response, before P2 expires. In case an RCRRP message is sent, the enhanced timeout P2* Time is activated.

Note that the P2 time which is specified by VolvoCar differs between the programming session (25 ms) and all other sessions (50 ms).

### 4.2.14 P2* Server

Specify the P2* diagnostic timeout here. This value is pre-configured and should not be changed unless explicit approval by VolvoCar.

The bootloader will re-send RCRRP messages while processing long-lasting service requests after P2*/2 has passed.

### 4.2.15 Pipelined Programming

With this feature enabled, the FBL immediately sends a positive response after the reception of a TransferData request, so that the next data transfer can be performed while the previous data block is programmed. The same parallelization applies to the decompression/decryption of the received data. In best case only the pure transmission time is required.



**Reference**
The feature “Pipelined Programming” is Vector’s implementation of REQPROD 53978 (“Writing of received data”). See chapter Deviations for details.

### 4.2.16 Pipelined Verification

If enabled, the verification (i.e. the calculation of the block hash) will take the data written to non-volatile memory (e.g. flash) as input.

The processing will be performed on the already written contents, while the reception of further download data is still ongoing. Reading back the memory and calculating the hash is performed in parallel to the reception. This can significantly reduce the verification time at the end of the block download.

### 4.2.17 Presence Pattern

By default, the bootloader uses so-called presence patterns to store the validity status of the logical blocks. If this option is enabled, the decision if the application software can be started can be made without loading a memory driver (e.g. EEPROM Emulation or external EEPROM).

If a logical block is determined as valid (by the implementation provided in fbl_apnv.c::`ApplFblValidateBlock()`), the presence pattern will be written to the flash memory. During system startup, the patterns of all logical blocks are checked by function fbl_apnv.c::`ApplFblIsValidApp()`. If the patterns of all mandatory blocks are set, the application will be started. Please have a look at chapter 5.7 Presence Pattern for further details.

#### Presence Pattern Location and Size

Each configured logical block has its own presence pattern area which is always located at the end of the logical block. The presence pattern area consists of the pattern and the corresponding mask which both have the same size. The actual size depends on the flash write segment size of the microcontroller:

> If the flash write segment size is less or equal than 4 byte, presence pattern and mask will each have a size of 4 byte.

> In any other case, the effective presence pattern size will equal the flash write segment size.



**Reference**
You can find the flash write segment size of the used memory devices in the Memory Device Table which is described in chapter 4.3

**Preconditions for Presence Pattern Usage**

> Enabling DaVinci option “Presence Pattern” implies the usage of the logical block table (Fbl_Lbt). Take care that this table is configured appropriately.

> The usage of presence patterns requires that the whole logical block is erased during the download. Otherwise, the presence pattern cannot be written.

> The bootloader will protect the presence pattern areas against intended or unintended overwrite. Take care that application code or data doesn’t overlap with these areas.



**Caution**
The presence pattern implementation is provided as an example and may not exactly fit your use case. E.g. if you are able to access an EEPROM from within the bootloader, it might be more convenient to store the validity information there.

If you decide to use the provided implementation of the presence pattern mechanism, it has to be carefully reviewed to verify the correctness for the intended purpose.

### 4.2.18 Project State

This option allows enabling or disabling certain code parts which are useful during the development/integration phase of the bootloader:

> Integration: Choose this state during development/integration of the bootloader. This configuration adds internal configuration and user code checks and hence helps you to find configuration errors easily. Errors found will trigger the user function fbl_ap.c::`ApplFblFatalError()`. A user specific error handling can be added, e.g. a flashing LED if no debugger is available.



**Info**
Please consult chapter 9.4.1 Bootloader Assertions for further details.

> Production (default): Choose this state when you are ready for production. Configuration checks are turned off.

> Production (reduced checks): Choose this state when ready for production and if you need to minimize code size. Some additional checks will also be disabled in order to reduce code size. The resulting configuration might report NRCs on runtime errors at a

later point in time. This will make it more difficult to analyze errors that occur on production ECUs. It is therefore recommended to configure "Production (default)" if possible.



**Caution**
It's highly recommended to use setting 'Integration' during the ECU development phase to catch configuration errors like e.g. insufficient watchdog or flash code buffer size. But don't forget to switch to setting 'Production' when you are finished (this will save runtime and code size)!

### 4.2.19 Reprogrammable LBT

Enable support for post-build update of the Logical Block Table. If this option is enabled, the generated Fbl_Lbt.c/Fbl_Lbt.h files will only contain the SBL and LBT blocks. The full version of the LBT will be generated into the file Fbl_Lbt_PBcfg.c.

See chapter 9.4.7 Reprogrammable LBT for implementation details.

### 4.2.20 S3 Server Programming Session

This attribute defines the S3 diagnostic timeout for the programming session. This timeout is used to abort the diagnostic programming session (by performing an ECU reset) if no diagnostic request message has been received within the configured time frame.

### 4.2.21 Start Function

This option provides a user-friendly way to start the bootloader from within the application.

If enabled, the bootloader provides the function fbl_main.c::`FblStart()` which will be called on execution of macro `CallFblStart()`.

**Preconditions**:

> The FBL Header Address is configured correctly

> The application module which uses `CallFblStart()` includes fbl_def.h (Fbl_Cfg.h has to be provided, too).



**Caution**
Remember to update the copy of Fbl_Cfg.h which is used in the application software if the address of FblHeader changes!

By disabling this option, the user can gain more influence on how the transition from application to bootloader is done.

This is the procedure which has to be followed:

> Set a flag in the application software

> Issue a reset

> Check flag in user adaptable function fbl_apnv.c::`ApplFblExtProgRequest()` and provide the appropriate return value.



**Caution**
Do not use macro `CallFblStart()` if the option is disabled!



**Note**
Please consult chapter 6.4 Transitioning from Application to Bootloader for further details.

### 4.2.22 Stay in Boot

If activated, the FBL waits a specified time (CAN: 5 ms, LIN: 20 ms) before jumping to an (apparently) valid application. If a DiagnosticSessionControl/programmingSession request is received during this time, the ECU stays in the bootloader.

This feature works as a backdoor solution which is useful in situations where the ECU is not accessible (via CAN) anymore due to a corrupt application software.

For VolvoCar bootloaders, the stay-in-boot mechanism is mandatory and cannot be disabled. An exception is the FlexRay bootloader – in this case, stay-in-boot will not be used.



**Reference**
See chapter 5.9 Validation OK - Application Faulty for more details about the stay-in-boot feature.

### 4.2.23 Unaligned Data Transfer

If enabled, the data transferred during download using the TransferData service can be of arbitrary length. Otherwise all data transfers have to be aligned to a multiple of the largest memory write segment size.

### 4.2.24 User Config File

To configure aspects which are not available through DaVinci Configurator yet, a user configuration file has to be added. An example file is included in the delivery (Demo\DemoFbl\Config\fbl_user.cfg). Please use this file as a basis for your own bootloader project.

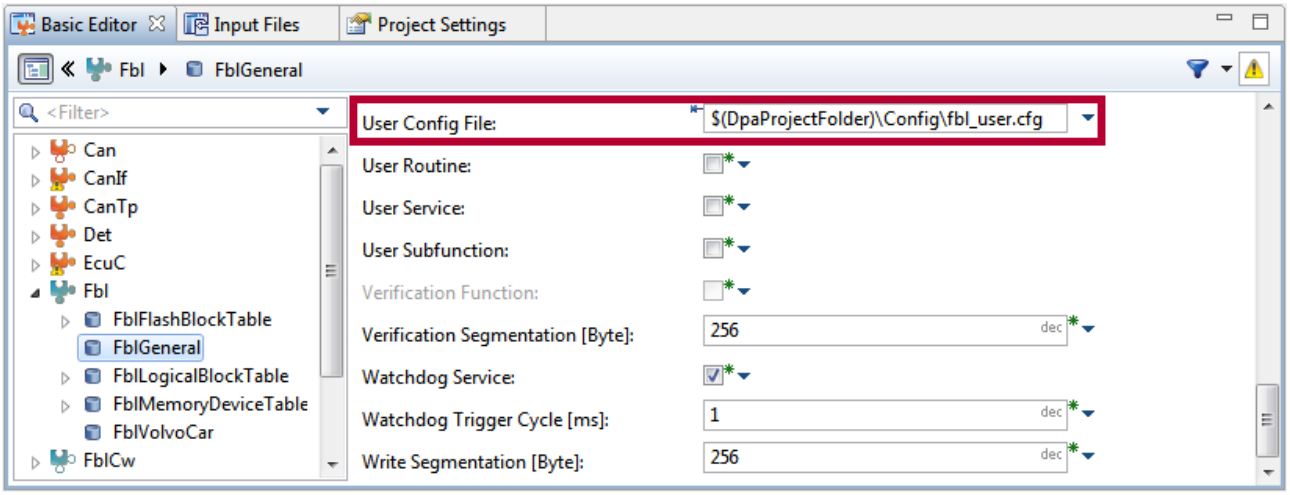



Figure 4-13 DaVinci Configurator: FblGeneral User Config File

Most of the switches in the user config file have been pre-configured and do not need to be modified. The following table lists the switches you should review and – if necessary – adapt to your hardware environment or your project specific requirements:

| Define | Description |
|---|---|
| Hardware specific settings | |
| FLASH_SIZE | This option defines the size of the RAM buffer which keeps the downloaded flash driver. Usually, the default value is already pre-configured reasonably. The actual size of the flash driver can be determined by looking at the respective .hex/.vbf file in folder BSW\Flash.<br><br>**Caution**<br>It's recommended to include a certain reserve when choosing the buffer size to account for potential future flash driver updates. |
| FBL_TIMER_RELOAD_VALUE | Base timer configuration of the bootloader (1ms); please refer to the hardware specific technical reference manual and the example provided in the demo bootloader. |
| <other hardware specific settings> | Please refer to the hardware specific technical reference manual and the example provided in the demo bootloader. |
| Project specific settings | |
| FBL_ENABLE_SEC_ACCESS_DELAY | This switch allows to enable/disable security access delay (usually required). The current delay state can be persisted using the ApplFbl*SecAccess*() API in fbl_apnv.c. |

| | |
|---|---|
| FBL_SEC_ACCESS_DELAY_TIME | Time penalty in [ms] which will be applied after two consecutive security access attempts have failed. |
| FBL_CW_ENABLE_QUEUED_REQUEST | Feature switch to enable or disable queued requests. According to VolvoCar requirements, this should always be enabled. |
| FBL_CCBLOCK_ADDR | Set this define to the appropriate address if you want to use a complete & compatible block embedded in the application software. Remove the define if you want to perform the compatibility check in the bootloader (ApplFblCheckCompatibility). |
| FBL_APPL_ENABLE_OTP | By default, the bootloader persists some information within the so-called OTP area (e.g. the public key for production). This is a one-time-programmable area within the protected PBL memory region. If you want to replace this functionality with an own solution, you can disable it by changing the define to FBL_APPL_DISABLE_OTP. |
| FBL_ENABLE_VALSTRUCT | Required for software authentication. This switch cannot be disabled. |
| FBL_VALSTRUCT_ENABLE_BLANK_CHECK | If enabled, the bootloader ensures that all memory regions which are not described in the verification structure are in ‘erased’ state. |
| FBL_VALSTRUCT_ENABLE_EXTERNAL_KEY | Enables the support for the programmable public key (production). If disabled, the ValStruct module will always use the built-in development key. |

Table 4-1 DaVinci Configuration: User Config File Settings

### 4.2.25 User Routine

This option can be activated if user specific diagnostic routines have to be supported which are not defined within the SWDL specification.

If the bootloader receives a request with an unsupported routine ID, the user adaptable function fbl_apdi.c::ApplDiagRoutineControl() will be called.



**Edit**
Support for customer specific routine IDs can be added to user adaptable function fbl_apdi.c::ApplDiagRoutineControl().

### 4.2.26 User Service

This option can be activated if user specific diagnostic services have to be supported which are not defined within the Software Download Specification (SWDL).

If the bootloader receives a request with an unsupported service identifier, the user adaptable function fbl_apdi.c::ApplDiagUserService() will be called.



**Edit**
Support for customer specific diagnostic services can be added to user adaptable function fbl_apdi.c::ApplDiagUserService().

### 4.2.27 User Subfunction

This option can be activated if user specific subfunction parameters have to be supported which are not defined within the Software Download Specification (SWDL).

If the bootloader receives a request with a supported service identifier and an unknown subfunction parameter, the user adaptable function fbl_apdi.c::ApplDiagUserSubFunction() will be called.



**Edit**
Support for customer specific subfunction parameters can be added to user adaptable function fbl_apdi.c::ApplDiagUserSubFunction().

### 4.2.28 Verification Segmentation

The configured value defines the maximum data length passed to the on-the-fly verification (pipelined verification) in a single processing cycle.

### 4.2.29 Watchdog Service

If the FBL is used in an ECU with activated (hardware) watchdog, the watchdog support of the FBL can be enabled with this switch.

### 4.2.30 Watchdog Trigger Cycle

Defines the watchdog trigger cycle in [ms]. Function fbl_wd.c::`FblLookForWatchdog()` will call the user adaptable function fbl_apwd.c::`ApplFblWDTrigger()` with the defined call cycle.



**Reference**
Details about the bootloader's watchdog support and the respective functions can be found in chapter 9.4.2 The Watchdog.

### 4.2.31 Write Segmentation

This option defines the maximum data length passed to the programming routine in a single processing cycle. Used for option "Pipelined Programming".

## 4.3 Module Fbl | FblMemoryDeviceTable

This table holds a list of the available memory devices. Two entries have already been pre-configured:

- Flash: device reflects the program flash

- Ram: device is used to handle the SBL download

Note: The definition of the memory device is a precondition for adding the respective memory blocks to the flash block table.

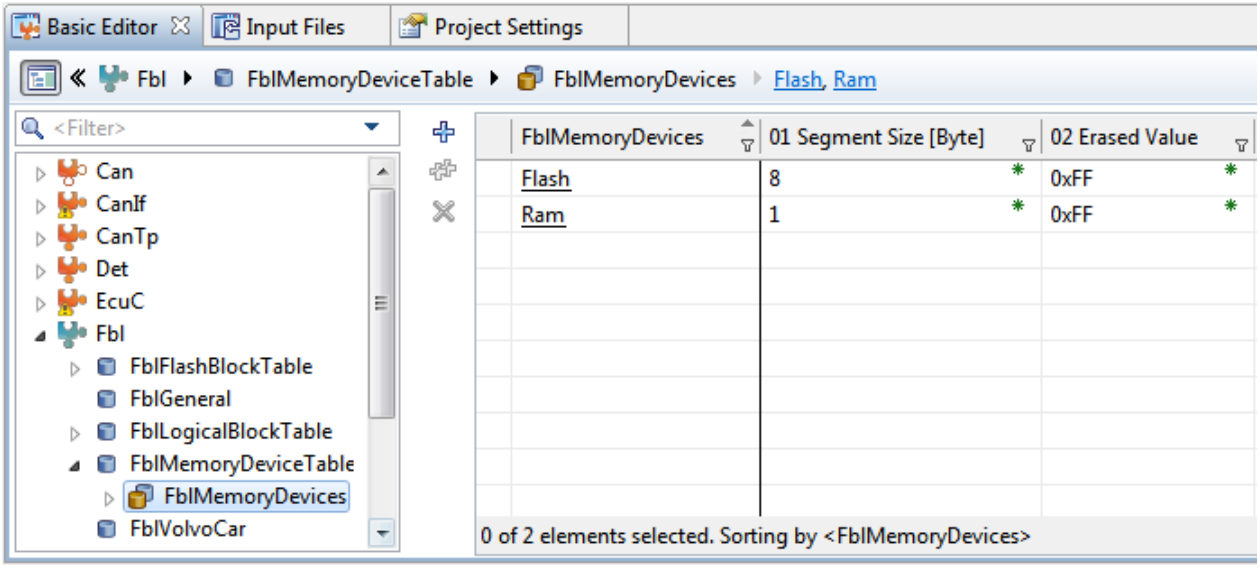



Figure 4-14 DaVinci Configurator: FblMemoryDevices

By adding a new memory device, the following attributes have to be configured:

### Short Name

This is the name of the memory device selectable in the flash block table. The memory device name is used to reference the memory device driver functions. The detailed API description can be found in chapter 9.1 Flash Driver Interface.

**Example**
If a device type named “**Eeprom**” is added, the following driver functions will be called for accessing the respective memory device:

> **Eeprom**Driver_InitSync
> **Eeprom**Driver_DeinitSync
> **Eeprom**Driver_RReadSync
> **Eeprom**Driver_RWriteSync
> **Eeprom**Driver_REraseSync

### Segment Size

Write segment size of the memory device. The write segment size is the smallest entity that can be programmed in one write access. Refer to the memory device specification for this value.

### Erased Value

Specify the value which is returned when reading a byte from the erased memory. Refer to the memory device specification for this value.

## 4.4 Module Fbl | FblFlashBlockTable

The actual memory mapping takes place in the flash block table configuration.

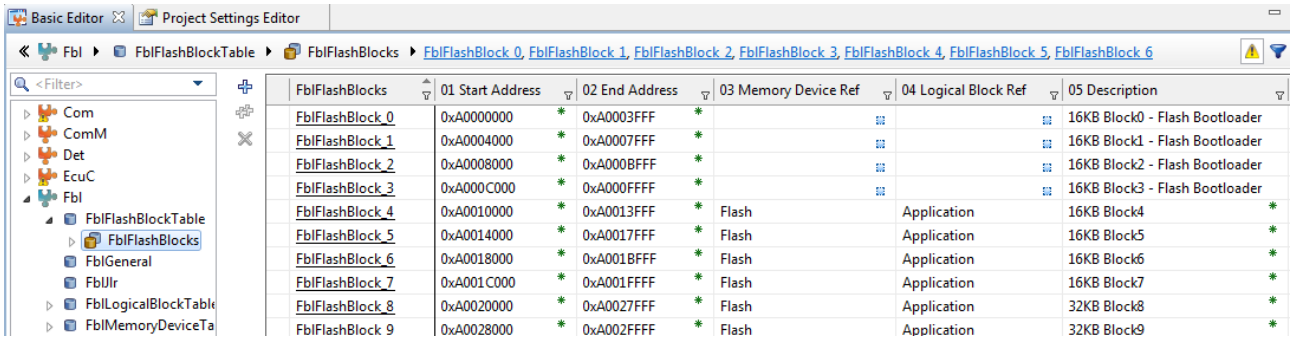



Figure 4-15 DaVinci Configurator: FblFlashBlockTable

**Note**
The table is already pre-configured for the ordered derivative and usually reflects the physical partitioning of the memory. In case of small and homogenous block sizes, some memory blocks may be combined to create a bigger block to reduce memory consumption and maintenance effort.

**Memory Device Ref** is used to assign a flash block to a certain memory device. To protect flash blocks from being accessed through the Bootloader, e.g. because the Bootloader code itself is located there, it isn't necessary to remove them from the table. Instead the **Memory Device Ref** of the block can be deleted. Setting the value to **<NONE>** will not work.

**Note**
Multiple entries can be set at once by selecting them in the grid view.

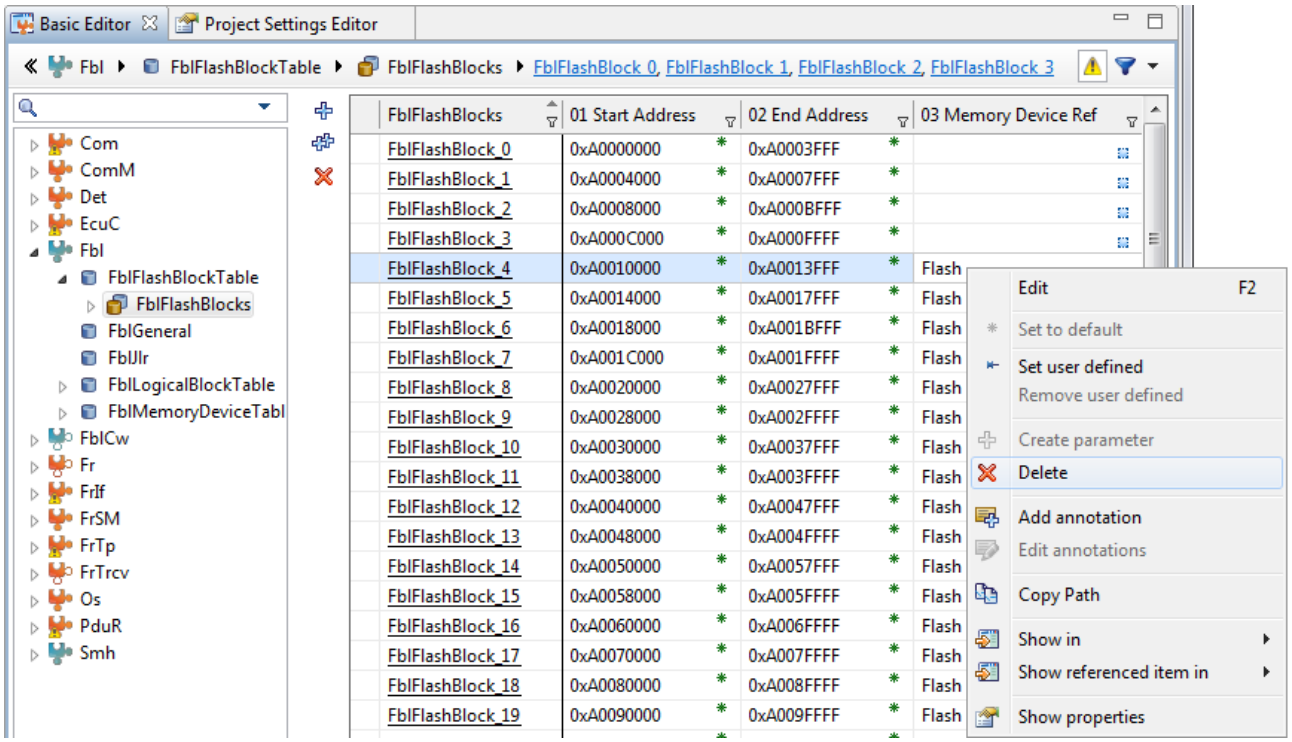



Figure 4-16 DaVinci Configurator: Block Protection

The same scheme used for the memory devices also applies when assigning flash blocks to logical blocks. Using **Logical Block Ref** entries in the logical block table can be referenced.

## 4.5 Module Fbl | FblLogicalBlockTable

The software parts which shall be reprogrammable are defined in the sub container **FblLogicalBlockTable | FblLogicalBlocks**. This table defines the logical partitioning of the available memory which is defined in the flash block table. In other words, the hardware related flash blocks can be grouped into logical partitions for e.g. application software or calibration data.

Each logical block allows configuring the following attributes:

- Name: In case of non-volatile memory, this is a user specific identifier for the logical block. Mandatory block “SBL” is a special case as it defines the RAM area which will contain the secondary bootloader. Take care that:
  - The start address of the block defined in the Flash Block Table equals the actual start address of the SBL (see BSW\Flash\FlashDrv.vbf)
  - The size of the block defined in the Flash Block Table equals the flash code buffer size (define `FLASH_SIZE` in user config file fbl_user.cfg)
- Block Index: This is a user-specific, unique 32-bit ID which can be defined by the user to address a certain block from within the bootloader code.
- Block Type:
  - “SBL” for the RAM area which will contain the Secondary Bootloader

- “SOFTWARE” for any other block in non-volatile memory
- “ESS” for the reprogrammable LBT/ESS (only required if option is enabled)

- Disposability: Defines if the block has to be ‘valid’ (mandatory) in order to start the application software or not (optional).

**Caution**
If a logical block is configured as ‘optional’, the integrity and authenticity will be verified by RoutineControl/CheckMemory nevertheless. If the check fails, the validity status of the block will remain ‘invalid’. Since it’s an ‘optional’ block, the application software will be started anyway. That means, the responsibility to check the validity of the block before accessing the content is passed to the application software!

- Start Address/End Address: Defines start and end address of the logical block. The fields are read-only and updated according to the selection in the Flash Block Table.
- Header Address: Points to the start address of the Validation Structure. Consider the expected maximum size of the structure when choosing this value! If the structure shall be located at the end of the logical block, keep the memory region in mind which is occupied by the Presence Pattern!
- Output Verification: Specifies the function which performs the hash calculation when the bootloader receives a RoutineControl/CheckMemory request. To override the built-in calculation (provided by SecM_Default), enter the name of a user specific function which fulfills the interface requirements (see description of ApplFblDiagVerification in chapter 9.2.2.50).
- Description: Allows entering user specific comments about the logical block.

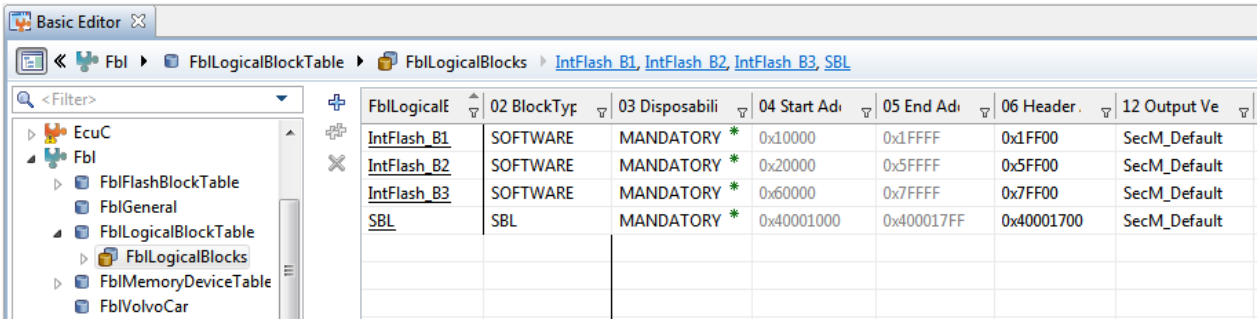



Figure 4-17 DaVinci Configurator: FblLogicalBlockTable

The start and end addresses will be automatically updated as soon as the logical blocks are referenced in the flash block table.

**Caution**
Logical blocks have to be *homogeneous* as well as *contiguous*.

Homogeneous means one logical block can only span flash blocks of the same memory device.

Contiguous means there are no memory gaps between the respective flash blocks.

## 4.6 Module Fbl | VolvoCar

This module comprises all configuration items that are specific to the VolvoCar flash bootloader.

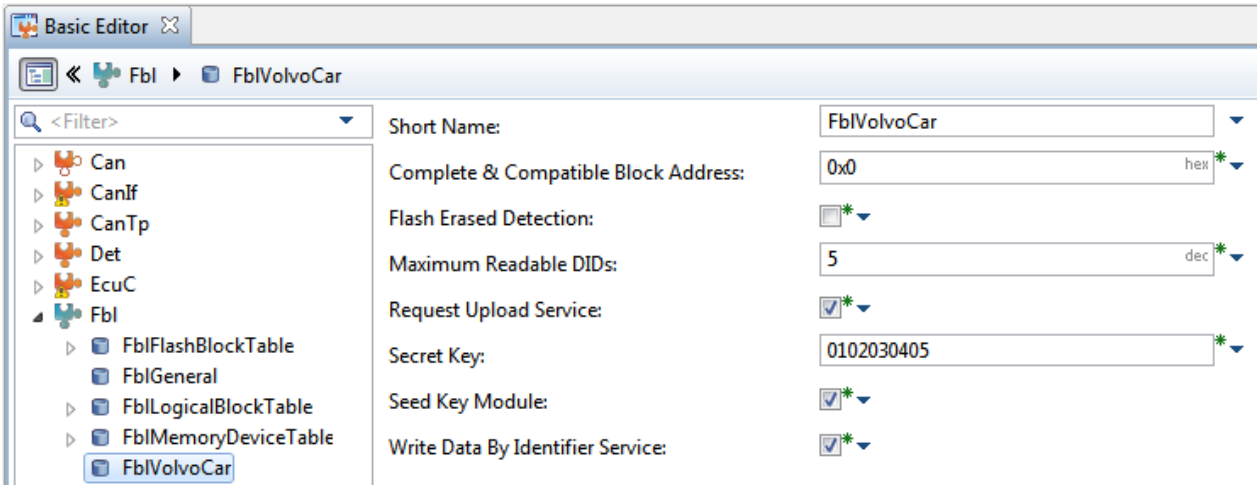



Figure 4-18 DaVinci Configurator: Module FblJlr

Please refer to window **Properties | Description** and the sub chapters below for a detailed description of the available options.

### 4.6.1 Complete & Compatible Block Address

Specify the address of the complete and compatible block in the application software. The complete and compatible block will be searched at this memory location by the bootloader.

**Note**
It's recommended to place the complete and compatible block at a memory location which doesn't interfere with the (potentially growing) application software.

### 4.6.2 Flash Erased Detection

The flash erased detection provides the possibility to deliver ECUs to the OEM with the complete programmable flash memory already erased to reduce the overall software download time.

If this option is enabled, the bootloader provides the user adaptable function fbl_apnv.c::ApplFblGetBlockErased() which is called prior to any erase operation. Depending on the return value, the actual erase operation is performed or not.

Like the presence pattern, the flash erased state is stored as a pattern which is located within a write protected area at the end of the logical block.



**Caution**
The default implementation in ApplFblGetBlockErased() is provided as an example only. In particular, it might be more convenient to store the flash erased status in an EEPROM (if available).

### 4.6.3 Maximum Readable DIDs

This attribute defines the maximum number of data identifiers which can be handled by the bootloader in a single ReadDataByIdentifier request. The specified value is used to allocate a RAM buffer which will keep the DIDs which have been extracted from the diag buffer.

If the specified maximum number is exceeded, the bootloader will respond with NRC 0x13 (IncorrectMessageLengthOrInvalidFormat).

### 4.6.4 RequestUpload Service

If activated, the optional diagnostic service RequestUpload (0x35) will be provided. Turn the option off to reduce the bootloader code size or if the service shall not be supported.

### 4.6.5 Secret Key

This is the ECU specific secret key (also known as “fixed security bytes”) for the seed/key functionality.

### 4.6.6 Seed/Key Module

With enabling this option, each access to the bootloader's core functionality will require that the ECU is unlocked before. For this purpose, a seed/key mechanism is used which is implemented according to the security algorithm(s) specified by the OEM. Details about the security module can be found in chapter 8 Security Module.

### 4.6.7 WriteDataByIdentifier Service

If activated, the optional diagnostic service WriteDataByIdentifier (0x2E) will be provided. Turn the option off to reduce the bootloader code size or if the service shall not be supported.



**Edit**
Customer specific data identifiers can be added to user adaptable function fbl_apdi.c::ApplFblWriteDataByIdentifier().

## 4.7 Module FblCw

The VolvoCar flash bootloader uses a sub-set of the MICROSAR communication stack, up to the PDU router. Above this, an additional communication wrapper layer adapts the

PduR interface to the respective FBL interfaces. From the ASR point of view, this wrapper (FblWrapperCom / FblCw) is implemented as a CDD with communication stack contribution.

The PDUs of the lower layer communication stack have to be mapped to the communication wrapper component. Navigate to **PduR | PduRBswModules**. Create an additional container, which references the **FblCw** module and is configured as an upper layer TP module, as shown in Figure 4-19.

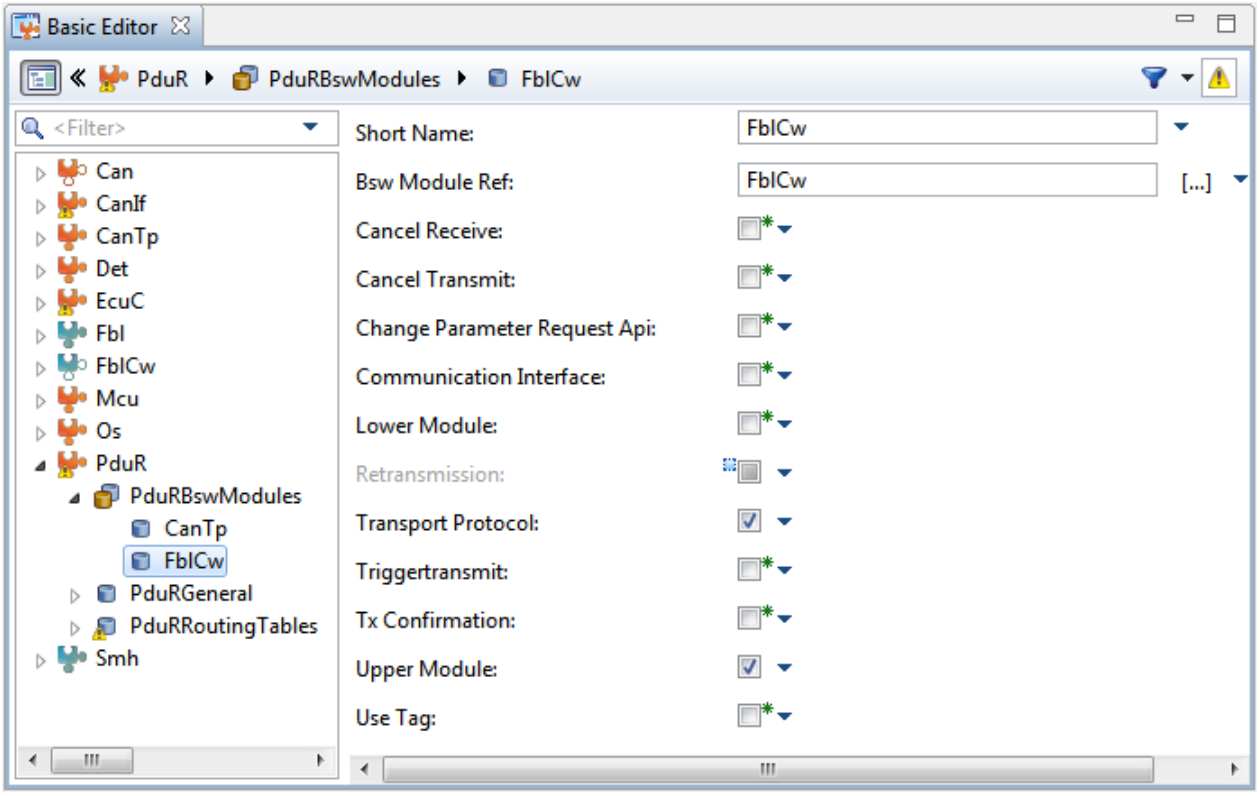



Figure 4-19 DaVinci Configurator: PduR BSW Modules

In the previously added CDD module **FblCw** a **CddPduRUpperLayerContribution** has to be added to the **CddComStackContribution** container.

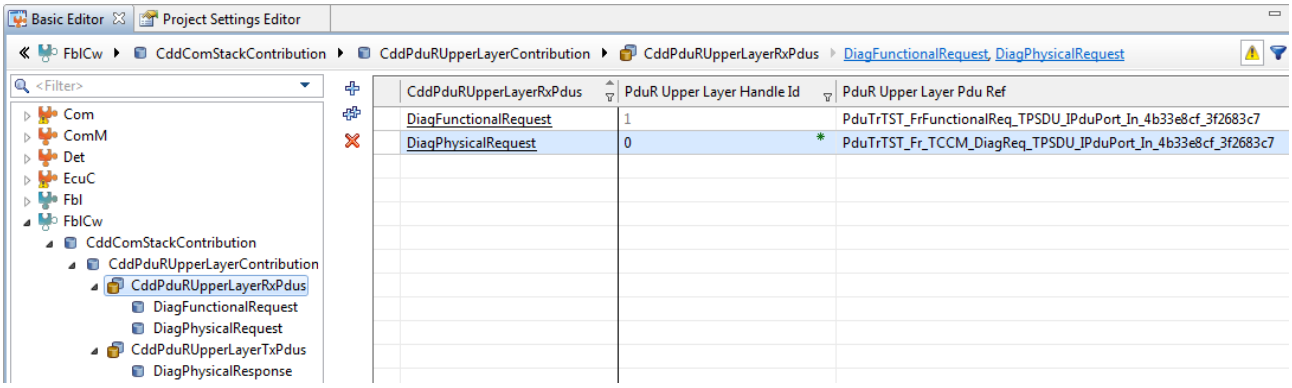



Figure 4-20 DaVinci Configurator: CDD ComStack Contribution

For each tester connection add two sub-containers to **CddPduRUpperLayerRxPdus** and one sub-container to **CddPduRUpperLayerTxPdus** respectively inside the upper layer contribution.

Rename the **Short Name** according to the following list:

- DiagPhysicalRequest
- DiagFunctionalRequest
- DiagPhysicalResponse

Assign the matching **NSdu** references from the transport layer to the **PduR Upper Layer Pdu Ref.**

Certain aspects of the communication stack require some additional manual configuration. The template files _FblCw_Cfg.c and _FblCw_Cfg.h located in BSW\Fbl_Template contain a pre-configured example.



**Edit**
The underscore has to be removed from the template files before use.
See the DemoFbl project located at Demo\DemoFbl\Appl for reference.

The configuration options can be split into two groups: the general settings as listed in Table 4-2 and the connection settings as shown in Table 4-3.

| Parameter | Default | Description |
|---|---|---|
| fbl_cw_cfg.h | | |
| FBLCW_ENABLE_BUSTYPE_CAN | - | Enable CAN communication handling |
| FBLCW_ENABLE_BUSTYPE_ETH | - | Enable Ethernet / IP communication handling |
| FBLCW_ENABLE_BUSTYPE_FR | - | Enable FlexRay communication handling |
| FBLCW_TASK_CYCLE | 5u | Cycle count of CanTp, EthSM and ComM (in ms) |
| FBLCW_ENABLE_MULTIPLE_CONNECTIONS | - | More than one connection configured, use connection table<br>Automatically set when FblCw PDUs are set up correctly in DaVinci Configurator |
| FBLCW_CANTRCV_INITMEMORY() | - | Memory initialization function for CAN transceiver (optional) |
| FBLCW_CANTRCV_INIT(cfgPtr) | - | Initialization function for CAN transceiver (optional) |
| FBLCW_ETHTRCV_INITMEMORY() | EthTrcv_30_88E1116R_InitMemory() | Memory initialization function for Ethernet transceiver (optional) |
| FBLCW_ETHTRCV_INIT(cfgPtr) | EthTrcv_30_88E1116R_Init(cfgPtr) | Initialization function for Ethernet transceiver (optional) |
| fbl_cw_cfg.c | | |
| fblCwEthPhyAddr | 0x02u, 0x00u, 0x00u, 0x00u, 0x00u, 0x02u | MAC address of ECU |

Table 4-2 FblCw Configuration: General Settings

Depending on the number of configured connections, the settings are either located in FblCw_Cfg.h (single connection) or FblCw_Cfg.c (multiple connections).

A connection consists of the following attributes:

- Physical Rx PDU

  As configured in **CddPduRUpperLayRxPdus**, with the symbolic handle build according to scheme FblCwConf_CddPduRUpperLayerRxPdu_<Short Name>

- Functional Rx PDU

  As configured in **CddPduRUpperLayRxPdus**, with the symbolic handle build according to scheme FblCwConf_CddPduRUpperLayerRxPdu_<Short Name>

- Local Tx PDU

  As configured in **CddPduRUpperLayTxPdus**, with the symbolic handle build according to scheme FblCwConf_CddPduRUpperLayerTxPdu_<Short Name>

- Remote Tx PDU

  Symbolic handle required for transmission through the PDU router API

  Name is build according to scheme PduRConf_PduRDestPdu_<Short Name>, with the short name comes from the **PduRDestPdus** container of the **PduRRoutingPath**.

- Bus type

  Bus type of connection

- Controller reference

  Reference to controller of respective interface module:

  > CAN: CanIfCtrlCfg

  > Ethernet: EthIfController

  > FlexRay: FrIfController

| Parameter | Example | Description |
|---|---|---|
| fbl_cw_cfg.h | | |
| FBLCW_RX_PHYS_PDU | FblCwConf_CddPduRUpperLayerRxPdu_ DiagRequest_0 | Physical Rx PDU |
| FBLCW_RX_FUNC_PDU | FblCwConf_CddPduRUpperLayerRxPdu_ DiagFuncRequest_0 | Functional Rx PDU |
| FBLCW_TX_LOCAL_PDU | FblCwConf_CddPduRUpperLayerTxPdu_ DiagResponse_0 | Local Tx PDU |
| FBLCW_TX_REMOTE_PDU | PduRConf_PduRDestPdu_DST_ Can_PT_Ch0_DiagDemoResponse0_0fb271d2 | Remote Tx PDU |
| FBLCW_BUSTYPE | FBLCW_BUSTYPE_CAN / FBLCW_BUSTYPE_ETH / | Bus type of connection |

| Parameter | Example | Description |
|---|---|---|
| fbl_cw_cfg.h | | |
| | FBLCW_BUSTYPE_FR | |
| FBLCW_CONTROLLER | CanIfConf_CanIfCtrlCfg_CT_CAN00_cce42491 | Controller reference from<br>■ CAN: CanIfCtrlCfg<br>■ Ethernet: EthIfController<br>■ FlexRay: FrIfController |
| fbl_cw_cfg.c / fblCwPduConnections table | | |
| rxPhysPdu | FblCwConf_CddPduRUpperLayerRxPdu_DiagRequest_0 | Physical Rx PDU |
| rxFuncPdu | FblCwConf_CddPduRUpperLayerRxPdu_DiagFuncRequest_0 | Functional Rx PDU |
| txLocalPdu | FblCwConf_CddPduRUpperLayerTxPdu_DiagResponse_0 | Local Tx PDU |
| txRemotePdu | PduRConf_PduRDestPdu_DST_Can_PT_Ch0_DiagDemoResponse0_0fb271d2 | Remote Tx PDU |
| bustype | FBLCW_BUSTYPE_CAN / FBLCW_BUSTYPE_ETH / FBLCW_BUSTYPE_FR | Bus type of connection |
| controller | CanIfConf_CanIfCtrlCfg_CT_CAN00_cce42491 | Controller reference from<br>■ CAN: CanIfCtrlCfg<br>■ Ethernet: EthIfController<br>■ FlexRay: FrIfController |

Table 4-3 FblCw Configuration: Connection Settings

## 4.8 Module Smh | SmhGeneral

This module (Security Module HIS) contains the configuration options for the security module which is delivered as part of the bootloader. Depending on the ordered bootloader options, only a subset of the available configuration items will apply.

Module **SmhGeneral** allows configuring the workspace, operation mode and RSA key size. Usually, there's no need to adapt the default options.

## 4.9 Module Smh | SmhCryptographicKeys

Module **SmhCryptographicKeys** works like a pool for all cryptographic keys that are used in the project.

Container **SmhRsaKeys** allows configuring the RSA key which is used by the bootloader to verify the downloaded data. The development key which is defined by VolvoCar has been preconfigured, so usually there's no need for change.

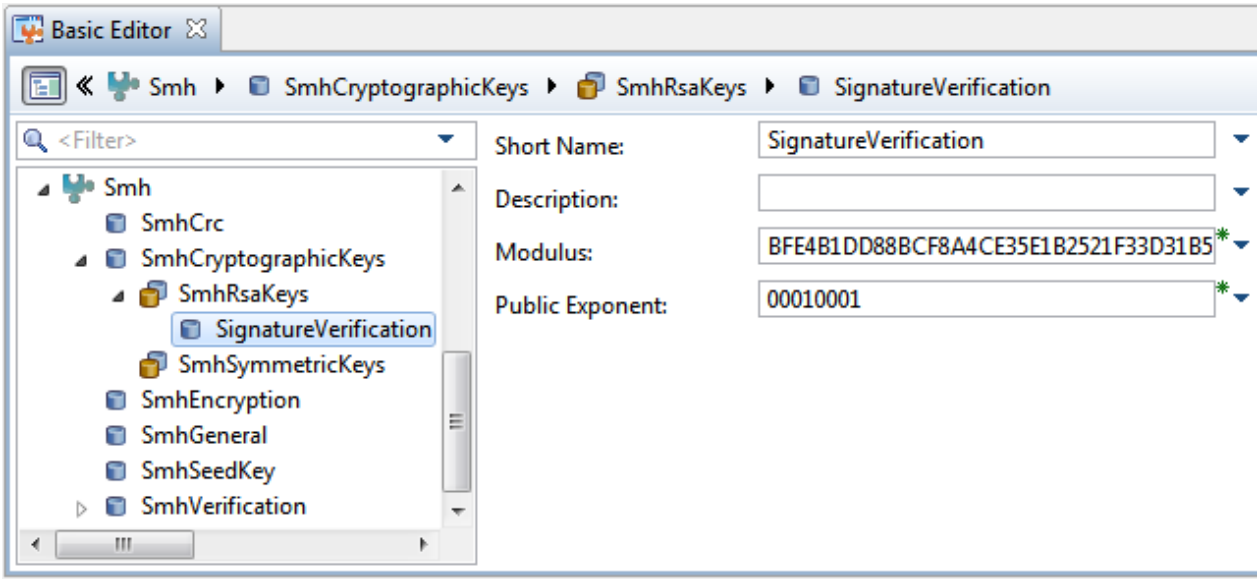



Figure 4-21 DaVinci Configurator: Configuration of RSA key

If the optional bootloader feature „AES128 Decryption“ (Security Class AAA) has been ordered, you can enter the respective symmetric key by adding a new container below **SmhSymmetricKeys**.

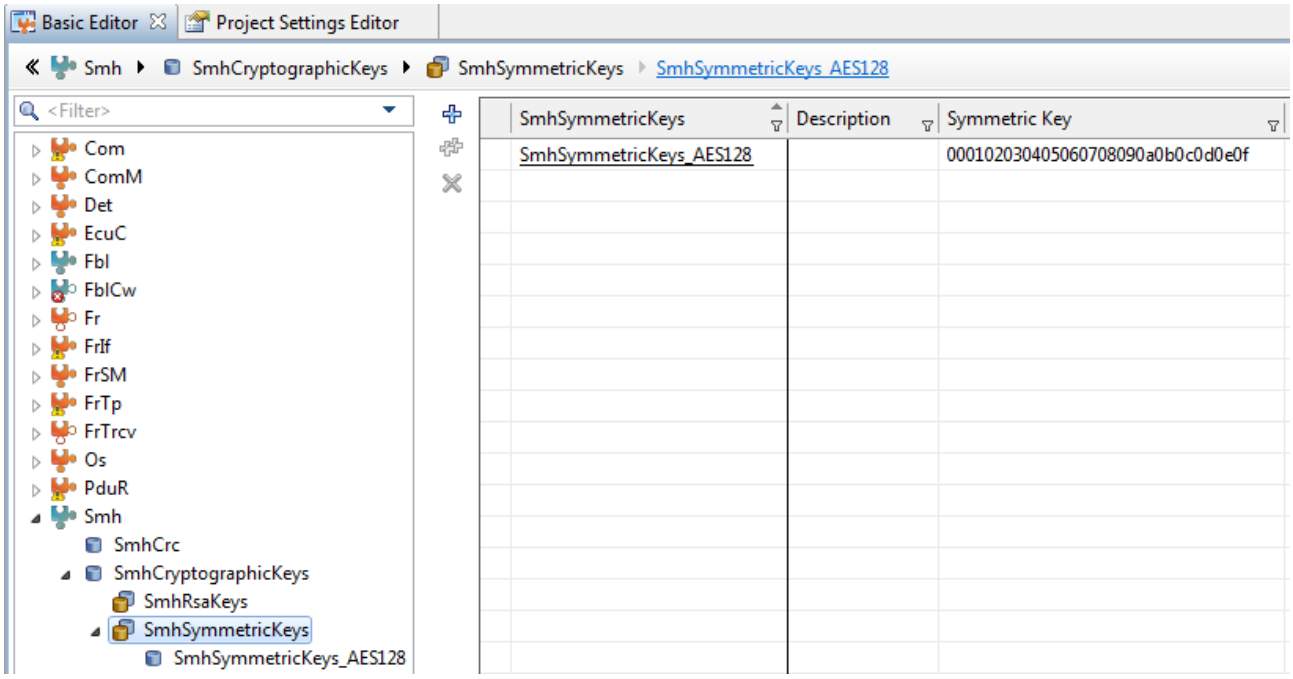



Figure 4-22 DaVinci Configurator: Configuration of Symmetric Key

## 4.10 Module Smh | SmhEncryption

Module **SmhEncryption** allows to configure all items relating to the optional bootloader feature „AES128 Decryption“ (Security Class AAA). Relevant options:

- Decryption: This option enables the support for decryption of encrypted download data. Note that you also have to enable the option „Encryption Mode“ in module **FblGeneral** to generally allow encrypted downloads.
- Default Cryptographic Key: Select the respective key which is defined in module **SmhSymmetricKeys**.
- Encryption Algorithm: Allows selecting the desired decryption algorithm.

### 4.11 Module Smh | SmhVerification

Module **SmhVerification** contains all configuration options which are relevant for verification of the downloaded data.

- Checksum Type Class DDD: HASH
- Cryptographic Verification  Keys: BOTH
  - The INTERNAL key corresponds to the development key
  - The EXTERNAL key corresponds to the production key which can be programmed once by WriteDataByIdentifier
- Default Cryptographic Key Class CCC: Refers to the key container that has been created below SmhRsaKeys.
- Hash Algorithm: SHA_256
- Security Class CCC: used for verification of the validation structure
- Security Class DDD: used for download segment verification

### 4.12 Module Smh | Crc

This module is not used by the VolvoCar bootloader.

### 4.13 Module Smh | SmhSeedKey

This module is not used by the VolvoCar bootloader.

# 5 Validation / Invalidation of Flashed Application

## 5.1 [#oem_valid] Introduction to VolvoCar Validation

For more general information about this see the UserManual_FlashBootloader in the chapter **Proposals for Handling the Validation Area.**

The Vector bootloader provides a general framework within which the validation is done. The validation framework requires that the bootloader is able to perform the following:

- Validate a memory region after that region is successfully downloaded
- Invalidate a memory region before it is about to be erased
- Check if a region is valid to decide if transitioning to the application is possible

The VolvoCar specification describes a validation mechanism which is split into a completeness check and a compatibility check.

- The **completeness check** is triggered by RoutineControl/CheckMemory after the download of a single software unit has been finished. The completeness check is pre-implemented in the bootloader. It will verify the downloaded memory content using a dual-step approach:
  - Authenticate verification structure which is part of the download using the RSA signature which is passed by RoutineControl/CheckMemory.
  - Check the integrity of the remaining download segments by using the information provided within the verification structure.
- The **compatibility check** is initiated by RoutineControl/CheckCompleteCompatible and during bootloader startup. The actual implementation has to be done by the user (because it is specific to the application software). The bootloader provides two different variants for this purpose:
  - The compatibility check is implemented inside the bootloader.
  - The compatibility check is implemented inside the application software. The bootloader performs a callout to the respective function.



**Reference**
For details about the OEM requirements, please refer to the following chapters in [3]:
- 4.1.15 Software Authentication
- 4.1.16 Validation of ECU software(s)



**Note**
Since the call-out to the complete & compatible function in the downloaded application software comprises some risks, this method should only be applied if the used hardware features a memory protection unit.

Furthermore, the execution of the complete & compatible function shall be aborted after a maximum runtime of 15ms. Note that this functionality is currently not included with the Vector bootloader and has to be added by the user if complete & compatible is used.

## 5.2 [#oem_valfunc] The Validation Functions

A validation strategy has to answer one central question: What makes an application valid?

Once that answer is determined by the user, the validation strategy can be easily integrated into the flash bootloader through the following callback functions (located in module fbl_apnv.c):

- **ApplFblIsValidApp**: This function is called to determine if all mandatory logical blocks are valid (i.e. the blocks have passed the authenticity check in ApplFblValidateBlock). Subsequently, the function has to ensure that all downloaded software units are compatible with each other. Based on the return value of this function, the bootloader will decide if it can transition to the application or not.
- **ApplFblInvalidateBlock**: Before a certain memory area is erased, the corresponding logical block has to be invalidated. Failing to do so, can lead to the FBL mistakenly assuming that the application is valid when parts of it are erased, and therefore executing an incomplete application.
- **ApplFblValidateBlock**: The basic goal of this function is to determine the validity of a single logical block. The result has to be persisted in non-volatile memory, so it can be evaluated by ApplFblIsValidApp (or optionally ApplFblValidateApp).
- **ApplFblValidateApp**: This function can be used to determine the overall validity of the ECU. In particular, this can be useful in multi-processor configurations or if complex dependencies between the downloaded software parts have to be checked at the end of the download. The result of this check has to be persisted in non-volatile memory so it can be read out by ApplFblIsValidApp during ECU startup.



**Info**
Note that memory driver functions (MemDriver_RWriteSync, MemDriver_RReadSync, MemDriver_REraseSync) can be called within any of these callbacks to allow the user to read, write and erase validity flags. When using these driver functions for validation purposes the user must ensure that the minimum number of bytes for programming (memory write segment size) and the minimum sector erase size are observed based on the memory type used.

## 5.3 Handling of Validity Information

Checking downloaded data usually takes some time to reveal the validity state. This is why the validity state is determined directly after the download and stored within a special memory region called “Validation Area”.

There are various kinds of methods for handling the validity information – the most common ones are setting a flag in EEPROM or storing a special pattern in flash memory. For the latter, an example can be found in chapter 5.7 Presence Pattern.

The corresponding implementation is done within the following user callback functions:

| User Callback Function | Validity Information |
|---|---|
| ApplFblValidateBlock | Store (set logical block to ‘valid’) |
| ApplFblValidateApp | Store (set application/system to valid) – optional |
| ApplFblInvalidateBlock | Clear (set logical block to ‘invalid’) |
| ApplFblIsValidApp | Check validity of all mandatory logical blocks |

Table 5-1 Handling of the Validity Information

## 5.4 Files, Blocks and Software Units

The description within the following chapters contains some special terms which shall be explained here. In the illustrations below the term **blocks** is mentioned. The block(s) to be flashed are transported within **files** (VBF container, see [4] for details). One file can contain more than one block. Separate blocks may result from unused memory regions or alignment gaps. In order to distinguish these blocks from individual download parts (e.g. application and calibration), the term **software unit** shall be used for the latter.

If your download consists of multiple software units (e.g. 3 parts: one application and two calibration units), every part has to be stored within a single VBF file [REQPROD 397292].

## 5.5 Logical Block Table

In order to facilitate the handling of multiple software units, the bootloader provides a logical block table. Each logical block in the table represents a separate software unit.

The information in the table will be used for the following purposes:

> Checking of erase and download requests: the erase requests that will be sent to the bootloader will only be accepted if they (exactly) match a configured logical block.

Consequently, that means, the erase information in the VBF header has to comply with the logical block configuration. If this is not the case, the bootloader will respond with NRC 0x31 (RequestOutOfRange).

> Location and protection of the block specific presence pattern area (see chapter 4.6.1): download requests which overlap with the presence pattern area will be responded with NRC 0x31 (RequestOutOfRange).

> Configuration of block specific verification function which performs the hash calculation when the bootloader receives a RequestTransferExit or RoutineControl/CheckProgrammingDependencies request.

> Extraction of the information which is contained in the block specific Validation Structure.

> Determine if a certain logical block is required to start the application software (Disposability).



**Caution**
Please consider that the layout of the reprogrammable memory area (as defined by the logical block table) cannot be changed after the production version of the primary bootloader has been programmed into the ECU (unless option Reprogrammable LBT is enabled).

That means, the logical block table might potentially restrict the structure and the size of future application software versions or data regions. Consequently, the size of the respective logical blocks should be chosen carefully.

## 5.6 Programming Sequence and User Callback Functions

The illustration below shows the programming sequence as specified by VolvoCar on the left side. The column on the right shows the user callback functions that will be executed at the corresponding steps of the download flow.

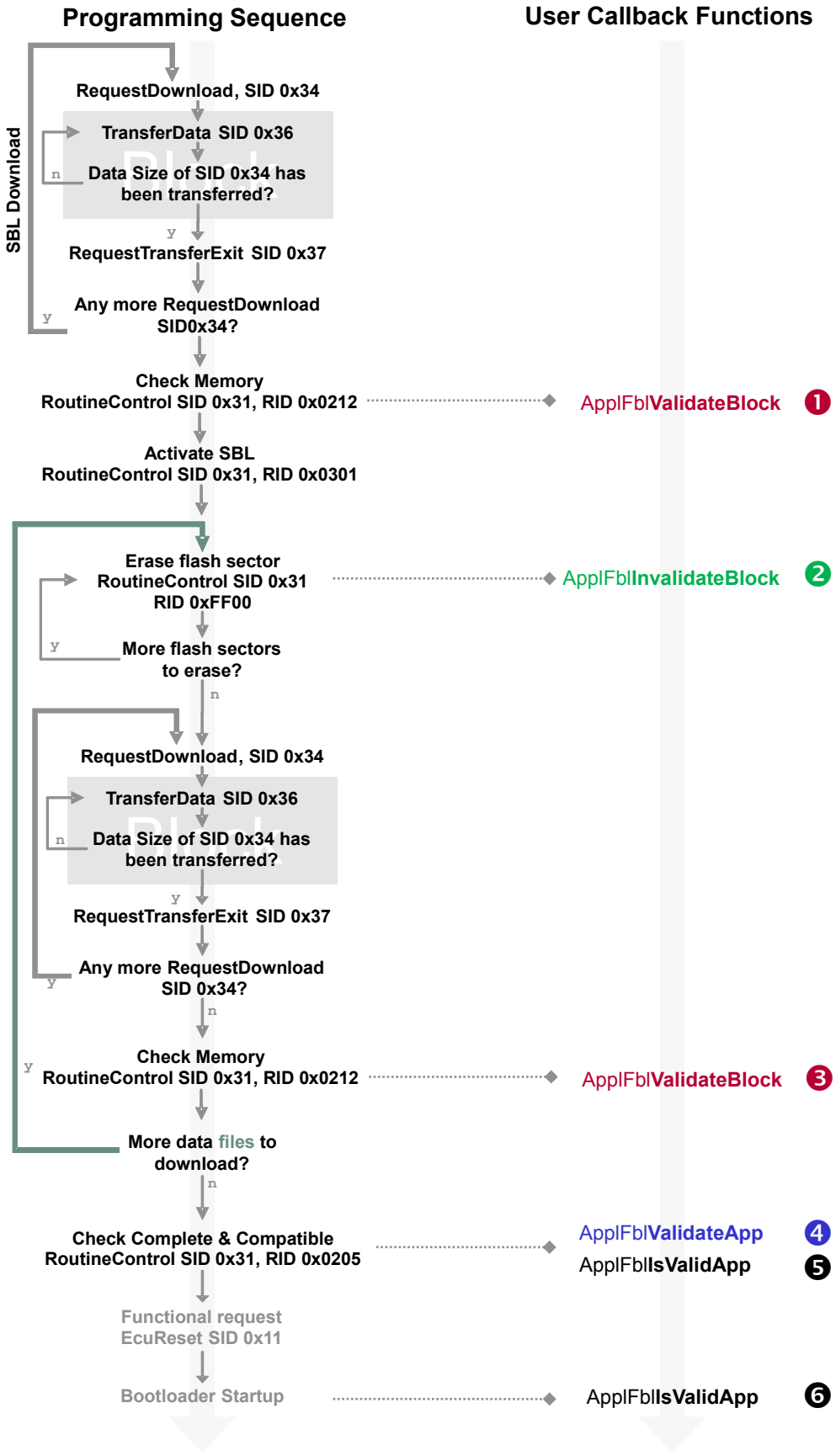



Figure 5-1 Detailed Validation Flow with User Callback Functions

Each download sequence begins with a request to start a programming session, followed by the prologue, security access and the SBL download. After the SBL has been downloaded, the bootloader will call user function ApplFblValidateBlock ❶ on reception of a RoutineControl/CheckMemory request. Based on the result of the block verification, the SBL will be activated by a subsequent request of routine “Activate Secondary Bootloader”.

When the bootloader receives a request to start the erase routine, callback function ApplFblInvalidateBlock will be executed ❷. The implementation has to ensure that the validity information is cleared for the respective logical block. After returning from ApplFblInvalidateBlock, the bootloader calls the flash driver’s erase routine.
The sequence of invalidation and flash erasure will be repeated for every request to start the erase routine – depending on the amount of erase entries within the VBF file header. Consequently, it has to be considered that function ApplFblInvalidateBlock might be called repeatedly.

After the erasure has completed successfully, the bootloader will be ready to receive data. The tester will send a download request for every single block of data (**Any more RequestDownload SID 0x34?**). After each block, service RequestTransferExit will be called and the bootloader will conclude all active operations for this block (e.g. memory programming, hash calculation).

The loop illustrated with the green arrow (**More data files to download?**) is the loop for the files. How often it is executed depends on the amount of files you want to use for download.

Every single software unit that is downloaded will be validated separately. If the tester requests routine CheckMemory, user function ApplFblValidateBlock will be called ❸. This function should perform the following steps:

- Check the validity information of the logical block (e.g. presence pattern)
- If the logical block is marked as invalid, perform actual validation of the block content (e.g. using the validation structure)
- If validation result is positive, persist the validity information (e.g. using presence pattern)

After the download of the last software unit, the tester will request routine “Check Complete & Compatible”. On receiving this request, the bootloader will call user function AppFblValidateApp ❹. This function can be used to determine the overall validity of the ECU. In particular, this can be useful in multi-processor configurations or if complex dependencies between the downloaded software parts have to be checked at the end of the download. The result of this check has to be persisted in non-volatile memory so it can be read out by ApplFblIsValidApp during ECU startup.

After ApplFblValidateApp returns, callback function ApplFblIsValidApp will be called at the end of the routine to check if all required software units are present and compatible ❺. The result is included by the response to the tester.

**Info**
Routine “Check Complete & Compatible” is requested after the download of one or more VBF files or at any other time that the tester needs to find out if the application is valid. If it is not requested by the tool at the end of the download, then the tool cannot expect any of the downloaded files to be validated.

Each download sequence ends with a request to reset the ECU. Following reset, the bootloader will start up and check the validity status of the application software by calling function ApplFblIsValidApp ❻. The implementation has to check the information inside the validation area (see chapter 5.3) and return the result to the bootloader which will eventually start the application or stay in the bootloader.

## 5.7 Presence Pattern

As described in chapter 5.3 Handling of Validity Information, the validity state has to be stored in nonvolatile memory to allow for a short system startup time. This chapter provides a description of the default mechanism which is used by the bootloader.

Since not all bootloaders have access to an EEPROM device to store the validity status, a presence pattern mechanism has been chosen which stores this information inside the reprogrammable application area.

The validity information storage is handled with two patterns of the same size: the presence pattern and the corresponding mask. Address and size of pattern and mask is hardware dependent (see chapter 4.2.17 Presence Pattern).

**Reference**
In addition to the storage of the validity state, the VolvoCar bootloader also uses this pattern mechanism to persist the ‘erased’ state of a logical block. See chapter 4.6.2 Flash Erased Detection.

As a consequence, the memory area which will be reserved for this purpose is twice the size of Figure 5-2 shown below. Have a look at fbl_nvpattern_oem.h in folder BSW\Fbl for details.

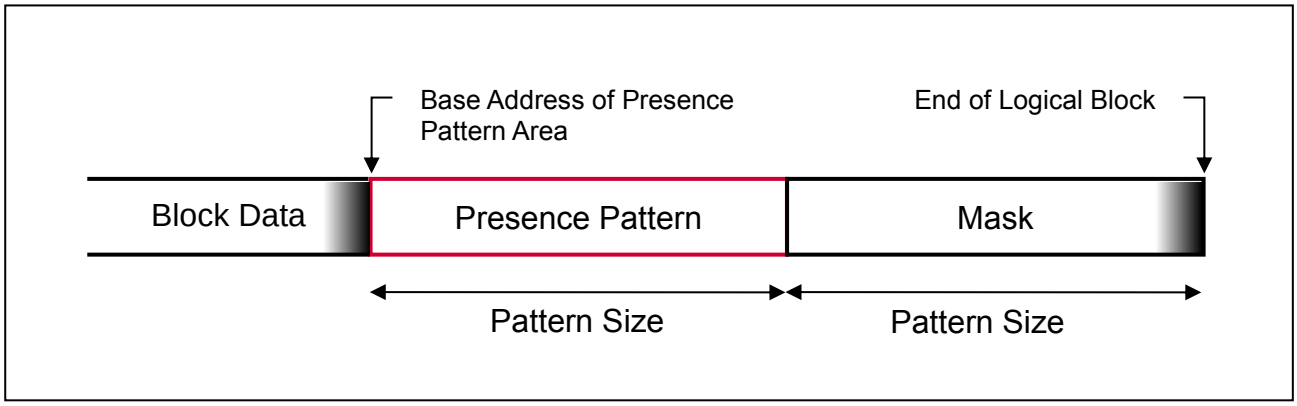



Figure 5-2 Illustration of the Presence Pattern Mechanism

The reason for having an additional mask (and not only a single presence pattern) is because most flash manufacturers do not allow for writing to a flash cell two times without a preceding erase. While the presence pattern will be written at the end of the download (if the block content is determined valid), the mask will be kept in ‘erased’ state. This allows inverting the mask value for invalidation just *before* the next erase procedure.

The presence pattern area (consisting of pattern and mask) is always located at the end of a logical block (see Figure 5-2). The following actions will be executed on the presence pattern area during a normal download procedure:

- Invalidation: Prior to the block erase operation, the mask value is written to the inverse of the erased state. This makes the block status invalid without re-writing to the pattern location.
- Erasure: The presence pattern area of one or more logical blocks is erased and thus the content is reverted to the flash erased value.
- Validation: Based on the result of the block validation check (ApplFblValidateBlock), the presence pattern is written to its location. Since both, the presence pattern and the mask have been erased; it is possible to re-write the pattern. The mask remains in erased state to allow the invalidation at a later date.
- Validity Check: The application software is treated as present and valid if the presence patterns of all mandatory logical blocks have been written and the respective masks are in erased state. The recognition of a valid/invalid status is based on a conjunction of the erased status of the mask and the pattern value.



**Caution**
If you want to use the presence pattern mechanism which is provided with the bootloader, implementation and configuration have to be carefully reviewed to verify the correctness for the intended purpose.

In particular, the following points should be ensured:

- The presence pattern area must be located in a memory region which will never be used (i.e. occupied) by the application software or data. The default implementation locates the presence pattern at the very end of each logical block and protects it against the download of data into this memory area.
- It has to be ensured that the presence pattern area is included to the flash memory erasure which precedes any reprogramming operation which is applied to the ECU. If this condition is not fulfilled, the valid pattern write operation will fail (at the end of a successful download).

## 5.8 Validation Structure

The “Validation Structure” option provides an advanced mechanism how to determine if the downloaded software/data is valid or not.

### 5.8.1 General Concept

By default, each software unit that is downloaded to the bootloader will be verified using a validation structure. A single software unit should align with a logical block as defined in

the bootloader configuration (see chapter 4.5). Each software unit contains a so-called “validation structure”, that lists all data blocks that are part of the software unit.

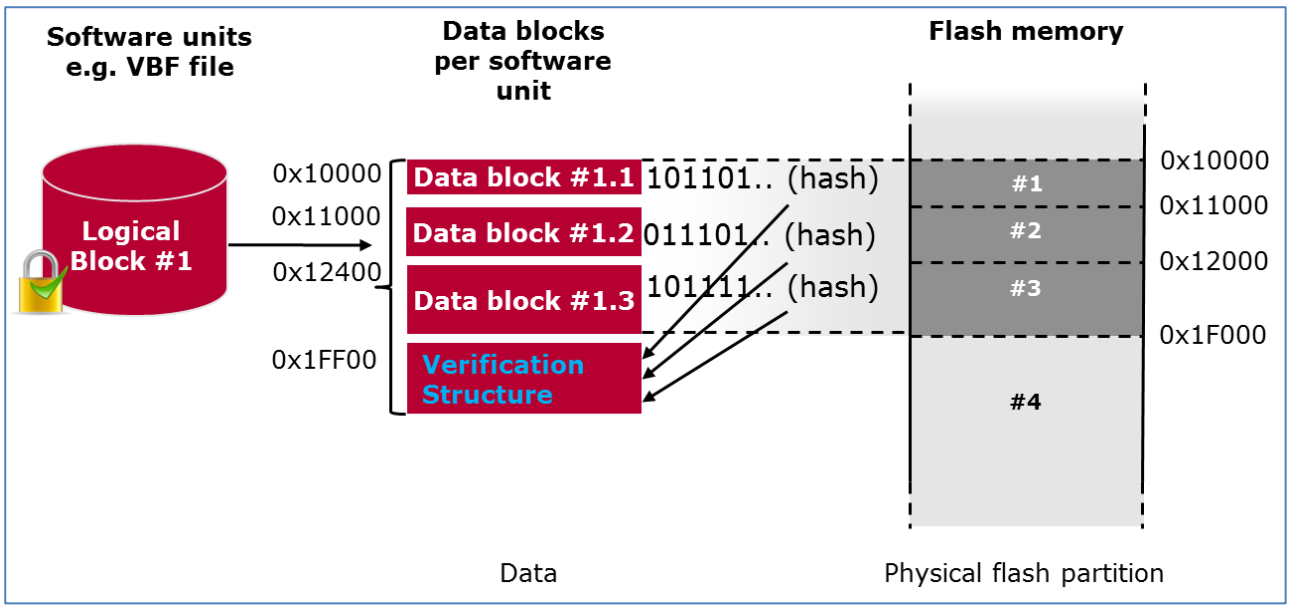



Figure 5-3 Definition: Software Units, Logical Blocks, Data Blocks

**FAQ**
To avoid misunderstandings, this document avoids the usage of the term “data blocks” in favor of “data segments” (or “download segments”).

In detail, the validation structure contains the following data:

- Number of data segments which are represented by the verification structure
- Start address, length and hash value of every data segment

The actual verification of the downloaded data follows these steps:

- The validation structure itself will be verified using the RSA signature passed by the RoutineControl/CheckMemory request
- The content of the verification structure will be used to verify the memory content of the respective logical block
- If all mandatory logical blocks have been verified successfully, the bootloader will start the application software at ECU startup



**Caution**
This concept implies that the verification structure contains all memory regions that will be written to the logical block. Data segments which are downloaded (as part of the VBF) but not contained in the verification structure cannot be verified by the bootloader.

The format of the data structure which will be recognized and evaluated by the implementation is detailed below. Note that all multi-byte records have to be stored in **big endian format** (byte order) regardless of the used hardware platform!

The actual size of the validation structure will vary depending on the number of application/data blocks which shall be downloaded/validated. Per default, the maximum size of the validation structure can be determined according to the following formula:

```
ValidationStructMaxSize = 4 + (MaximumNumberOfSegments - 1) * 40
```

So if the Maximum Number of Segments is set to default value 16, the maximum validation structure size will be 604 byte. Note that the bootloader will abort the evaluation of the validation structure if the maximum number of blocks which has been defined at configuration time is exceeded.

To create or update the validation structure in an efficient way, an appropriate tooling is required. With Hexview, Vector includes a versatile and free tool which fulfills this requirement. Hexview is located within folder Misc\Hexview of the delivered bootloader package.



**Note**
The validation structure has to be downloaded as a **separate download segment**. No other data is allowed directly before or after the structure. Consequently, a download file which contains a validation structure will always consist of at least two download segments.

### 5.8.2 Validation Structure Format

| Address Offset | Size [byte] | Record |
|---|---|---|
| Prolog (the validation structure starts at the address which is configured in the LBT | | |
| 0x00 | 2 | Structure version (currently fixed to 0x0000) |
| 0x02 | 2 | Block Count: Number of blocks which are described in the block info section |
| Block Info Section (i...block index, each block record has a size of 10 byte) | | |
| 0x06 + (i * 40) | 4 | Block Address: Start address of the download block which shall be checked |
| 0x0A + (i * 40) | 4 | Block Size: Size of the download block which shall be checked |
| 0x0E + (i * 40) | 32 | Block Hash: 32 byte SHA256 hash value of the download block which shall be checked |

Table 5-2 Format of the Validation Structure for Security Class CCC

The entry “Header Address” in the logical block table will always point to the start address of the validation structure. This should be kept in mind for the memory layout of the ECU, so that there resides no overlapping application code or data in this memory area.

Up to the present, the validation structure and the respective signature cannot be generated via the Hexview GUI. The bootloader delivery includes a batch script to perform this task in a convenient way. The following chapter provides usage instructions for this script.

### 5.8.3 Batch Scripts for Validation Structure Generation

The delivery contains two different batch scripts which support the generation of the VBF files with included validation structure/signature. One script supports the generation of an SBL VBF container, while the other one is an example for the generation of the application software/data VBF container.

#### 5.8.3.1 Validation Structure Generation for SBL/Flash Driver

The batch script to generate an SBL specific VBF container including the validation structure is located in folder BSW\Flash\Build. The name of the script is HexView_CreateVbfContainer.bat and it takes a single HEX file as input parameter. You can also drag & drop a HEX file onto the batch script.

The following configuration options have to be configured inside the batch file:

| Option | Description |
|---|---|
| HEXVIEW | Path to HexView binary (usually located in folder Misc\HexView). |
| ECU_ADDRESS | CAN Rx ID of the ECU (0x7??). See chapter 10.1 for details about the CAN IDs. |
| HDR_ADDR | Start address of the SBL/flash driver. Can be retrieved by looking at the start address of FlashDrv.hex in the same folder. |
| VS_ADDR | Location of the validation structure |
| SIG_PARAM | Path to the key file which contains the public key. This is the same key file which is used by the configuration tool. An example key is provided in folder Demo\DemoKeys. |

Table 5-3 Configuration Parameters of VBF Generation Script for SBL

The batch script will generate the following files to target folder BSW\Flash:

| File | Description |
|---|---|
| FlashDrv_sig.vbf | Flash driver with validation structure (security class CCC) |
| FlashDrv_cmpr_sig.vbf | Compressed flash driver with validation structure (security class CCC) |

Table 5-4 Output Files of VBF Generation Script for SBL

#### 5.8.3.2 Validation Structure Generation for Application Software/Data

The batch script to generate an application software/data VBF container including the validation structure is located in folder BSW\Misc\VbfGen. The name of the script is

HexView_CreateVbfContainer.bat and it takes a single HEX file as input parameter. You can also drag & drop a HEX file onto the batch script.

The following configuration options have to be configured inside the batch file:

| Option | Description |
|---|---|
| HEXVIEW | Path to HexView binary (usually located in folder Misc\HexView). |
| ECU_ADDRESS | CAN Rx ID of the ECU (0x7??). See chapter 10.1 for details about the CAN IDs. |
| SEG_SIZE | Memory write segment size (minimum programming length of the memory device). The configured value will be used to align the data inside the input HEX file. |
| SIG_PARAM | Path to the key file which contains the public key. This is the same key file which is used by the configuration tool. An example key is provided in folder Demo\DemoKeys. |
| Logical Block Parameters (x = 1..3) | |
| Bx_ADDRESS | Start address of the logical block |
| Bx_LENGTH | Length of the logical block |
| Bx_VS_ADDR | Location of the block specific validation structure |

Table 5-5 Configuration Parameters of VBF Generation Script for Application Software/Data

The script supports the configuration of up to three logical blocks. Please refer to the settings in DaVinci Configurator or the generated Fbl_Lbt.c for the respective information. The target block for the HEX file which shall be converted is configured by setting variable ‘BLOCK’, e.g.:

```
set BLOCK=B1
```

The batch script will generate the following files to target folder BSW\Flash:

| File | Description |
|---|---|
| <name of input file>_sig.vbf | Application software/data with validation structure (security class CCC) |
| <name of input file>_cmpr_sig.vbf | Compressed application software/data with validation structure (security class CCC) |

Table 5-6 Output Files of VBF Generation Script for SBL

## 5.9 [#oem_time] Validation OK – Application Faulty

For more general information about this see the UserManual_FlashBootloader in the chapter **Validation OK – Application faulty.**

A common feature of Vector bootloaders is the ability to stay in the bootloader even if the operating software is ready to run (ApplFblIsValidApp() returns kApplValid). VolvoCar requires that this feature be turned on at all times (both during development and production).

For example, if you download an operating software module that contains an issue that causes the ECU to reset. The result is that you will be unable to replace the module, since the FBL will start the operating software, which then resets. This starts the FBL, which starts the Operating System again, over and over. In this situation, there is no means to start a download.

To escape this situation, the stay-in-boot feature inserts a small delay between the time the ECU is reset, and the time the FBL starts the operating software. If a “ping” message is received during this delay, the FBL will stay in its main loop instead of starting the operating software. Once the FBL is executing its main loop, a new file may be downloaded normally.

For VolvoCar, stay-in-boot is mandatory. Therefore, the respective configuration option is enabled per default (see chapter 4.2.22 Stay in Boot).

When enabled, the FBL will respond to a StartDiagnosticSession ($10) service request containing the Session ID of $02. This message can be sent functionally or physically with the suppressed bit clear or set. The message must be sent less than 5 milliseconds after resetting the ECU.

# 6 Interaction with the Application

## 6.1 FBL Header Files

There are some header files which have to be included by the application code to have access to the bootloader interface.

| Header File | Description |
|---|---|
| fbl_def.h | Basic FBL types and macros |
| Fbl_Cfg.h | FBL configuration (implicitly included by fbl_def.h) |
| comdat.h | Common Data Structure (optional); see chapter 6.3 |

Table 6-1 FBL Header Files to be included by the Application

fbl_def.h has to be included in the diagnostic callback function file of the application to provide the access to the `CallFblStart()` macro. fbl_def.h in turn includes Fbl_Cfg.h, therefore that file will need to be part of the include path of the application.

## 6.2 FBL Version Information

The application may need to extract the bootloader version for diagnostic purposes. The FBL provides a general header structure that contains the FBL version information. To be able to read the FBL version from the application, the following macros are provided by fbl_def.h:

| Macro | Description |
|---|---|
| `GetFblMainVersion()` | FBL main version number |
| `GetFblSubVersion()` | FBL sub version number |
| `GetFblReleaseVersion()` | FBL release version number |
| `GetFblBuildVersion()` | FBL build version number |

Table 6-2 FBL Version Information Macros

To be able to use these macros, make sure that configuration option “Header Address” (see chapter 4.2.8) is set up properly. The FBL header address configuration must correspond to the linker mapping of the FBL header structure.

## 6.3 Common Data Structure

VolvoCar mandates that certain data identifiers (DIDs) are reported by both bootloader and application software. In this case, the respective DID values shall be gathered from the same storage location. To provide a convenient way of accessing ECU specific DID values which are stored within the bootloader area, the common data structure has been introduced.

The template file _comdat.h (see chapter 3.1.2 Application Files) provides an initial data structure (type definition tFblCommonData) which comprises all DIDs that are reported by the default implementation of the bootloader. The definition of the respective data structure values is done within file fbl_ap.c (constant fblCommonData). Thus, the structure and the included data can be changed or extended according to the user requirements.

In order to use the common data structure from within the application software, simply add an #include instruction for comdat.h to the module which shall be able to access the shared data. Note that the bootloader file fbl_def.h has to be included too and that the presence of Fbl_Cfg.h is implicitly required (these files are also used for the transition from application to bootloader; see chapter 6.1 FBL Header Files).



**Example**

The following code shows an example on how to access the common data structure from within the application software.

```
#include "fbl_def.h"
#include "comdat.h"

void DESC_API_CALLBACK_TYPE
ApplDescReadDID_F111_ECU_Core_Assembly_Number(DescMsgContext*
pMsgContext)
{
  vuint8 i;

  for (i = 0; i < 24; i++)
  {
    pMsgContext->resData[i] =
fblCommonData.EcuCoreAssemblyNumber[i];
  }
  pMsgContext->resDataLen = 24;

  /* User service processing finished. */
  DescProcessingDone(DESC_CONTEXT_PARAM_WRAPPER_ONLY(pMsgContext-
>iContext));
}
```

## 6.4 [#oem_trans] Transitioning from Application to Bootloader

VolvoCar requires that the application always handles the response to transition to the programming session of the flash bootloader. If the suppressed bit is set in the request, then the application simply transitions to the bootloader without sending a response. If the suppressed bit is clear in the request, then the application is required to transmit the response and then transition to the flash bootloader. This chapter focuses on the different options available to perform the transition.

**Info**
The time delay between the reception of the service request “DiagnosticSession-Control” and the execution of the reset, respectively the time until the bootloader can accept a next service request **shall not exceed 500ms**.

The bootloader allows the user to transition from the application to the bootloader via two methods:

> Call `FblStart()`: With this method, the FBL provides the `CallFblStart()` macro which will jump to the bootloader function fbl_main.c::`FblStart()`. This function will set a pattern in RAM and issue a reset. After the reset, the bootloader will be started in programming session if the pattern is detected. This method can be chosen by enabling configuration option “Start Function” (see chapter 4.2.21).

> Start over Reset: Force a reset to switch to the bootloader. The FBL does not provide the FblStart function, so it is the user’s responsibility to set the external reprogramming request flag in a fixed known location, issue a reset and check the flag after startup in function fbl_apnv.c::`ApplFblExtProgRequest()`. This method can be chosen by disabling configuration option “Start Function” (see chapter 4.2.21).

### 6.4.1 Method 1: Call FblStart()

This is the simplest approach to enter the programming session from the application. The application uses the macro `CallFblStart()` to transition to the bootloader. The bootloader then sets a flag in RAM and performs a reset. After the reset, the bootloader can detect the reprogramming request by checking the flag, and sets the state of the bootloader to programming mode to prevent starting the application.

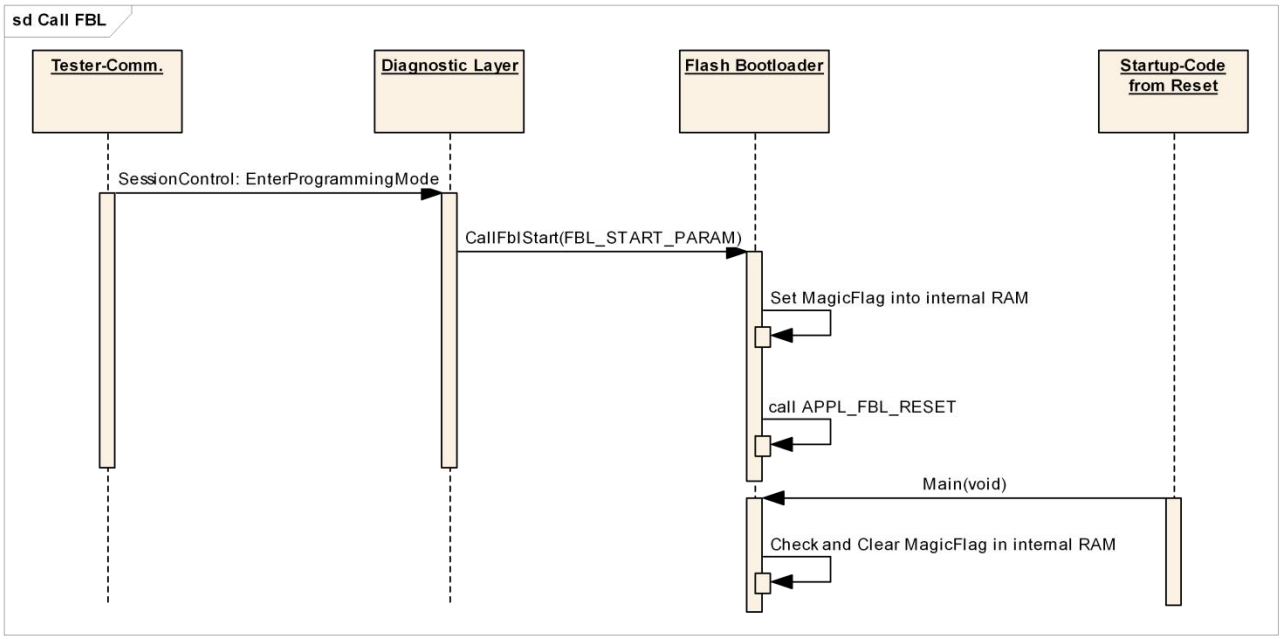



Figure 6-1 Transition from Application to Bootloader via FblStart()

### Bootloader Configuration

Enable configuration option “Start Function” (chapter 4.2.21) and check the correct setting of option “Header Address” (chapter 4.2.8). Generate the configuration and take care that bootloader and application use the same versions of the generated files.

When this option is enabled, the function `FblStart()` will be defined which allows the usage of the macro `CallFblStart()`. The macro `CallFblStart()` and its parameter “FBL_START_PARAM” is defined in fbl_def.h.

When using this option, care must be taken to ensure that the start-up code of the bootloader does not initialize the magic flags that indicate the start from application (`fblStartMagicFlag[0-7]`).



**Caution**
Do not use `CallFblStart()` if configuration option “Start Function” is not enabled (FBL_ENABLE_FBL_START has NOT been generated into Fbl_Cfg.h).

### User Callback Implementation

When the bootloader is started via `CallFblStart()`, the callback function `ApplFblExtProgRequest()` needs to check for the internal reprogramming flags in RAM. If detected, the function will clear the flags and set the bootloader mode to START_FROM_APPL. That will cause the bootloader to stay in programming session to allow the tester to reprogram the ECU as shown below:

```
tFblProgStatus ApplFblExtProgRequest( void )
{
#if defined(FBL_ENABLE_FBL_START)
   if (FblChkFblStartMagicFlag() == 0)
   {
      /* No programming request was received */
      return kNoProgRequest;
   }
   else /* MagicFlags are set */
   {
      /* Clear magic flags */
      FblClrFblStartMagicFlag();
      /* Indicate that FBL started from APPL */
      SetFblMode(START_FROM_APPL);

      /* Application already handled the response, */
      /* the bootloader does not send any response */
      return kProgRequestNoResponse;
   }
#endif
}
```

When a reprogramming request is detected, the function above returns ”kProgRequestNoResponse”. This is necessary to prevent the bootloader from responding to the programming request, because it’s assumed that the application is handling the response.

### 6.4.2 Method 2: Start over Reset

Another option to transition to the bootloader is for the application to generate the reset instead of using macro `CallFblStart()`. In this case, the application must set a reprogramming request in non-volatile memory (e.g. EEPROM) or a non-initialized shared RAM location, before generating the reset.

Below is a chart that describes the sequence of events to transition to the bootloader via a reset:

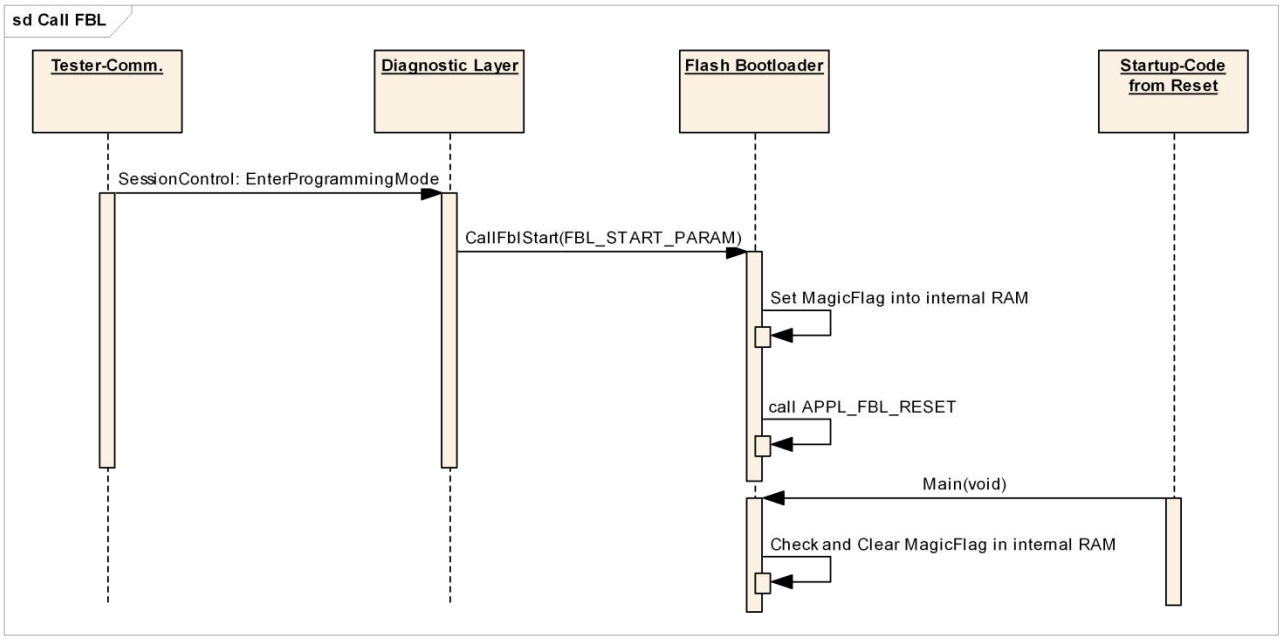



Figure 6-2 Transition from Application to Bootloader via Reset

#### Bootloader Configuration

Disable configuration option “Start Function” (see chapter 4.2.21).

**Caution**
Do not use macro `CallFblStart()` if configuration option “Start Function” is disabled (FBL_DISABLE_FBL_START is set in Fbl_Cfg.h)!

#### User Callback Implementation

When the bootloader is started via a reset, the callback function `ApplFblExtProgRequest()` needs to check for the reprogramming flag in non-volatile memory. If detected, the function will clear the flag and set the bootloader mode to “START_FROM_APPL”. That will cause the bootloader to stay in programming session to allow the tester to reprogram the ECU as shown below:

```
tFblProgStatus ApplFblExtProgRequest( void )
{
   if (ApplFblCheckMagicFlag() == 0)
   {
      /* No programming request was received */
      return kNoProgRequest;
   }
   else /* MagicFlags are set - jump from application was called!
*/
   {
      /* Clear magic flags */
      ApplFblClearMagicFlag();
      /* Indicate that FBL started from APPL */
      SetFblMode(START_FROM_APPL);

      /* Application already handled the response, */
      /* the bootloader does not send any response */
      return kProgRequestNoResponse;
   }
}
```

Note that when a reprogramming request is detected, the function above returns ”kProgRequestNoResponse”. This is necessary to prevent the bootloader from responding to the programming request, because it’s assumed that the application is handling the response.

`ApplFblCheckMagicFlag()` and `ApplFblClearMagicFlag()` are two functions to be provided by the user.



**Caution**
It’s important to clear the flags immediately after checking to ensure that subsequent resets are not interpreted falsely as programming requests from the application.

### 6.4.3 Conclusion

As you consider the different options available, below are some hints to help you make the decision.

Advantage of using the start over reset to transition to the bootloader:

- Greater flexibility to adjust the jump to the bootloader
- No care must be taken to use uninitialized RAM with the startup-code

Disadvantage:

- Shared location for application and bootloader must be defined in the EEPROM

> Driver to handle EEPROM must be added to the bootloader.

> Must be implemented by the user

## 6.5 Starting the Application from the Bootloader

At the end of each download, the bootloader will check the validity of the downloaded code/data (see chapter 5, Validation / Invalidation of Flashed Application). The result of this check has to be stored in non-volatile memory, from where the bootloader will extract the information at ECU startup. Afterwards, an ECU reset is initiated.

During ECU startup, the bootloader is always started first. The bootloader will check if a reprogramming request from the application is pending. If this is not the case, the bootloader enters a time limited loop which will check for the reprogramming message (see chapter 5.9, Validation OK - Application Faulty).

If no message has been received, the bootloader calls user function fbl_apnv.c::ApplFblIsValidApp() in order to check the validity state of the application. In case the function returns positive (valid application present), the bootloader will call user function fbl_ap.c::ApplFblStartApplication(). The default implementation of this function will use the entry address provided in variable `nvApplStartAddr` to start the application. Depending on the bootloader configuration, the variable will be set…

- …according to the information provided in the complete & compatible block
- …to the entry/reset address which is stored in the application vector table

# 7 Flash Driver

The flash driver consists of basic routines for programming and erasing flash memory. The Vector implementation of the flash code is based on the HIS interface standard. Please see [8] for more details on the HIS interface.

On most hardware platforms, code cannot be executed in flash memory while performing a flash operation. Therefore the flash driver has to be copied to a RAM buffer prior to flash code execution.

The VolvoCar download specification requires downloading the flash code into RAM and activating it using the RoutineControl ”Activate SBL” routine. The flash code is then available in RAM until an ECU reset occurs, or a DiagnosticSessionControl request is received to switch to programming or default sessions. In both cases, the SBL is deactivated either by being cleared from RAM, or by the reset that clears the activation flag.



**Info**
Throughout this document, the term “Secondary Bootloader” (SBL) is always used synonym for the flash driver. Unlike the definition of the SBL in the VolvoCar Software Download Specification, the SBL/flash driver which is provided by Vector does not contain any diagnostic services but only the flash driver code. However, the diagnostic services which are restricted to the SBL are protected from being executed until the SBL is activated.
This design decision has been taken to reduce the size of the SBL and the required download time.

## 7.1 Flash Driver Functions

For the API description, please refer to Flash Driver API

When writing or erasing non-volatile memory is required in a user-callback function, the memory driver interface shall be used, instead of the direct call to the flash interface functions.

The memory driver interface is described in the next section.

## 7.2 Memory Driver Interface

The VolvoCar flash bootloader can support multiple memory devices by using a standard memory device interface defined by the HIS group. When a request to write, read or erase a memory region is received, the bootloader calls the corresponding function in the fbl_mio interface. Using the FlashBlock table, the fbl_mio can look up the corresponding function to be executed for that particular memory type. For e.g. if address region 0x10000-0x13FFF is defined as external flash in the FlashBlock table, and a request is received to erase that region, the fbl_mio will call the erase function of the external flash driver to perform this operation. When erasing or writing to non-volatile memory in a user-callback function, the user must pay attention to initialize the global variable memSegment prior to calling any write, read or erase function. memSegment is used by the fbl_mio to lookup the

proper memory driver, so without it, the fbl_mio could call the wrong function which would lead to a failure. To initialize memSegment, you can call FblMemSegmentNrGet(sourceAddress), where sourceAddress is the start address of the region which is being operated on, for e.g.:

```
memSegment = FblMemSegmentNrGet(sourceAddress);
if (memSegment >= 0)
/* sourceAddress belongs to the FlashBlock table */
{
   flashErrorCode =
MemDriver_RWriteSync(&programData[bufferIndex],
   currentLength, programAddress);

   if (flashErrorCode != kFlashOk)
   {
      /* A write error occured return an error code */
   }
}
```



**Caution**
Note that any memory operation such as erase, write or read, should be done on regions of homogeneous memory types, i.e. the bootloader cannot erase a region that starts in internal flash for e.g. and ends in external flash. To erase such a region, separate function calls are needed for each memory type.

# 8 Security Module

## 8.1 Components

The Security Module (secmod) is integrated with the bootloader as a separate component. The following figure depicts the hierarchy and the dependencies of the modules.

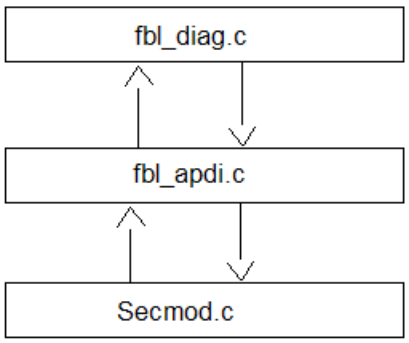



Figure 8-1 Security Module Dependencies

The following table describes the basic functionality of the different modules.

| Module | Description |
|---|---|
| fbl_diag | Module implements diagnosis service functions. Security Access requests are dispatched through this module as well as the response |
| fbl_apdi | Contains the user configurable callback functions to handle the seed calculation and the key validation |
| secmod | Security Module, contains the VolvoCar Specific seed and key algorithms |

Table 8-1 Description of Files Involved in Security

### 8.1.1 [#oem_sec] Security Access Diagnostic Service

The security access service follows the procedure in the table below:

| Step | Tester | ECU |
|---|---|---|
| 1. | RequestSeed | |
| 2. | | Generate Random Seed |
| 3. | | Send Seed |
| 4. | Calculate Key From Seed | Calculate Key From Seed |
| 5. | Send Key | |
| 6. | | Compare Calculated and Received Key |

| Step | Tester | ECU |
|---|---|---|
| 7. | | Send Response |
| 8. | | Set security Level (lock/unlock) |

Table 8-2 Steps of the SecurityAccess Service

The VolvoCar specific security access algorithm is pre-implemented in callback functions ApplFblSecuritySeed and ApplFblSecurityKey:

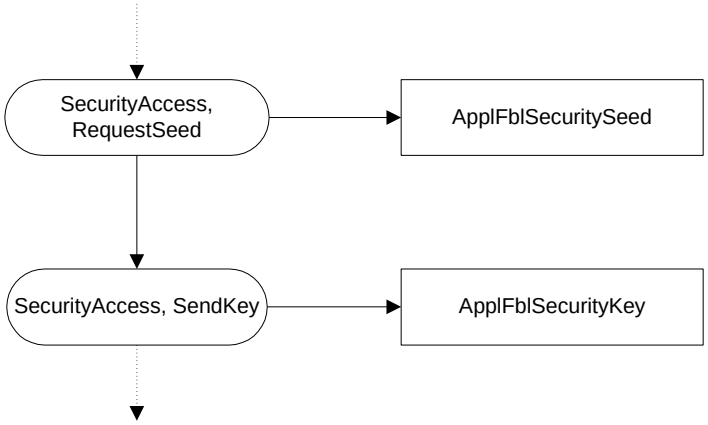



Figure 8-2 Seed Generation and Key Verification with the Security Access Procedure

When the tester requests the seed value by sending the SecurityAccess request message to the ECU, the diagnostic layer (fbl_diag) checks the request, decides whether it's a seed or key request and calls the corresponding callback function. In case of a seed request, the `ApplFblSecuritySeed()` function is called to generate a seed value. The default implementation performs this task by calling `SecM_GenerateSeed()` in the security module.

**Note**
The default implementation of the bootloader uses a linear congruential generator (LCG) to produce pseudo-randomized numbers as basis for the seed. Entropy is generated using a free-running hardware timer in combination with the indication of received messages from the communication bus.

Please make sure that this implementation fulfills the requirements for your project.

The software download specification requires the use of security level one for programming purposes. User specific security levels are currently not supported. Please get in contact with Vector if this is a requirement for your project.

Next, the generated seed is transmitted back to the tester in the positive response message. The generated seed is stored temporarily by the user specific code to be able to verify the security key which is sent to the ECU afterwards.

When the tester sends a key request, the diagnostic layer will call `ApplFblSecurityKey()`. There, the user shall calculate the key based on the existing seed value and compare that to the key sent by the tester in the key request. If they match, then the user shall return a positive response and access is granted. Otherwise, access is denied and a negative response is sent back to the tester. When calculating the key, the function `SecM_CompareKey()` uses the OEM specific security algorithm to do so.



**Note**
The security module (secmod.c) is delivered and integrated with the bootloader. This means that the seed and key functionality is provided and no further user adaptation shall be required to comply with the VolvoCar security requirements.

## 8.2 Integration and Configuration

To activate the seed/key module in the bootloader, configuration option “Security Module” (chapter 4.6.6) has to be enabled. In addition to enabling security, the security constant bytes which are unique to the ECU (described in reference [5]), have to be defined as well. See configuration option Secret Key (chapter 4.6.5).

# 9 API Description

## 9.1 Flash Driver Interface

### 9.1.1 Overview

All flash driver interface functions are implemented in the fbl_flio.c file. This file is not user modifiable. These functions are only interface functions that do not contain code for flash programming, but call the real flash driver functions located in RAM.

| Function API | Location |
|---|---|
| IO_ErrorType FlashDriver_InitSync( void* address ) | fbl_flio.c |
| IO_ErrorType FlashDriver_DeinitSync( void* address ) | fbl_flio.c |
| IO_ErrorType FlashDriver_RWriteSync( unsigned char* writeBuffer, IO_SizeType writeLength, IO_PositionType writeAddress ) | fbl_flio.c |
| IO_ErrorType FlashDriver_REraseSync(IO_SizeType eraseLength, IO_PositionType eraseAddress) | fbl_flio.c |
| IO_ErrorType FlashDriver_RReadSync( IO_MemPtrType readBuffer, IO_SizeType readLength, IO_PositionType readAddress ) | fbl_flio.c |

Table 9-1 Flash Driver Interface

### 9.1.2 Function Description

#### 9.1.2.1 FlashDriver_InitSync

| Prototype | |
|---|---|
| `IO_ErrorType FlashDriver_InitSync( void* address )` | |
| **Parameter** | |
| `address` | Start address of SBL is passed to the flash driver in case needed for initialization purposes |
| **Return code** | |
| `IO_E_OK`<br>`IO_E_NOT_OK` | Flash driver initialization successful<br>Initialization failure. There may be other return codes other than kFlashOk on different hardware platforms |
| **Functional Description** | |
| After this function is called, the flash driver API is available for flash operations. | |
| **Particularities and Limitations** | |
| - | |

#### 9.1.2.2 FlashDriver_DeinitSync

| Prototype |
|---|
| `IO_ErrorType FlashDriver_DeinitSync( void* address )` |

| Parameter | |
|---|---|
| address | Currently not used by the bootloader |
| **Return code** | |
| IO_E_OK | De-initialization succeeded |
| IO_E_NOT_OK | De-Initialization failed. There may be other return codes other than kFlashOk on different hardware platforms |
| **Functional Description** | |
| This function is called to remove the flash driver code from RAM and to perform hardware-specific de-initialization operations if necessary. | |
| **Particularities and Limitations** | |
| - | |

#### 9.1.2.3 FlashDriver_RWriteSync

| Prototype | |
|---|---|
| `IO_ErrorType FlashDriver_RWriteSync( IO_MemPtrType writeBuffer, IO_SizeType writeLength, IO_PositionType writeAddress )` | |
| **Parameter** | |
| writeBuffer | Pointer to source data buffer |
| writeLength | Number of bytes to be programmed |
| writeAddress | Target address in flash memory |
| **Return code** | |
| IO_E_OK | Flash programming succeeded |
| IO_E_NOT_OK | Flash programming failure. There may be other return codes other than kFlashOk on different hardware platforms |
| **Functional Description** | |
| Function for flash programming. The data bytes in the writeBuffer are programmed to the target address. While programming, the watchdog support function is called within the specified call-period. | |
| **Particularities and Limitations** | |
| The parameter values have to be aligned to the flash segment size; otherwise there will be an error code in return. | |

#### 9.1.2.4 FlashDriver_RReadSync

| Prototype | |
|---|---|
| `IO_ErrorType FlashDriver_RReadSync( IO_MemPtrType readBuffer, IO_SizeType readLength, IO_PositionType readAddress)` | |
| **Parameter** | |
| readBuffer | Pointer to target data buffer |
| readLength | Number of bytes to be read |
| readAddress | source address in flash memory |

| Return code | |
|---|---|
| IO_E_OK | Flash read operation succeeded |
| IO_E_NOT_OK | Flash read operation failure. |
| IO_E_ERASED | Flash read operation failure (memory seems to be erased) |

| Functional Description |
|---|
| Certain flash types cannot be accessed by direct memory access; instead they require a specific operation to read data. This function is designed to do that and to provide a uniform interface for accessing memory. While reading data, the watchdog support function is called within the specified call-period. |

| Particularities and Limitations |
|---|
| The parameter values have to be aligned to the flash segment size; otherwise an error might be generated during the read operation. |

### 9.1.2.5 FlashDriver_REraseSync

| Prototype |
|---|
| `IO_ErrorType FlashDriver_REraseSync (IO_SizeType eraseLength, IO_PositionType eraseAddress)` |

| Parameter | |
|---|---|
| eraseLength | Number of bytes to be erased |
| eraseAddress | Erase start address |

| Return code | |
|---|---|
| IO_E_OK | Erase process succeeded |
| IO_E_NOT_OK | Error occurred during erase procedure |

| Functional Description |
|---|
| The given flash memory area is erased by this function. While erasing, the watchdog support function is called within the specified call-period. |

| Particularities and Limitations |
|---|
| Depending on the hardware platform, flash can only be erased within the physical restrictions of the current flash technology. That means that there is a minimum amount of flash that has to be erased at once. Also the erase start address must correspond to physical restrictions. |

## 9.2 Callback-Function API



**Caution**
The implementation of the provided user callback functions which are provided in the bootloader demo are for demonstration purposes only. You MUST review the default implementation and adapt it based on your requirements and hardware setup.

### 9.2.1 Overview

| Function API |
|---|
| fbl_ap.c |
| void ApplFblInit( void ) |
| void ApplFblStartup(vuint8 initposition) |
| void ApplTrcvrNormalMode( void ) |
| void ApplFblSetVfp( void ) |
| void ApplFblResetVfp( void ) |
| void ApplFblReset( void ) |
| void ApplFblTask( void ) |
| void ApplFblCanBusOff( void ) |
| void ApplFblFatalError( const vuint8* module, vuint16 line, vuint8 errorCode ) |
| void ApplFblStartApplication( void ) |
| tFblResult ApplFblInitDataProcessing( tProcParam *procParam ) |
| tFblResult ApplFblDataProcessing( tProcParam *procParam ) |
| tFblResult ApplFblDeinitDataProcessing( tProcParam *procParam ) |
| fbl_apdi.c |
| vuint8 ApplFblSecurityInit( void ) |
| vuint8 ApplFblSecuritySeedInit( void ) |
| vuint8 ApplFblSecuritySeed( void ) |
| vuint8 ApplFblSecurityKey( void ) |
| void ApplDiagUserSubfunction( vuint8 *pbDiagData, tTpDataType diagReqDataLen ) |
| void ApplDiagUserService( vuint8 *pbDiagData, tTpDataType diagReqDataLen ) |
| void ApplDiagRoutineControl( vuint8 *pbDiagData, tTpDataType diagReqDataLen ) |
| vuint8 ApplFblReadDataByIdentifier(vuint8 *pbDiagData, tTpDataType diagReqDataLen ) |
| vuint8 ApplFblWriteDataByIdentifier(vuint8 *pbDiagData, tTpDataType diagReqDataLen ) |
| fbl_apnv.c |
| tFblProgStatus ApplFblExtProgRequest( void ) |
| tApplStatus ApplFblIsValidApp( void ) |
| tFblResult ApplFblValidateBlock( tBlockDescriptor blockDescriptor ) |

| |
|---|
| void ApplFblInvalidateBlock( tBlockDescriptor blockDescriptor ) |
| tFblResult ApplFblValidateApp( void ) |
| vuint8 ApplFblSetSecAccessDelayFlag( void ) |
| vuint8 ApplFblGetSecAccessDelayFlag( void ) |
| vuint8 ApplFblClrSecAccessDelayFlag( void ) |
| vuint8 ApplFblReadSecAccessInvalidCount( vuint8 * invalidCnt ) |
| vuint8 ApplFblWriteSecAccessInvalidCount( vuint8 * invalidCnt ) |
| tFblResult ApplFblGetBlockErased( void ) |
| void ApplFblReadSecurityBytes( vuint8 * key, vuint16 length ) |
| tFblResult ApplFblWriteSecurityBytes( vuint8 * key, vuint16 length ) |
| tFblResult ApplFblReadPublicKey( SecM_AsymKeyType * publicKey ) |
| tFblResult ApplFblReadPublicKeyChecksum( vuint8 * buffer, vuint16 length ) |
| tFblResult ApplFblWritePublicKey( vuint8 * key ) |
| static tFblResult ApplFblVerifyPublicKeyChecksum(vuint8 * key, vuint8 * checksum) |
| tFlashStatus ApplFblOtpGetMemAreaErased( tFblAddress memAddress, tFblLength memLength ) |
| static IO_ErrorType ApplFblOtpReadEntry(tFblAddress address, tFblLength length, vuint8 * readBuffer) |
| static tFblResult ApplFblOtpWriteEntry(tFblAddress address, tFblLength length, vuint8 * writeBuffer) |
| static vsint16 ApplFblGetPresencePatternBaseAddress( vuint8 blockNr, IO_PositionType * pPresPtnAddr, IO_SizeType * pPresPtnLen ) |
| tFblResult ApplFblAdjustLbtBlockData( tBlockDescriptor * blockDescriptor ) |
| tFblResult ApplFblGetPatternState(tBlockDescriptor * blockDescriptor, tPatternId patternId) |
| tFblResult ApplFblClrPatternState( tBlockDescriptor * blockDescriptor, tPatternId patternId ) |
| tFblResult ApplFblSetPatternState( tBlockDescriptor * blockDescriptor, tPatternId patternId ) |
| tApplStatus ApplFblSetCcReturnValue( vuint32 blockState ) |
| tFblResult ApplFblGetCcBlockInfo( tFblAddress ccbAddr, tFblAddress * ccFuncAddr, tFblAddress * applStartAddr ) |
| tApplStatus ApplFblCheckCompatibility( void ); |
| fbl_apwd.c |
| void ApplFblWDInit( void ) |
| void ApplFblWDShort( void ) |
| void ApplFblWDLong( void ) |
| void ApplFblWDTrigger( void ) |

Table 9-2 Callback Function API

### 9.2.2 Function Description

#### 9.2.2.1 ApplFblInit

| Prototype | |
|---|---|
| `void ApplFblInit( void )` | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| The function ApplFblInit is called after reset. You can do basic initializations here, like memory mapping, setting up the PLL, etc. Usually the watchdog is also initialized here. | |
| **Particularities and Limitations** | |
| - | |

#### 9.2.2.2 ApplFblStartup

| Prototype | |
|---|---|
| void ApplFblStartup(vuint8 initposition) | |
| **Parameter** | |
| `kStartupPreInit` | Used to indicate that the function was called before `FblInit()` was executed |
| `kStartupPostInit` | Used to indicate that the function was called after `FblInit()` was executed |
| `kStartupStayInBoot` | Function is called if the start message has been received (stay-in-boot only) |
| **Return code** | |
| – | - |
| **Functional Description** | |
| This function is called before and after the bootloader timer and CAN controller are setup in the function `FblInit()`. This is useful for hardware which may only be initialized once, e.g. hardware watchdogs. The function is called with the parameter(kStartupPreInit) before the call to `FblInit()` and with (kStartupPostInit) after the call to `FblInit()`. | |
| **Particularities and Limitations** | |
| Due to the mandatory activation of stay-in-boot, this function will be called every time the bootloader starts up, even if a valid application is present. However, the function will only be called with parameter 'kStartupStayInBoot' if the start message has been actually received. | |

#### 9.2.2.3 ApplTrcvrNormalMode

| Prototype |
|---|
| void ApplTrcvrNormalMode( void ) |

| Parameter | |
|---|---|
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| In connection with the initialization of the CAN interface the FBL calls this function to switch the CAN transceiver to normal operating mode. | |
| **Particularities and Limitations** | |
| - | |

#### 9.2.2.4 ApplFblSetVfp

| Prototype | |
|---|---|
| void ApplFblSetVfp( void ) | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| The FBL calls this function when programming mode is entered to switch on the programming voltage. This operation depends on the flash hardware. If the flash memory does not require external programming voltage, this function may remain empty. | |
| **Particularities and Limitations** | |
| - | |

#### 9.2.2.5 ApplFblResetVfp

| Prototype | |
|---|---|
| void ApplFblResetVfp( void ) | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| After download, this function is called to turn off the programming voltage. As with ApplFblSetVfp, it depends on the flash memory hardware if some action is necessary in this function. | |
| **Particularities and Limitations** | |
| - | |

#### 9.2.2.6 ApplFblReset

| Prototype | |
|---|---|
| void ApplFblReset( void ) | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| This function performs a reset (depending on the hardware used). This can be done e.g. by using the watchdog | |
| **Particularities and Limitations** | |
| - | |

#### 9.2.2.7 ApplFblTask

| Prototype | |
|---|---|
| void ApplFblTask( void ) | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| Optional cyclic function that is called with the same period as FblDiagTask usually at a 2ms rate. | |
| **Particularities and Limitations** | |
| This function is enabled only if the configuration option ”Application Task” (chapter 4.3.5) is enabled. | |

#### 9.2.2.8 ApplFblCanBusOff

| Prototype | |
|---|---|
| void ApplFblCanBusOff( void ) | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| This call-back function is called if the CAN controller enters bus-off state | |
| **Particularities and Limitations** | |
| - | |

### 9.2.2.9 ApplFblSecuritySeedInit

| Prototype | |
|---|---|
| `vuint8 ApplFblSecuritySeedInit( void )` | |
| **Parameter** | |
| – | |
| **Return code** | |
| `kFblOk` | Seed initialization successful |
| `kFblFailed` | Seed initialization failed |
| **Functional Description** | |
| This function can be used to initialize seed values. For example, a random number generator can be started here. This function will be called, when a diagnostic message has been received by the bootloader. | |
| **Particularities and Limitations** | |
| > The return value will be ignored. | |
| Call context | |
| > fbl_diag.c::`FblDiagRxIndication()` | |

### 9.2.2.10 ApplFblSecuritySeed

| Prototype | |
|---|---|
| vuint8 ApplFblSecuritySeed(void) | |
| **Parameter** | |
| – | |
| **Return code** | |
| `kFblOk` | Seed generation successful |
| `kFblFailed` | Seed generation failed |
| **Functional Description** | |
| This function is called when the tester requests a seed for the security access service. The function has to generate a seed value and write this into the diag buffer. The generated seed also has to be stored internally for the verification of the key. | |
| **Particularities and Limitations** | |
| > It's not allowed to send RCRRP messages from within this function (e.g. by calling FblRealTimeSupport or DiagExRCRResponsePending)! | |

### 9.2.2.11 ApplFblSecurityKey

| Prototype | |
|---|---|
| vuint8 ApplFblSecurityKey(void) | |
| **Parameter** | |
| – | |

| Return code | |
|---|---|
| kFblOk | Key value from tester accepted, security access shall be granted by FBL |
| kFblFailed | Key value not accepted, ECU shall remain locked |
| **Functional Description** | |
| This function compares the receive key with the internally computed key. | |
| **Particularities and Limitations** | |
| > It's not allowed to send RCRRP messages from within this function (e.g. by calling FblRealTimeSupport or DiagExRCRResponsePending)! | |

### 9.2.2.12 ApplFblSecurityInit

| Prototype | |
|---|---|
| void ApplFblSecurityInit(void) | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | |
| **Functional Description** | |
| Initializes the security state and security level. | |
| **Particularities and Limitations** | |
| - | |

### 9.2.2.13 ApplDiagUserSubFunction

| Prototype | |
|---|---|
| void ApplDiagUserSubFunction( vuint8 *pbDiagData, tTpDataType diagReqDataLen ) | |
| **Parameter** | |
| pbDiagData | Pointer to received data (service ID not included) |
| diagReqDataLen | Number of received data bytes (service ID not included) |
| **Return code** | |
| – | |
| **Functional Description** | |
| This function is called if a diagnostic request with a supported service ID but an unsupported subfunction is received by the FBL. The function code has to examine the subfunction ID of the requesting service and handle the user subfunction IDs which are not supported by the bootloader by default. For unsupported subfunction IDs, the negative response 'Subfunction Not Supported' (0x12) has to be sent. | |

| Particularities and Limitations |
|---|
| > This function is used only if the configuration option “User Subfunction” (chapter 4.2.27) is enabled.<br>> Note that the user function will not be called for service RoutineControl.<br>> The diagnostic service ID can be extracted by using macro `DiagGetRequestSId()`.<br>> To account for the bootloader’s background tasks (e.g. watchdog service and handling of RCRRP), the user implementation has to call `FblRealTimeSupport()` cyclically during the runtime of this function.<br>> DiagProcessingDone(<response length w/o SID>) has to be called for successfully executed user subfunctions.<br>> Use the DiagNRC* macros provided by fbl_diag.h to flag an error and send a negative response. |

### 9.2.2.14 ApplDiagUserService

| Prototype | |
|---|---|
| void ApplDiagUserService( vuint8 *pbDiagData, tTpDataType diagReqDataLen ) | |
| **Parameter** | |
| `pbDiagData` | Pointer to received data (service ID not included) |
| `diagReqDataLen` | Number of received data bytes (service ID not included) |
| **Return code** | |
| – | |
| **Functional Description** | |
| This function is called for received diagnostic service requests which are not handled by the FBL. The function code has to examine the service ID and handle the user service IDs which are not supported by the bootloader by default. For unsupported service IDs, the negative response ‘Service Not Supported’ (0x11) has to be sent (use the provided macro DiagNRCServiceNotSupported). | |
| **Particularities and Limitations** | |
| > This function is used only if the configuration option “User Service” (chapter 4.2.26) is enabled.<br>> The diagnostic service ID can be extracted by using macro `DiagGetRequestSId()`.<br>> To account for the bootloader’s background tasks (e.g. watchdog service and handling of RCRRP), the user implementation has to call `FblRealTimeSupport()` cyclically during the runtime of this function.<br>> DiagProcessingDone(<response length w/o SID>) has to be called for successfully executed user services.<br>> Use the DiagNRC* macros provided by fbl_diag.h to flag an error and send a negative response. | |

### 9.2.2.15 ApplDiagRoutineControl

| Prototype | |
|---|---|
| void ApplDiagRoutineControl( vuint8 *pbDiagData, tTpDataType diagReqDataLen ) | |
| **Parameter** | |
| `pbDiagData` | Pointer to received data (service ID not included) |

| | |
|---|---|
| diagReqDataLen | Number of received data bytes (service ID not included) |
| **Return code** | |
| – | |
| **Functional Description** | |
| This function is called for RoutineControl requests with an unsupported routine ID (i.e. the routine ID is not handled by the default implementation of the bootloader). The function code has to examine (at least) the following items and send a negative response using the provided DiagNRC* macros if applicable:<br>> Routine ID: For unsupported routine IDs, the negative response 'Request Out Of Range' (0x31) has to be sent (DiagNRCRequestOutOfRange).<br>> Subfunction/routine control type: For unsupported subfunction/routine control type values, the negative response 'Subfunction Not Supported' (0x12) has to be sent (DiagNRCSubFunctionNotSupported).<br>> Request length: If the request length does not comply with the expected value, the negative response 'Incorrect Message Length Or Invalid Format' (0x13) has to be sent (DiagNRCIncorrectMessageLength¬OrInvalidFormat). | |
| **Particularities and Limitations** | |
| > This function is used only if the configuration option "User Routine" (chapter 4.2.25) is enabled.<br>> The template file _fbl_apdi.c contains an example for the handling of (optional) routine "Update Software Download Security Bytes". The actual storage of the security bytes in non-volatile memory has to be added by the user.<br>> To account for the bootloader's background tasks (e.g. watchdog service and handling of RCRRP), the user implementation has to call `FblRealTimeSupport()` cyclically during the runtime of this function. | |

### 9.2.2.16 ApplFblReadDataByIdentifier

| **Prototype** | |
|---|---|
| vuint8 ApplFblReadDataByIdentifier( vuint8 *pbDiagData, tTpDataType diagReqDataLen) | |
| **Parameter** | |
| pbDiagData | Pointer to diag service data: pbDiagData points to the location in the diag buffer where the DID is located - i.e. the actual DID data starts at offset +2. |
| diagReqDataLen | Describes the actual data length of the requested DID (not including the 2-byte DID number!) |
| **Return code** | |
| kFblOk | DID supported by user implementation, the respective data has been inserted into the diag buffer |
| kFblFailed | DID not supported by user implementation or error during retrieval of the respective data |

| Functional Description |
|---|
| This function is called by the bootloader:<br>> When a ReadDataByIdentifier request for an unsupported DID is received from the tester (i.e. the DID is not handled by the diagnostic core implementation)<br>> The DID cannot be found in fblDidTable (located in fbl_apdi.c)<br><br>For unsupported DIDs, “kFblFailed” has to be set as return code.<br>See chapter 9.4.1 ApplFblReadDataByIdentifier for further implementation hints. |
| **Particularities and Limitations** |
| > It’s not allowed to set a NRC in this function; just set the return code to “kFblFailed” if the DID cannot be handled<br>> It’s not allowed to send RCRRP messages from within this function (e.g. by calling FblRealTimeSupport or DiagExRCRResponsePending)! |

### 9.2.2.17 ApplFblWriteDataByIdentifier

| Prototype | |
|---|---|
| vuint8 ApplFblWriteDataByIdentifier( vuint8 *pbDiagData, tTpDataType diagReqDataLen) | |
| **Parameter** | |
| `pbDiagData` | Pointer to received data (service ID not included); pbDiagData points to the location in the diag buffer where the DID is located - i.e. the actual DID data starts at offset +2 |
| diagReqDataLen | Describes the actual data length of the requested DID plus the 2-byte DID number |
| **Return code** | |
| `kFblOk` | DID supported by user implementation, the respective data has been written to the target memory |
| `kFblFailed` | DID not supported by user implementation, incorrect format/length or error during write operation |

| Functional Description |
|---|
| This function is called by the bootloader when a WriteDataByIdentifier request for an unsupported DID is received from the tester (i.e. the DID is not handled by the default implementation of the bootloader). The function code has to examine (at least) the following items and send a negative response using the provided DiagNRC* macros if applicable:<br>> DID: For unsupported DIDs, the negative response ‘Request Out Of Range’ (0x31) has to be sent (DiagNRCRequestOutOfRange).<br>> Request format/length: If the request format/length does not comply with the respective DID specification, the negative response ‘Incorrect Message Length Or Invalid Format’ (0x13) has to be sent (DiagNRCIncorrectMessageLengthOrInvalidFormat).<br>> DID specific: Check if security access has been unlocked. If the ECU is still locked, the negative response ‘Security Access Denied’ (0x33) has to be sent (DiagNRCSecurityAccessDenied).<br>> Driver for target memory present and initialized: Macro `GetMemDriverInitialized()` can be used to check this condition.<br>> Result of write operation: If writing to the target memory failed, negative response code ‘General Programming Failure’ (0x72) has to be sent (DiagNRCGeneralProgrammingFailure).<br>See chapter 9.4.1 ApplFblWriteDataByIdentifier for further implementation hints. |

| Particularities and Limitations |
|---|
| > This function is used only if the configuration option WriteDataByIdentifier Service (chapter 4.6.7) is enabled.<br>> To account for the bootloader's background tasks (e.g. watchdog service and handling of RCRRP), the user implementation has to call `FblRealTimeSupport()` cyclically during the runtime of this function.<br>> In case 'kFblFailed' shall be returned, the user implementation is responsible for setting the corresponding negative response code (use DiagNRC* macros for this purpose). Failing to do so will result in an assertion and the core implementation will ignore the return value. |

### 9.2.2.18 ApplFblExtProgRequest

| Prototype | |
|---|---|
| tFblProgStatus ApplFblExtProgRequest(void) | |
| **Parameter** | |
| – | |
| **Return code** | |
| `kProgRequest` | External reprogramming request |
| `kNoProgRequest` | No external reprogramming request |
| **Functional Description** | |
| This function is called after reset to check if a reprogramming request is available. If this function returns kProgRequest, the FBL does not jump to the application. | |
| **Particularities and Limitations** | |
| > If configuration option Start Function is disabled, the macros `FblChkFblStartMagicFlag()` and `FblClrFblStartMagicFlag()` have to be implemented by the user.<br>> Macro `ApplFblChkWarmStart()` is specific to the used hardware and has to be implemented by the user according to the VCC requirements (see [3]). | |

### 9.2.2.19 ApplFblIsValidApp

| Prototype | |
|---|---|
| tApplStatus ApplFblIsValidApp( void ) | |
| **Parameter** | |
| – | |
| **Return code** | |
| `kApplValid` | Valid application available |
| `!= kApplValid` | No valid application available. Return code `kApplInvalid` means undetermined complete & compatible result (e.g. complete & compatible block not found at the specified address). The return value consists of 32 bit which have to report the actual software state. See [3] for details. |

| Functional Description |
|---|
| This function is called after reset to determine if a valid application is available to be started. For this purpose, the validity status is read from the Validation Area.<br>If this function returns kApplInvalid the ECU remains in the FBL. |
| **Particularities and Limitations** |
| > This function is called during execution of service RoutineControl/CheckCompleteCompatible (routine ID 0x0205), too. The result will be reported in the service response.<br>> The implementation should take into account that this function will be called during every ECU startup. In particular, the runtime of the function should be kept as short as possible to conform with the OEM specific startup time constraints. In this context, it's not allowed to send RCRRP messages from within this function (e.g. by calling FblRealTimeSupport or DiagExRCRResponsePending)!<br>> Note that the function shall have a maximum runtime of 20ms according to the VCC specifications. If the complete and compatible functionality is used (see chapter Validation of Flashed Application), the complete and compatible function has to be aborted after 15ms. This requires some kind of timeout observation which is currently not included with the Vector bootloader. |

### 9.2.2.20 ApplFblValidateBlock

| Prototype | |
|---|---|
| tFblResult ApplFblValidateBlock( tBlockDescriptor blockDescriptor ) | |
| **Parameter** | |
| `blockDescriptor` | Contains information about the logical block which shall be validated (see tBlockDescriptor, chapter 10.4.1.5). |
| **Return code** | |
| `kFblOk` | Block information storage/validation successfully performed |
| `kFblFailed` | Block information storage/validation failed |
| **Functional Description** | |
| This function is called during service RoutineControl/CheckMemory to perform the validation of the respective logical block (SBL or non-volatile memory block). The default implementation uses an embedded validation structure to verify the block content and stores the resulting block validity using presence pattern.<br>Generally, the function should perform these two steps:<br>> Check the validity of the download (e.g. compare checksum/hash)<br>> Store the validity information inside the Validation Area. The validity information will be read by function ApplFblIsValidApp. | |
| **Particularities and Limitations** | |
| > To account for the bootloader's background tasks (e.g. watchdog service and handling of RCRRP), the user implementation has to call FblRealTimeSupport() cyclically during the runtime of this function. | |

### 9.2.2.21 ApplFblInvalidateBlock

| Prototype |
|---|
| tFblResult ApplFblInvalidateBlock( tBlockDescriptor blockDescriptor ) |

| Parameter | |
|---|---|
| `blockDescriptor` | Contains information about the logical block which shall be invalidated (see tBlockDescriptor, chapter 9.3.1.7). |
| **Return code** | |
| `kFblOk` | Invalidation successfully performed, perform erasure |
| `kFblFailed` | Invalidation failed, omit erasure |
| **Functional Description** | |
| This function is called before the FBL erases (flash) memory contents. The block descriptor parameter provides information about the logical block which shall be invalidated. | |
| **Particularities and Limitations** | |
| > The sequence of invalidation and flash erasure will be repeated for every request to start the erase routine - depending on the amount of erase entries within the VBF file header. Consequently, it has to be considered that function ApplFblInvalidateBlock might be called repeatedly.<br>> To account for the bootloader's background tasks (e.g. watchdog service and handling of RCRRP), the user implementation has to call `FblRealTimeSupport()` cyclically during the runtime of this function.<br>> In case 'kFblFailed' shall be returned, the user implementation is responsible for setting the corresponding negative response code (use DiagNRC* macros for this purpose). Failing to do so will result in an assertion and the core implementation will ignore the return value.<br>> The user implementation has to take into account that returning 'kFblFailed' will result in omission of the actual erase operation. Consequently, if the failure to invalidate is permanent, it will be impossible to erase the respective memory area!<br>> Some controllers (e.g. V850, 78K0R) feature a hardware specific verification process which has to be executed after programming the flash memory. If this verification process fails, the bootloader will try to invalidate the downloaded software/data by calling `ApplFblInvalidateBlock()` for every logical block. | |

### 9.2.2.22 ApplFblValidateApp

| Prototype | |
|---|---|
| tFblResult ApplFblValidateApp( void ) | |
| **Parameter** | |
| – | |
| **Return code** | |
| `kFblOk` | Validation successfully performed |
| `kFblFailed` | Validation failed (this might be due to the download being invalid or a general error during the execution of the function) |
| **Functional Description** | |
| This function can be used to determine the overall validity of the ECU. In particular, this can be useful in multi-processor configurations or if complex dependencies between the downloaded software parts have to be checked at the end of the download. The result of this check has to be persisted in non-volatile memory so it can be read out by ApplFblIsValidApp during ECU startup. | |

| Particularities and Limitations |
|---|
| > Because the SWDL specification mandates that routine CheckCompleteCompatible shall be executable in the PBL (i.e. without activated SBL), the user is responsible for checking if the respective memory driver is available prior to any write/erase operation (e.g. for writing a valid pattern).<br>> To account for the bootloader's background tasks (e.g. watchdog service and handling of RCRRP), the user implementation has to call FblRealTimeSupport() cyclically during the runtime of this function.<br>> Depending on the return code and if a negative response code (NRC) has been set (by using the DiagNRC* macros), the bootloader will take different actions:<br>  > kFblOk: download is valid, conclude service processing (call ApplFblIsValidApp and send response)<br>  > kFblFailed, no NRC set: download is invalid, conclude service processing (call ApplFblIsValidApp and send response)<br>  > kFblFailed, NRC set: an error occurred during validity check or validity information update – abort service processing and send a negative response with the specified NRC. |

### 9.2.2.23 ApplFblGetBlockErased

| Prototype | |
|---|---|
| tFlashStatus ApplFblGetBlockErased ( tBlockDescriptor blockDescriptor ) | |
| **Parameter** | |
| `blockDescriptor` | Contains information about the memory area which shall be erased (see tBlockDescriptor, chapter 9.3.1.7). |
| **Return code** | |
| `kFblOk` | Flash memory is in initial erased state. The memory erase routine will not be executed! |
| `kFblFailed` | Flash memory has been written before. The memory erase routine will be executed as usual. |

| Functional Description |
|---|
| Function is called by the diagnostic layer to check if the erase procedure can be omitted or not. |

| Particularities and Limitations |
|---|
| > This function is used only if the configuration option Flash Erased Detection (chapter 4.6.2) is enabled. Reading chapter 9.4.6 Flash Erased Detection is an indispensable precondition for the application of this feature!<br>> Depending on the amount of erase block entries within the VBF file header, it has to be considered that function `ApplFblGetBlockErased()` might be called repeatedly. See chapter 11.5 Flash Erased Detection for details.<br>> The runtime of the function should be kept as short as possible (considerably smaller than the P2 time) to avoid having to start the RCRRP handling. |

### 9.2.2.24 ApplFblReadSecurityBytes

| Prototype | |
|---|---|
| `void` **`ApplFblReadSecurityBytes`**`( vuint8 * key, vuint16 length )` | |
| **Parameter** | |
| `key` | Target pointer to key buffer |
| `length` | Requested key length |

| Return code | |
|---|---|
| – | - |
| **Functional Description** | |
| Allow user specific provision of the ECU's secret key. The key can be read from any non-volatile memory. The default implementation uses the secret key that's configured in DaVinci Configurator (see chapter 4.6.5). | |
| **Particularities and Limitations** | |
| > The secret key may be updatable by means of a diagnostic service request (e.g. RoutineControl). This service can be implemented in fbl_apdi.c::ApplDiagRoutineControl().<br>> If the secret key is updatable and the value hasn't been written yet, the function shall report all security bytes with value 0xFF. | |
| Call context | |
| > ApplFblSecurityKey | |

### 9.2.2.25 ApplFblWriteSecurityBytes

| Prototype | |
|---|---|
| `tFblResult` **`ApplFblWriteSecurityBytes`**`( vuint8 * key, vuint16 length )` | |
| **Parameter** | |
| `key` | Target pointer to key buffer |
| `length` | Requested key length |
| **Return code** | |
| `kFblOk` | The security bytes have been written successfully. |
| `kFblFailed` | There was an error during writing of the security bytes. An error shall be returned if the security bytes have already been written (NRC 0x31 RequestOutOfRange) or if the write operation itself fails (NRC 0x72 GeneralProgrammingFailure). |
| **Functional Description** | |
| Allow user specific storage of the ECU's secret key. The key can be written to any non-volatile memory. The default implementation writes the secret key to the OTP area. | |
| **Particularities and Limitations** | |
| > The default implementation currently doesn't use these API functions.<br>> The secret key may be updatable by means of a diagnostic service request (e.g. RoutineControl). This service can be implemented in fbl_apdi.c::ApplDiagRoutineControl().<br>> The secret key update has been designed as a one-time programmable (OTP) mechanism. The implementation has to ensure that the key can only be programmed once. All further attempts shall yield NRC 0x31 (RequestOutOfRange). | |

### 9.2.2.26 ApplFblReadPublicKey

| Prototype |
|---|
| `tFblResult ApplFblReadPublicKey( SecM_AsymKeyType * publicKey )` |

| Parameter | |
|---|---|
| publicKey | Target pointer to key buffer |
| **Return code** | |
| kFblOk | Key available in buffer |
| kFblFailed | Read operation failed, buffer content not valid |
| **Functional Description** | |
| Allow user specific provision of the public key for production. The key can be read from any non-volatile memory. The default implementation uses the public key that's configured in DaVinci Configurator (see chapter 4.6.5). | |
| **Particularities and Limitations** | |
| > The public key is updatable by means of a diagnostic service request (WriteDataByIdentifier, DID 0xD01C).<br>> If the key has not been programmed yet (return code 'kFblFailed'), the built-in public key will be used for verification. | |
| Call context | |
| > Validation structure handling (fbl_valstruct) | |

### 9.2.2.27 ApplFblReadPublicKeyChecksum

| Prototype | |
|---|---|
| tFblResult ApplFblReadPublicKeyChecksum( vuint8 * buffer, vuint16 length ) | |
| **Parameter** | |
| buffer | Target pointer to checksum buffer |
| length | Size of buffer (maximum expected checksum length) |
| **Return code** | |
| kFblOk | Checksum available in buffer |
| kFblFailed | Read operation failed, buffer content not valid |
| **Functional Description** | |
| Provide the checksum of the programmed public key (production). This information is used by ReadDataByIdentifier DID 0xD01C to indicate a programmed public key (production). | |
| **Particularities and Limitations** | |
| > The public key is updatable by means of a diagnostic service request (WriteDataByIdentifier, DID 0xD01C).<br>> If the key has not been programmed yet, return code 'kFblFailed' has to reflect that (the bootloader will report NRC 0x22 (ConditionsNotCorrect) in this case; see implementation of function ApplFblReadDataByIdentifier). | |
| Call context | |
| > ReadDataByIdentifier (DID 0xD01C) | |

### 9.2.2.28 ApplFblWritePublicKey

| Prototype | |
|---|---|
| `tFblResult ApplFblWritePublicKey( vuint8 * key )` | |
| **Parameter** | |
| `key` | Target pointer to key buffer |
| **Return code** | |
| `kFblOk` | The public key (production) has been written successfully. |
| `kFblFailed` | There was an error during writing of the public key. An error shall be returned if the public key has already been written (NRC 0x31 RequestOutOfRange) or if the verification of the public key checksum failed (NRC 0x22 ConditionsNotCorrect). |
| **Functional Description** | |
| Allow user specific storage of the public key (production). The key can be written to any non-volatile memory. The default implementation writes the public key to the OTP area. | |
| **Particularities and Limitations** | |
| > The function is called when a WriteDataByIdentifier request for DID 0xD01C has been received (fbl_apdi.c::ApplFblWriteDataByIdentifier).<br>> The public key update has been designed as a one-time programmable (OTP) mechanism. The implementation has to ensure that the key can only be programmed once. All further attempts shall yield NRC 0x31 (RequestOutOfRange). | |

### 9.2.2.29 ApplFblVerifyPublicKeyChecksum

| Prototype | |
|---|---|
| `static tFblResult ApplFblVerifyPublicKeyChecksum(vuint8 * key, vuint8 * checksum)` | |
| **Parameter** | |
| `key` | Target pointer to key buffer |
| `checksum` | Target pointer to checksum buffer |
| **Return code** | |
| `kFblOk` | Checksum matches |
| `kFblFailed` | Checksum comparison failed |
| **Functional Description** | |
| Verify the passed public key by usage of the provided checksum. | |
| **Particularities and Limitations** | |
| > The checksum will be calculated by using the same security module functions which are used to calculate the block specific SHA256 value.<br>> The checksum has to be verified before the new public key is written to non-volatile memory. | |
| Call context | |
| > WriteDataByIdentifier (DID 0xD01C) | |

### 9.2.2.30 ApplFblOtpGetMemAreaErased

| Prototype | |
|---|---|
| `tFlashStatus ApplFblOtpGetMemAreaErased( tFblAddress memAddress, tFblLength memLength )` | |
| **Parameter** | |
| `memAddress` | Non-volatile memory address inside the OTP area |
| `memLength` | Length of the memory region inside the OTP area which shall be checked |
| **Return code** | |
| `kFlashErased` | Memory area are in state 'erased' |
| `kFlashNotErased` | Non-erased memory area detected |
| **Functional Description** | |
| Check whether the given memory area is erased (contains the respective erased value). | |
| **Particularities and Limitations** | |
| > The memory drivers which are used by the bootloader will determine the ‘erased’ state based on the value of the respective memory location. There’s no guarantee that this memory location has never been written since the last erase operation. | |
| Call context | |
| > ApplFblOtpReadEntry | |

### 9.2.2.31 ApplFblOtpReadEntry

| Prototype | |
|---|---|
| `static IO_ErrorType ApplFblOtpReadEntry(tFblAddress address, tFblLength length, vuint8 * readBuffer)` | |
| **Parameter** | |
| `address` | Non-volatile memory address of the OTP entry which shall be read |
| `length` | Length of the data buffer which shall be used for the read operation |
| `readBuffer` | Pointer to the buffer which shall keep the data which has been read from the OTP area |
| **Return code** | |
| `IO_E_OK` | Read data available in buffer |
| `IO_E_NOT_OK` | Error occurred during memory read operation, buffer content invalid |
| `IO_E_ERASED` | OTP entry is still in erased state |
| **Functional Description** | |
| This function shall provide status and data of the requested OTP entry which is specified by parameter ‘address’. | |

| Particularities and Limitations |
|---|
| > The usage of the OTP area for data storage is subject to certain preconditions and restrictions which have to be considered before usage. See chapter 9.4.5 OTP Area for further information! |
| Call context |
| > All functionality that uses one-time programmable data storage (e.g. Write public key, Update Security Bytes) |

### 9.2.2.32 ApplFblOtpWriteEntry

| Prototype | |
|---|---|
| `static tFblResult ApplFblOtpWriteEntry(tFblAddress address, tFblLength length, vuint8 * writeBuffer)` | |
| **Parameter** | |
| `address` | Non-volatile memory address of the OTP entry which shall be read |
| `length` | Length of the data buffer which shall be used for the read operation |
| `writeBuffer` | Pointer to the buffer which shall keep the data which has been read from the OTP area |
| **Return code** | |
| `kFblOk` | The OTP entry has been written successfully. |
| `kFblFailed` | There was an error during writing of the OTP entry. An error shall be returned if the passed memory address is invalid or if the write operation itself fails. |
| **Functional Description** | |
| This function shall write the requested OTP entry which is specified by parameter ‘address’. | |
| **Particularities and Limitations** | |
| > The usage of the OTP area for data storage is subject to certain preconditions and restrictions which have to be considered before usage. See chapter 9.4.5 OTP Area for further information! | |
| Call context | |
| > All functionality that uses one-time programmable data storage (e.g. Write public key,, Update Security Bytes) | |

### 9.2.2.33 ApplFblGetCcBlockInfo

| Prototype | |
|---|---|
| `tFblResult ApplFblGetCcBlockInfo(FBL_ADDR_TYPE ccbAddr, FBL_ADDR_TYPE * ccFuncAddr, FBL_ADDR_TYPE * applStartAddr)` | |
| **Parameter** | |
| ccbAddr | Start address of complete & compatible block (input parameter) |
| ccFuncAddr | Start address of complete & compatible function (output parameter) |

| | |
|---|---|
| applStartAddr | Start address of application software (output parameter) |
| **Return code** | |
| kFblOk | Complete and compatible block is present/consistent, the complete & compatible function address and the application software start address is returned via the respective output parameters. |
| kFblFailed | Complete and compatible block is not present/corrupt, the output parameters are not updated. |
| **Functional Description** | |
| Check presence and integrity of the complete & compatible block. Return information that is contained within the structure. | |
| **Particularities and Limitations** | |
| > The function can only be used in module fbl_apnv.c as it is declared static. | |

### 9.2.2.34 ApplFblCheckCompatibility

| **Prototype** | |
|---|---|
| `tApplStatus` **`ApplFblCheckCompatibility`**`( void )` | |
| **Parameter** | |
| - | - |
| **Return code** | |
| `kApplValid` | Hardware and software compatibility check succeeded. |
| `!= kApplValid` | One or several software units (logical blocks) are not compatible with other units or with the hardware. The return value consists of 32 bit which have to report the actual software state. See [3] for details. |
| **Functional Description** | |
| User function which shall implement the hardware and software compatibility check according to REQPROD 53958.<br>If this function returns a value != kApplValid, the ECU remains in the FBL. | |
| **Particularities and Limitations** | |
| > This function is called by ApplFblIsValidApp() during execution of service RoutineControl/CheckCompleteCompatible (routine ID 0x0205) and during bootloader startup.<br>> The implementation should take into account that this function will be called during every ECU startup. In particular, the runtime of the function should be kept as short as possible to conform with the OEM specific startup time constraints. In this context, it's not allowed to send RCRRP messages from within this function (e.g. by calling FblRealTimeSupport or DiagExRCRResponsePending)!<br>> Note that the function shall have a maximum runtime of 20ms according to the VCC specifications. If the complete and compatible functionality is used (see chapter Validation of Flashed Application), the complete and compatible function has to be aborted after 15ms. This requires some kind of timeout observation which is currently not included with the Vector bootloader. | |

### 9.2.2.35 ApplFblSetCcReturnValue

| Prototype | |
|---|---|
| `tApplStatus ApplFblSetCcReturnValue ( void )` | |
| **Parameter** | |
| – | |
| **Return code** | |
| `<C&C return code>` | The return code according to REQPROD 53958. |
| **Functional Description** | |
| This function is used to map the logical block indices to the corresponding flags as specified by REQPROD 53958 (e.g. executable, signal configuration). | |
| **Particularities and Limitations** | |
| > The default implementation simply returns the indices of the invalid logical blocks (starting at bit 6). | |

### 9.2.2.36 ApplFblWDInit

| Prototype | |
|---|---|
| `void ApplFblWDInit( void )` | |
| **Parameter** | |
| – | - |
| **Return code** | |
| – | - |
| **Functional Description** | |
| Function is called always after reset, unless the reset was caused by a programming request. This function can be used to setup the watchdog. | |
| **Particularities and Limitations** | |
| - | |

### 9.2.2.37 ApplFblWDShort

| Prototype | |
|---|---|
| `void ApplFblWDShort( void )` | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| Function is called after a successful transition from the application to the FBL. It can be used to readjust the watchdog timeout if necessary. If the watchdog is setup in ApplFblWDInit, then this function must be used to setup the watchdog as well to ensure that the watchdog is configured properly after a programming request is received in the application. | |

| Particularities and Limitations |
| --- |
| - |

### 9.2.2.38 ApplFblWDLong

| Prototype | |
| --- | --- |
| `void ApplFblWDLong( void )` | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| Function is called before the FBL attempts to transition to the application. Use this function to readjust the watchdog timeout if necessary, or to synchronize the watchdog timer to a new watchdog cycle before transitioning to the application. That will prevent a watchdog timeout from occurring while the transition is taking place. | |
| **Particularities and Limitations** | |
| - | |

### 9.2.2.39 ApplFblWDTrigger

| Prototype | |
| --- | --- |
| `void V_CALLBACK_NEAR ApplFblWDTrigger( void )` | |
| **Parameter** | |
| – | |
| **Return code** | |
| – | - |
| **Functional Description** | |
| Function is called at the rate configured with configuration attribute Trigger Cycle. This function has to refresh the watchdog timer and may not contain any function calls, because it is copied to RAM to refresh the watchdog while flash programming is taking place. | |
| **Particularities and Limitations** | |
| - | |

### 9.2.2.40 ApplFblFatalError

| Prototype | |
| --- | --- |
| `Default` | `void ApplFblFatalError( vuint8 errorCode )` |
| `Extended Info` | `void ApplFblFatalError( const vuint8* module, vuint16 line, vuint8 errorCode )` |

| Parameter | |
|---|---|
| module | Name of the module in which the assertion occurred |
| line | Module line number where the assertion occurred |
| errorCode | Identification code of the encountered assertion |
| **Return code** | |
| – | - |
| **Functional Description** | |
| If the bootloader assertions are enabled, this function will be called in case an invalid bootloader state is encountered. | |
| **Particularities and Limitations** | |
| > See chapter 9.4.1 for details about Bootloader Assertions. | |
| Call context | |
| > The assertion handler will be called from various source code locations in case an erroneous bootloader state has been detected. | |

### 9.2.2.41 ApplFblInitDataProcessing

| Prototype |
|---|
| `tFblResult ApplFblInitDataProcessing( tProcParam *procParam )` |
| **Parameter** |
| procParam: Data processing parameter structure (see tProcParam, chapter 9.3.1.10) |
| **Return code** |
| kFblOk: RequestDownload service execution will be continued. |
| kFblFailed: RequestDownload service execution will be aborted and a negative response with NRC 0x31 (Request out of Range) will be issued. |
| **Functional Description** |
| This function is called to initialize the application specific data processing function (e.g. data decompression and/or decryption). |
| **Particularities and Limitations** |
| > This function is only provided if the decompression module is part of the bootloader package. In this case, the function will be used as a wrapper for the dedicated decompression module. |
| Call context |
| > Called during processing of a RequestDownload service request with a data format identifier value != 0x00. |

### 9.2.2.42 ApplFblDataProcessing

| Prototype |
|---|
| `tFblResult ApplFblDataProcessing( tProcParam *procParam )` |

| Parameter | |
|---|---|
| procParam | Data processing parameter structure (see tProcParam, chapter 9.3.1.10) |
| **Return code** | |
| kFblOk | TransferData service execution will be continued with the processed data. |
| kFblFailed | TransferData service execution will be aborted and a negative response with NRC 0x22 (Conditions not Correct) will be issued. |
| **Functional Description** | |
| This function is called to allow for user specific download data processing (e.g. data decompression and/or decryption). | |
| **Particularities and Limitations** | |
| > This function is only provided if the decompression module is part of the bootloader package. In this case, the function will be used as a wrapper for the dedicated decompression module.<br>> If a user specific data processing algorithm shall be implemented within this callback, function FblRealTimeSupport() has to be called cyclically to account for the background tasks of the bootloader (e.g. watchdog service and RCRRP handling). | |
| Call context | |
| > Called during processing of a TransferData service request which follows a RequestDownload with a data format identifier value != 0x00. The input data corresponds to the data which has been received with the TransferData request, the output data will be written to the target memory. | |

### 9.2.2.43 ApplFblDeinitDataProcessing

| Prototype | |
|---|---|
| tFblResult **ApplFblDeinitDataProcessing**( tProcParam *procParam ) | |
| **Parameter** | |
| procParam | Data processing parameter structure (see tProcParam, chapter 9.3.1.10) |
| **Return code** | |
| kFblOk | RequestTransferExit service execution will be continued. |
| kFblFailed | RequestTransferExit service execution will be aborted and a negative response with NRC 0x22 (Conditions not Correct) will be issued. |
| **Functional Description** | |
| This function is called so the data processing can be concluded. All remaining input data has to be processed and passed to the output buffer. | |
| **Particularities and Limitations** | |
| > This function is only provided if the decompression module is part of the bootloader package. In this case, the function will be used as a wrapper for the dedicated decompression module. | |
| Call context | |
| > Called during processing of a RequestTransferExit service request which follows a RequestDownload with a data format identifier value != 0x00. The input data corresponds to the remaining data which has not been processed yet, the output data will be written to the target memory. | |

### 9.2.2.44 ApplFblStartApplication

| Prototype | |
|---|---|
| `void ApplFblStartApplication( void )` | |
| **Parameter** | |
| - | |
| **Return code** | |
| - | - |
| **Functional Description** | |
| This function is called by the bootloader in order to the start the application software. | |
| **Particularities and Limitations** | |
| > Since the application software will be started from here, the function will never return.<br>> The default implementation of the function uses the entry address of the application which will be set by ApplFblIsValidApp. | |
| Call context | |
| > The function is called during bootloader startup if a valid application has been detected (ApplFblIsValidApp returned positive). | |

### 9.2.2.45 ApplFblGetPatternLocation

| Prototype | |
|---|---|
| `static vsint16 ApplFblGetPatternLocation( vuint8 blockNr, tPatternId patternId, IO_PositionType * pPatternAddress, IO_SizeType * pPatternLength )` | |
| **Parameter** | |
| `blockNr` | Number of the logical block |
| `patternId` | Pattern ID |
| `pPatternAddress` | Pointer to memory to put the pattern start address in |
| `pPatternLength` | Pointer to memory to put the length of a pattern into. |
| **Return code** | |
| `memSegment (>=0)` | Index into the flashBlock table where the pattern will be located to. The variable is used to call the respective memory driver. |
| < 0 | No valid pattern address found. |
| **Functional Description** | |
| This function calculates the start address of the pattern of a specific logical block. It uses the logical block table to calculate the address, which is normally at the very end of a logical block. | |
| **Particularities and Limitations** | |
| - | |
| Call context | |
| > Pattern functions (ApplFbl[Get\|Clr\|Set]PatternState) | |

### 9.2.2.46 ApplFblAdjustLbtBlockData

| Prototype | |
|---|---|
| `tFblResult ApplFblAdjustLbtBlockData( tBlockDescriptor * blockDescriptor )` | |
| **Parameter** | |
| `blockDescriptor` | Pointer to the logical block descriptor |
| **Return code** | |
| `kFblOk` | Adjustment of the blockDescriptor successfully performed. |
| `kFblFailed` | Adjustment has failed or block descriptor is invalid. |
| **Functional Description** | |
| This function is called by the diagnostic layer after the target logical block for the current download has been determined. In this function, the logical block size can be re-adjusted to the flashable area. This gives the possibility to restrict the download of data into a particular area. With the use of the pattern functionality, this function is used to spare the pattern area from the download, so that no data can be downloaded into this section. Only after successful validation of the logical block the function ApplFblSetPatternState is allowed to write into this area. Otherwise, an invalid download could lead to a valid pattern and thus lead to e.g. an accidental start of an invalid application. | |
| **Particularities and Limitations** | |
| - | |
| Call context | |
| > RequestDownload, RequestUpload, RoutineControl/CheckMemory | |

### 9.2.2.47 ApplFblGetPatternState

| Prototype | |
|---|---|
| `tFblResult ApplFblGetPatternState( tBlockDescriptor * blockDescriptor, tPatternId patternId )` | |
| **Parameter** | |
| `blockDescriptor` | Pointer to the logical block descriptor |
| `patternId` | Pattern ID |
| **Return code** | |
| `kFblOk` | Return this to indicate that pattern and mask in combination indicate a valid pattern state (e.g. valid logical block). |
| `kFblFailed` | Pattern state is invalid (e.g. invalid logical block). |
| **Functional Description** | |
| This function checks pattern and mask and returns the respective state. | |
| **Particularities and Limitations** | |
| - | |
| Call context | |
| > Validation functions, flash erased state handling | |

### 9.2.2.48 ApplFblClrPatternState

| Prototype | |
|---|---|
| `tFblResult ApplFblClrPatternState( tBlockDescriptor * blockDescriptor, tPatternId patternId )` | |
| **Parameter** | |
| `blockDescriptor` | Pointer to the logical block descriptor |
| `patternId` | Pattern ID |
| **Return code** | |
| `kFblOk` | Return this to indicate that the pattern has been successfully invalidated. |
| `kFblFailed` | Write to the pattern mask has failed. |
| **Functional Description** | |
| This function writes a value into the mask. This invalidates the pattern state. Note that the pattern/mask area must be cleared by a following erase operation before the pattern can be set to valid again (ApplFblSetPatternState). | |
| **Particularities and Limitations** | |
| - | |
| Call context | |
| > Validation functions, flash erased state handling | |

### 9.2.2.49 ApplFblSetPatternState

| Prototype | |
|---|---|
| `tFblResult ApplFblSetPatternState( tBlockDescriptor * blockDescriptor, tPatternId patternId)` | |
| **Parameter** | |
| blockDescriptor | Pointer to the logical block descriptor |
| `patternId` | Pattern ID |
| **Return code** | |
| kFblOk | Return this to indicate that the pattern has been successfully written. |
| kFblFailed | Writing the pattern has failed. |
| **Functional Description** | |
| This function sets the pattern for the given ID. | |
| **Particularities and Limitations** | |
| > Previous erase operation must have cleared this memory area. | |
| Call context | |
| > Validation functions, flash erased state handling | |

### 9.2.2.50 ApplFblDiagVerification

| Prototype | |
|---|---|
| `static SecM_StatusType` **`ApplFblDiagVerification`**`(V_MEMRAM1 SecM_VerifyParamType V_MEMRAM2 V_MEMRAM3 * verifyPar)` | |
| **Parameter** | |
| `verifyPar` | Parameter structure for verification |
| **Return code** | |
| `SECM_VER_OK` | Verification successful |
| `SECM_VER_ERROR` | Verification failed |

**Functional Description**

User specific function which can entered as “Output Verification” attribute in the Logical Block Table.



**Example**

```
SecM_StatusType ApplFblDiagVerification (V_MEMRAM1
SecM_VerifyParamType V_MEMRAM2 V_MEMRAM3 * verifyPar)
{
   verifyPar->verificationData[0u] = 0xCAu;
   verifyPar->verificationData[1u] = 0xFEu;
   return SECM_OK;
}
```

**Particularities and Limitations**

- There’s no default implementation of this function in the fbl_ap* modules. The function has to be added manually if required.

**Call context**

- Diagnostic services RequestTransferExit and RoutineControl/CheckProgrammingDependencies

### 9.2.2.51 ApplFblSetSecAccessDelayFlag

| Prototype | |
|---|---|
| `vuint8` **`ApplFblSetSecAccessDelayFlag`**`( void )` | |
| **Parameter** | |
| - | |
| **Return code** | |
| kFblOk | Return code is not evaluated by calling instance |

**Functional Description**

This callout function from the diagnostic layer can be used to store the state of the security access delay flag in non-volatile memory. It will be called every time the internal flag is set (due to multiple invalid security access attempts).

| Particularities and Limitations |
|---|
| > The default implementation of fbl_apnv.c stores the flag status within variable ‚nvSecAccessDelayFlag'. This variable is initialized with value ‚0' in fbl_ap.c::ApplFblInit(). Consequently, the state of the flag will be reset on reboot of the ECU. |
| Call context |
| > Security access implementation within fbl_diag.c (FblDiagSecAccessKey) |

### 9.2.2.52 ApplFblGetSecAccessDelayFlag

| Prototype | |
|---|---|
| `vuint8 ApplFblGetSecAccessDelayFlag( void )` | |
| **Parameter** | |
| - | |
| **Return code** | |
| state of security access delay flag | 0: Security access delay is inactive (state cleared)<br>Other values: Security access delay is active (state set) |
| **Functional Description** | |
| This callout function from the diagnostic layer can be used to restore the state of the security access delay flag from non-volatile memory. It will be called during startup of the bootloader to initialize the internal state of the diagnostic implementation. If a value != 0 is returned (delay active), the delay counter will be started and all security access requests will be responded with NRC 0x37 (RequiredTimeDelayNotExpired) until the configured delay time has expired. | |
| **Particularities and Limitations** | |
| > The default implementation of fbl_apnv.c stores the flag status within variable ‚nvSecAccessDelayFlag'. This variable is initialized with value ‚0' in fbl_ap.c::ApplFblInit(). Consequently, the state of the flag will be reset on reboot of the ECU. | |
| Call context | |
| > Initialization of fbl_diag.c (FblDiagInit) | |

### 9.2.2.53 ApplFblClrSecAccessDelayFlag

| Prototype | |
|---|---|
| `vuint8 ApplFblClrSecAccessDelayFlag( void )` | |
| **Parameter** | |
| - | |
| **Return code** | |
| kFblOk | Return code is not evaluated by calling instance |
| **Functional Description** | |
| This callout function from the diagnostic layer can be used to clear the state of the security access delay flag in non-volatile memory. It will be called every time the internal flag is cleared (after the configured delay time has expired). | |

| Particularities and Limitations |
|---|
| > The default implementation of fbl_apnv.c stores the flag status within variable ‚nvSecAccessDelayFlag‘. This variable is initialized with value ‚0‘ in fbl_ap.c::ApplFblInit(). Consequently, the state of the flag will be reset on reboot of the ECU. |
| Call context |
| > Task handling implementation within fbl_diag.c (FblDiagTimerTask) |

### 9.2.2.54 ApplFblReadSecAccessInvalidCount

| Prototype | |
|---|---|
| `vuint8 ApplFblReadSecAccessInvalidCount( vuint8 * invalidCnt )` | |
| **Parameter** | |
| invalidCnt | Invalid counter |
| **Return code** | |
| kFblOk | Variable has been set (NVM read operation succeeded) |
| kFblFailed | NVM read operation failed |
| **Functional Description** | |
| This callout function from the diagnostic layer can be used to persist the value of the security access delay counter in non-volatile memory. It will be called every time a wrong key is requested to read the current counter value. | |
| **Particularities and Limitations** | |
| > The default implementation of fbl_apnv.c stores the flag status within variable , nvSecAccessInvalidCnt ‘. This variable is initialized with value ‚0‘ in fbl_ap.c::ApplFblInit(). Consequently, the value of the counter will be reset on reboot of the ECU. | |
| Call context | |
| > Main handler of diagnostic service SecurityAccess/SendKey | |

### 9.2.2.55 ApplFblWriteSecAccessInvalidCount

| Prototype | |
|---|---|
| `vuint8 ApplFblWriteSecAccessInvalidCount( vuint8 * invalidCnt )` | |
| **Parameter** | |
| invalidCnt | Invalid counter |
| **Return code** | |
| kFblOk | Counter value has been persisted (NVM write operation succeeded) |
| kFblFailed | NVM write operation failed |
| **Functional Description** | |
| This callout function from the diagnostic layer can be used to persist the value of the security access delay counter in non-volatile memory. It will be called every time a wrong key is requested to read the current counter value. | |

| Particularities and Limitations |
| --- |
| > The default implementation of fbl_apnv.c stores the flag status within variable , nvSecAccessInvalidCnt ‘. This variable is initialized with value ‚0‘ in fbl_ap.c::ApplFblInit(). Consequently, the value of the counter will be reset on reboot of the ECU. |
| Call context |
| > Main handler of diagnostic service SecurityAccess/SendKey |

## 9.3 Data API

### 9.3.1 Types

The following data types are used in connection with the FBL callback functions

#### 9.3.1.1 tFblResult

| Structure | Elements | Type | Description |
|---|---|---|---|
| tFblResult | | unsigned char | Type is used to return result codes ‘kFblOk’ or ‘kFblFailed’ |

#### 9.3.1.2 tFblAddress

| Structure | Elements | Type | Description |
|---|---|---|---|
| tFblAddress | | see fbl_def.h (depends on hardware platform) | Type is used to store address information. This type equals FBL_ADDR_TYPE |

#### 9.3.1.3 tFblLength

| Structure | Elements | Type | Description |
|---|---|---|---|
| tFblLength | | see fbl_def.h (depends on hardware platform) | Type is used to store length information. This type equals FBL_MEMSIZE_TYPE |

#### 9.3.1.4 tApplStatus

| Structure | Elements | Type | Description |
|---|---|---|---|
| tApplStatus | | unsigned long | Type is used to return result codes as specified by REQPROD 53958 |

#### 9.3.1.5 tFblProgStatus

| Structure | Elements | Type | Description |
|---|---|---|---|
| tFblProgStatus | | unsigned char | Type is used to return result codes ‘kProgRequest’ and ‘kNoProgRequest’ |

#### 9.3.1.6 tFlashStatus

| Structure | Elements | Type | Description |
|---|---|---|---|
| tFlashStatus | | unsigned char | Type is used to return result codes ‘kFlashErased’ and ‘kFlashNotErased’ |

#### 9.3.1.7 tBlockDescriptor

| Structure | Elements | Type | Description |
|---|---|---|---|
| tBlockDescriptor | blockNr | vuint8 | Index of logical block in Fbl_Lbt.c |
| | blockType | vuint32 | Type of the logical block (software or SBL) |
| | blockStartAddress | tFblAddress | Start address of logical block |
| | blockLength | tFblLength | Length of logical block |
| | headerAddress | tFblAddress | Address of block specific validation structure |
| | verifyOutput | tExportFct | Verification function for block hash calculation |

#### 9.3.1.8 tDiagSegmentList

| Structure | Elements | Type | Description |
|---|---|---|---|
| tDiagSegmentList | nrOfSegments | unsigned char | Number of download segments |
| | segmentInfo | tSegmentInfo | Segment data |

#### 9.3.1.9 tDiagSegmentInfo

| Structure | Elements | Type | Description |
|---|---|---|---|
| tDiagSegmentInfo | targetAddress | tFblAddress | Start address of segment |
| | length | tFblLength | Segment length |
| | checksum | vuint8 [] | SHA256 calculated over segment data |

#### 9.3.1.10 tProcParam

| Structure | Elements | Type | Description |
|---|---|---|---|
| tProcParam | dataBuffer | vuint8* | Pointer to the input data buffer |
| | dataLength | vuint16 | Size of the data in the input buffer |
| | dataOutBuffer | vuint8* | Pointer to the output data buffer |
| | dataOutLength | vuint16 | Size of the data in the output buffer |
| | dataOutMaxLength | vuint16 | Size of the output data buffer |
| | wdTriggerFct | vuint8 (*wdTriggerFct) (void) | Pointer to the watchdog service function |
| | mode | vuint8 | Data processing mode; equals the data format identifier value (DFI) which is passed with RequestDownload |

#### 9.3.1.11 SecM_VerifyParamType

| Structure | Elements | Type | Description |
|---|---|---|---|
| SecM_VerifyParam-Type | segmentList | FL_SegmentListType | Address- and length info for segments |
| | blockStartAddress | SecM_AddrType | Start address of download block |
| | blockLength | SecM_SizeType | Length of download block |
| | verificationData | SecM_VerifyDataType | Pointer to verification data |
| | crcTotal | SecM_CRCType | Resulting CRC value |
| | wdTriggerFct | FL_WDTriggerFctType | Pointer to watchdog trigger function |
| | readMemory | FL_ReadMemoryFctType | Pointer to memory read function |

#### 9.3.1.12 SecM_StatusType

| Structure | Elements | Type | Description |
|---|---|---|---|
| SecM_StatusType | | vuint8 | Return type of SecM functions which can be one of the following:<br>• SECM_VER_OK<br>• SECM_VER_ERROR |

## 9.4 Implementation Details

### 9.4.1 Bootloader Assertions

The flash bootloader features an assertion mechanism which can be used to identify illegal bootloader states (e.g. caused by invalid configuration settings or incorrect return values from user functions). It is strongly recommended to enable this option during integration of the bootloader.

The bootloader assertions can be enabled by setting configuration option Project State to ‘Integration’. With the default configuration, the bootloader does only pass the ID code of the encountered assertion to the assertion handler.

If ‘#define FBL_ENABLE_ASSERTION_EXTENDED_INFO’ is set additionally in Fbl_Cfg.h (this cannot be enabled in the configuration tool), module name and line number of the source code location is passed, too.

The assertion handler ApplFblFatalError will be called in case an erroneous bootloader state has been detected. In order to reliably identify occurring assertions, it is a best practice to add a while(1) loop to ApplFblFatalError.

A list which comprises all possible assertion codes can be found in:

- BSW\Fbl\fbl_assert.h
- BSW\Fbl\fbl_assert_oem.h



**Caution**
Do not forget to turn off the bootloader assertions before going into production!

### 9.4.2 The Watchdog

For more general information about this see the UserManual_FlashBootloader in the chapter **The Watchdog**.

If the bootloader needs to refresh an on-chip or external watchdog, the configuration option Watchdog Service has to be enabled.

Serving the watchdog is hardware/application specific and therefore must be implemented by the user. There are two main watchdog functions to be considered by the user:

- FblLookForWatchdog: Internal bootloader function to generate the time base for triggering the watchdog (must not be changed by the user). The bootloader will call this function at a high rate to ensure that the timing is accurate. In addition, this function must be called from user specific code parts (e.g. long loops) to avoid a watchdog timeout.
- ApplFblWDTrigger: Hardware/application specific user function for refreshing the hardware watchdog. Together with FblLookForWatchdog, both functions have to be accessible during flash programming. That's why they are copied to RAM at startup. For this reason, the implementation of ApplFblWDTrigger should avoid any calls to other functions (or those functions have to be copied to RAM, too).

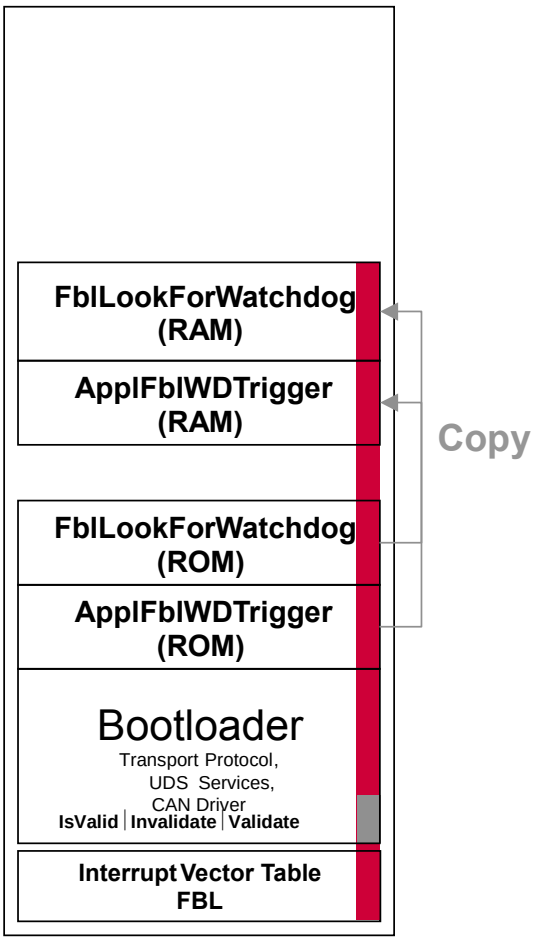



Figure 9-1 Memory Map of the Watchdog Code

Both functions are copied into RAM before programming the flash memory as it is not possible on most MCUs to execute code from flash memory while reprogramming. Copying the functions into RAM is either accomplished by a copy function of the bootloader (**FblCopyWatchdog**) or by the startup code (which is the most commonly used method nowadays). In the first case, the watchdog trigger functions must be relocatable!

#### 9.4.2.1 Watchdog Synchronization Support

There are several callback functions which permit flexible synchronization methods for watchdog handling.

- AppIFblWDLong: This function is called before the FBL performs a reset or before transitioning to the application. It can be either used to switch a hardware watchdog device to a longer period if the transition time is too long for accurate watchdog triggering or to synchronize watchdog triggering if a window watchdog is used.
- AppIFblWDShort: This function is called after the FBL is started from application. It can be used to switch the hardware watchdog device to the normal short period or to synchronize the trigger for a window watchdog.
- AppIFblWDInit: This function is called when the FBL is started from reset except when the reset is caused by a programming request received by the application before transition to the bootloader.

Additional watchdog callback functions are sometimes provided based on the hardware used. These functions usually support special features like windowed watchdog timers. For more details on these functions, consult the hardware technical reference of your flash bootloader.

### 9.4.3 ApplFblReadDataByIdentifier

This function can be used to implement so-called “complex” DIDs. “Complex” means the data which shall be reported by the DID is not a constant value or is not memory mapped (i.e. reading requires a special memory driver). Constant values can be added or modified by adapting the `fblDidTable[]` which can be found at the top of fbl_apdi.c. Each table entry consists of three members:

- DID number (e.g. 0xF111 for the ECU Core Assembly Number)
- DID data size in byte (without service ID and DID number; e.g. 24 for 0xF111)
- Pointer to the constant DID data. If the data shall be shared between bootloader and application, it’s recommended to point to the Common Data Structure.

If a complex DID shall be reported, you still have to add DID number and data size to the table. However, if you enter the define `kDiagComplexDidData` instead of a pointer to the data, the diagnostic core implementation will call user function fbl_apdi.c::ApplFblReadDataByIdentifier.

The current implementation of this callback function illustrates how to report a complex DID with the bootloader. The user needs to check for which DID the function has been called. If it’s called for a supported complex DID, the pbDiagData buffer has to be loaded with the response data and the returnCode is set to kFblOk. The data length will be set automatically based on the value in the table.



**Caution**
You shouldn’t set any NRC (using the provided DiagNRC* macros) in the callback function. Returning kFblFailed in case of failure is sufficient.

### 9.4.4 ApplFblWriteDataByIdentifier

If the ECU is required to support the diagnostic service $2E, then the callback function ApplFblWriteDataByIdentifier shall be used to handle the incoming DID requests. Prior to using this service, option “WriteDataByIdentifier Service” (chapter 4.6.7) has to be enabled in the configuration tool.

Prior to programming a DID, the user implementation should check if:

- The DID is supported
- The message length and format of the corresponding DID is correct
- Security access has been granted (optional)
- The SBL has been activated

In case of detecting an error in any of these checks, the user shall issue the corresponding error using the DiagNRCxx macros. A list of these macros along with the corresponding

NRCs can be found in fbl_diag.h. In addition to that, the user shall return kFblFailed in case of an error and kFblOk in case of a successful operation.

### 9.4.5 One-Time Programmable Area

The bootloader uses a one-time programmable (OTP) area to store non-volatile meta data. This area shall be located inside the bootloader's protected area, so it cannot be modified using software download.

Currently, the bootloader will use this area to retain the erased state of the logical blocks (Flash Erased Detection) and the value of the public key for production. The OTP area can be extended with user specific entries if required.

The OTP framework is enabled by default. If you don't want to use the OTP area, you can simply remove the respective functions from fbl_apnv.c.

The structure of the OTP area is defined in module fbl_apnv.c:

- FBL_OTP_*_SIZE: Specifies the actual size of the OTP entry.
- FBL_OTP_TOTAL_SIZE: Reflects the total size of the OTP area. This define has to be adapted if additional OTP entries are added. The total size is calculated by summarizing the lengths of the individual entries after aligning them to the hardware specific write segment size (see chapter 4.3).
- FBL_OTP_*_OFFSET: Defines the offset of the respective OTP entry in relation to the start address of the OTP area.
- FBL_OTP_BASE_ADDRESS: This is the start address of the OTP area.
- FBL_OTP_*_ADDRESS: This define provides the absolute address of the OTP entry by combination of OTP area's start address and the offset.



**Example**

```
/* One-time-programmable (OTP) storage location in PBL area */
/* Helper macro to align size of OTP entries to memory write segment size*/
# define FblOtpAlignSize(size)          ((((size) + FLASH_SEGMENT_SIZE - 1) /
                                         FLASH_SEGMENT_SIZE) * FLASH_SEGMENT_SIZE)

/* Size of OTP storage members */
# define FBL_OTP_PRODKEY_MOD_SIZE       (sizeof(SecM_RsaMod))
# define FBL_OTP_PRODKEY_EXP_SIZE       (sizeof(SecM_RsaExp))
# define FBL_OTP_PRODKEY_CSUM_SIZE      (SEC_SIZE_HASH_SHA256)
# if defined( FBL_DIAG_ENABLE_UPDATE_SEC_BYTES )
#  define FBL_OTP_SECURITY_BYTES_SIZE   (SEC_ECU_KEY_LENGTH)
# else
#  define FBL_OTP_SECURITY_BYTES_SIZE    0u
# endif
# define FBL_OTP_TOTAL_SIZE             (FblOtpAlignSize(FBL_OTP_PRODKEY_MOD_SIZE)  + \
                                         FblOtpAlignSize(FBL_OTP_PRODKEY_EXP_SIZE)  + \
                                         FblOtpAlignSize(FBL_OTP_PRODKEY_CSUM_SIZE) + \
                                         FblOtpAlignSize(FBL_OTP_SECURITY_BYTES_SIZE))

/* Offset values for OTP area members */
# define FBL_OTP_PRODKEY_MOD_OFFSET      0u
# define FBL_OTP_PRODKEY_EXP_OFFSET      FBL_OTP_PRODKEY_MOD_OFFSET + \
                                         FblOtpAlignSize(FBL_OTP_PRODKEY_MOD_SIZE))
# define FBL_OTP_PRODKEY_CSUM_OFFSET     FBL_OTP_PRODKEY_EXP_OFFSET + \
                                         FblOtpAlignSize(FBL_OTP_PRODKEY_EXP_SIZE))
# define FBL_OTP_SECURITY_BYTES_OFFSET   FBL_OTP_PRODKEY_CSUM_OFFSET + \
                                         FblOtpAlignSize(FBL_OTP_PRODKEY_CSUM_SIZE))

/* Absolute addresses of OTP area members */
# define FBL_OTP_BASE_ADDRESS            ((tFblAddress)nvOtpArea)
# define FBL_OTP_PRODKEY_MOD_ADDRESS \
                              ((tFblAddress)(&nvOtpArea[FBL_OTP_PRODKEY_MOD_OFFSET]))
# define FBL_OTP_PRODKEY_EXP_ADDRESS \
                              ((tFblAddress)(&nvOtpArea[FBL_OTP_PRODKEY_EXP_OFFSET]))
# define FBL_OTP_PRODKEY_CSUM_ADDRESS \
                              ((tFblAddress)(&nvOtpArea[FBL_OTP_PRODKEY_CSUM_OFFSET]))
# define FBL_OTP_SECURITY_BYTES_ADDRESS \
                              ((tFblAddress)(&nvOtpArea[FBL_OTP_SECURITY_BYTES_OFFSET]))
```

The address and size defines are intended to be used with the OTP area access functions AppIFblOtpReadEntry() and AppIFblOtpWriteEntry(). Please see chapter 9.2 for details about these functions.

To avoid potential overlapping of bootloader code and OTP area, the default implementation of fbl_apnv.c reserves the respective memory space by defining ROM array nvOtpArea[FBL_OTP_TOTAL_SIZE]. By using MemMap.h, the array can be linked to a dedicated memory space. The following constraints shall be taken into account:

- The OTP area shall be placed inside a protected flash block (bootloader area). Otherwise, the pattern value can be inadmissibly reset if the memory is erased. Per definition, this is not allowed for a one-time programmable area.
- The start address of the OTP area has to be aligned to the memory specific write segment size
- It's recommended to reserve some extra space following the OTP area to avoid problems if the area grows (e.g. because an additional logical block is added to the configuration).

In order to successfully use the OTP area, it's recommended to follow the process described below.

- Configure the bootloader with OTP framework enabled (FBL_APPL_ENABLE_OTP).
- Adapt the linker configuration so that the OTP area is located according to the constraints described above.
- Build the bootloader and generate a .hex file.
- Remove the OTP area from the .hex file (e.g. using HexView)
- Use a flash programmer to erase the whole available flash memory which is addressable by the ECU.
- Program the prepared bootloader .hex file into the ECU.
- Execute a test run. Verify that the OTP area is written correctly according to the expectations.



**Caution**
The OTP framework is provided as an example implementation. In particular, it might be more convenient (and definitely more secure!) to store meta information in an EEPROM (if available).

**Any failure during the OTP write operation (e.g. voltage drop, watchdog reset) will irreversibly corrupt the respective OTP entry.**



**Caution**
It's strongly recommended to disable OTP support for bootloader updates (i.e. if a new bootloader version is programmed over an old version using a so-called bootloader updater).

### 9.4.6 Flash Erased Detection

The flash erased detection provides the possibility to deliver ECUs to the OEM with the complete programmable flash memory already erased to reduce the overall software download time.

If this option is enabled, the bootloader uses API function ApplFblGetBlockErased() which is called prior to any erase operation. Depending on the return value, the actual erase operation is performed or not.

The default implementation in ApplFblGetBlockErased() checks the presence of a certain pattern which is located at the end of the logical block. If the pattern is valid, the bootloader assumes that the flash memory is in 'erased' state. In this case, the actual erase operation is omitted. The pattern will be invalidated as soon as a RequestDownload request for the respective block has been received. Further erase requests will result in the normal

execution of the erase operation. Each time the block has been erased successfully, the pattern will be set again.



**Note**
The pattern handling will only work reliably, if all erase/write access to the non-volatile memory is performed by software download (i.e. using the diagnostic services RequestDownload and TransferData). If this cannot be ensured, the flash erased detection should be disabled.

If an ECU shall be delivered pre-erased and with flash erased detection enabled, there are certain preparation steps which have to be executed:

> Configure the bootloader with flash erased detection enabled.

> Remove any parts from the bootloader .hex file which are located in the memory area(s) that are reprogrammable by the bootloader. In particular, this affects the application vector table which is always linked with the bootloader.

> Add the valid flash erased pattern(s) to the .hex file. Each logical block has its own pattern stored at the end of the block. The actual value is defined by constant `blockPresencePattern[]` in fbl_apnv.c. The location of the pattern has to be calculated depending on the memory write segment size.



**Note**
Don’t confuse the pattern/mask area which is used for the flash erased detection with those which is used for the block specific Presence Pattern. The pattern for the flash erased detection uses the exact same mechanism but is located below the presence pattern. See definition of `tPatternId` in fbl_apnv.h.

> Execute a test run. Verify that the time for the first erasure is approximately 0 ms and that the download completes successfully. Afterwards, start another test run and verify that the erase time is considerably higher than 0 ms.

### 9.4.7 Reprogrammable LBT/ESS

Usually, the LBT (VolvoCar specific term: ESS) is built into the (primary) bootloader. Thus, the memory layout of the ECU cannot be changed after the bootloader has been programmed into the ECU. The functionality described as “Reprogrammable LBT” in this chapter has been designed to allow post-build reprogramming of the LBT.

The reprogrammable LBT can be enabled or disabled using the respective configuration switch. If the option is enabled, the generated Fbl_Lbt.c/Fbl_Lbt.h files will only contain the SBL and LBT blocks. The full version of the LBT will be generated into the file Fbl_Lbt_PBcfg.c.

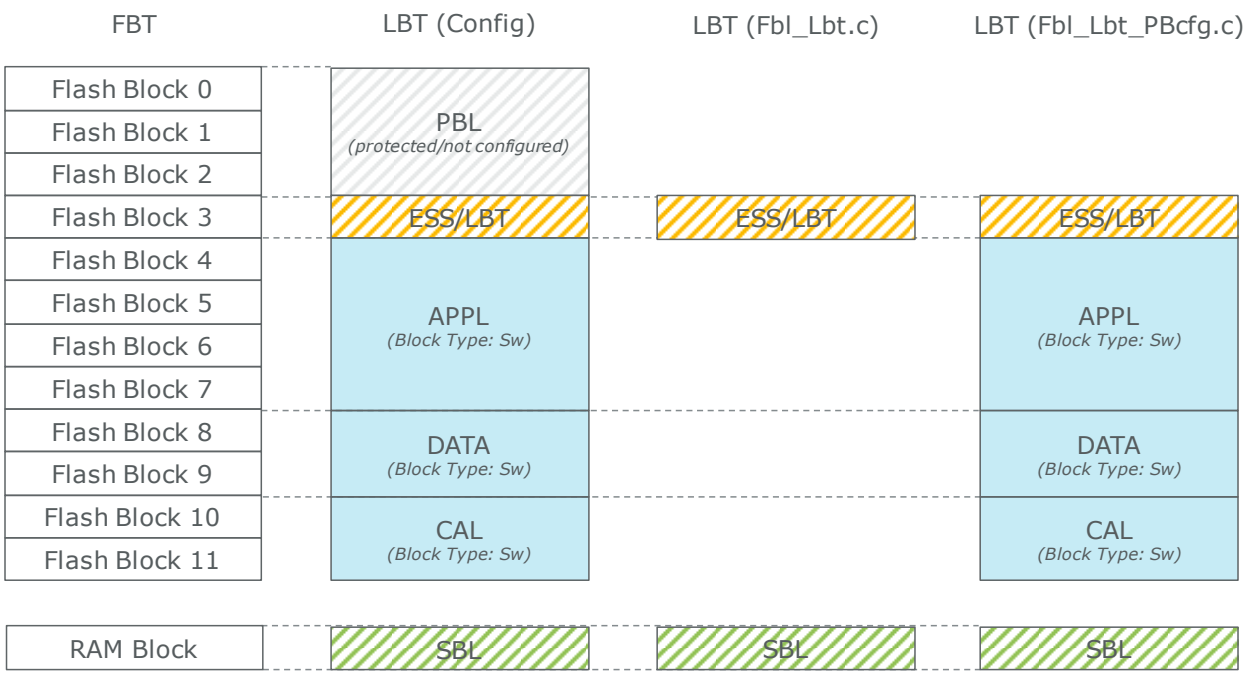



Figure 9-2 LBT Configuration with Option “Reprogrammable LBT”

### 9.4.7.1 Constraints

Please consider the following constraints regarding the usage of the reprogrammable LBT option:

- The reprogrammable LBT must be placed into a separate flash block/logical block. No other code or data can be placed into this block.
- The maximum number of logical blocks which can be programmed with an LBT update is limited by configuration option Maximum Amount Of Logical Blocks. This value is built into the PBL and cannot be changed post-build.
- LBT and SBL block parameters are fixed. The bootloader will deny the LBT update if the block parameters don’t match between built-in and reprogrammable LBT.
- The reprogrammable LBT is still based on the (hardware specific) FBT. The FBT is compiled into the (primary) bootloader and cannot be changed. This has the following implications:
  - The FBT should be configured with a reasonable granularity which allows a flexible re-adjustment of the logical blocks.
  - Update LBTs (generated into Fbl_Lbt_PBcfg.c) must be generated based on the exact same FBT as those of the (primary) bootloader.
  - The bootloader will deny an LBT update if it detects that the new LBT has been created based on a changed FBT.
- References, which are used inside the LBT cannot be changed by an LBT update. This comprises the following items:
  - Block Type

- Any kind of verification function (e.g. Output Verification)

Effectively, this means that you cannot introduce a new block type or verification function after the (primary) bootloader has been programmed into the ECU. The bootloader will deny an LBT update request if it detects inapplicable references.

#### 9.4.7.2 Configuration

The configuration of the reprogrammable LBT option comprises the following steps in DaVinci Configurator:

- Enable the option Reprogrammable LBT
- Set the Maximum Amount Of Logical Blocks
- Configure the flash block which shall contain the reprogrammable LBT
- Create a logical block with the following parameters:
  - Short Name: LBT
  - Block Type: ESS
  - Disposability: MANDATORY
  - All other parameters are user specific, please consider the corresponding hints in the configuration chapter
- Assign the logical block to the configured flash block

#### 9.4.7.3 Build and Post-Processing

It's recommended to add the generated file Fbl_Lbt_PBcfg.c to the build process of the bootloader. You can use the MemMap.h mechanism to create a dedicated section which can be linked to the LBT block address.

While this LBT version won't be used by the bootloader directly (at least if you don't set the LBT block to 'valid'), you can extract the post-build version of the LBT directly from the bootloader's hex binary and create a download container (e.g. with HexView).



**Note**
During integration of the bootloader with option 'Reprogrammable LBT' enabled, each build might lead to changing addresses of the verification functions which are referenced in the LBT. Thus, it's recommended to automate the creation of the LBT/ESS download container based on the current bootloader binary to avoid inconsistencies between PBL and reprogrammable LBT.

#### 9.4.7.4 LBT Update Process

The first download after programming of the flash bootloader must always contain a valid LBT block (see "Step 2" in the figure below). All other blocks (except the SBL) will be rejected if no valid LBT block has been downloaded.

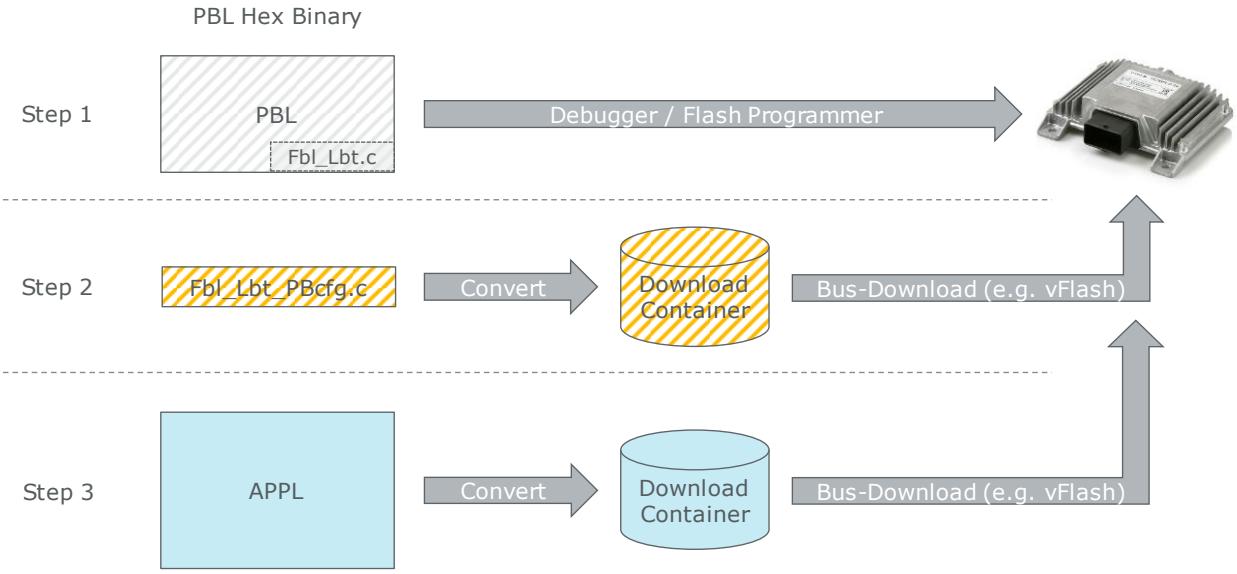



Figure 9-3 Initial Download Sequence with Option “Reprogrammable LBT”

When the bootloader receives a request for (re-)programming of the LBT block, a special handling is activated:

- Invalidate all software blocks which are configured in the current LBT
- Erase all software blocks which are configured in the current LBT
- Activate the built-in LBT (LBT and SBL block only)

As for any other software block, the bootloader will perform the software authentication check for the LBT download. If this check has been passed successfully, the bootloader will apply additional consistency checks before the block is set to ‘valid’ (see chapter 9.4.7.1 Constraints for details).

If authentication and consistency check have been passed successfully, the bootloader will instantly switch to the new LBT. That means, an LBT download can directly be followed by other software modules which are built against the new LBT version.

On startup, the bootloader will always check the validity of the LBT block and activate the reprogrammable LBT if applicable.

### 9.4.7.5 LBT Access from User Code

If the option “Reprogrammable LBT” has been enabled, direct access to the structure `FblLogicalBlockTable` must be avoided. Please use the API functions which are provided by module fbl_lbt_access. The name of the `FblLbtGet…` functions should be self-explanatory.

Note that you cannot rely on the block number of a logical block. The block number might be subject to change, e.g. if a block is added or removed. For this purpose, you can define a user specific block index in DaVinci Configurator. This index must be unique and can be used as a handle to address a certain block. E.g. you can use function `FblLbtGetBlockDescriptorByIndex` to get the block parameters based on the index.

Module fbl_lbt_access also provides a filter-based iterator, which allows easy iteration over certain types of blocks. See example below.



**Example**

```
tBlockDescriptor blockDescriptor;
tFblLbtBlockFilter blockFilter;

/* Iterate over all mandatory logical blocks of type 'Software' */
FblLbtBlockFilterInit(&blockFilter);
FblLbtBlockFilterSetMandatoryType(&blockFilter, TRUE);
FblLbtBlockFilterSetBlockType(&blockFilter, kBlockTypeSw);

for (blockDescriptor = FblLbtBlockFirst(&blockFilter);
     FblLbtBlockDone() == FALSE;
     blockDescriptor = FblLbtBlockNext())
{
   /* Access block parameters via blockDescriptor */
}
```

# 10 Miscellaneous

## 10.1 Functional vs. Physical

The VolvoCar flash bootloader supports two diagnostic message IDs: Functional and Physical.

The functional message ID is $7xF and can be received by any SPA ECU. The physical request IDs are ECU specific and are configured based on the CAN database. VolvoCar uses the following convention: Physical Response ID = Physical Request ID - 0x100.

One restriction on functional requests is that they can be sent only as single frame requests. Large data requests may not be transmitted using a functional message. For more details please refer to [3].

# 11 Deviations

The following table lists deviations between the VolvoCar specifications and the implementation inside the Vector bootloader.

| Requirement | Deviation |
|---|---|
| REQPROD 53978<br>REQPROD 74142<br>REQPROD 128908<br>REQPROD 128928 | The Vector bootloader starts to write data to the non-volatile memory when a complete TransferData message has been received.<br>The Vector bootloader uses the 'Pipelined Programming' function which responds positively to the tester once all data in the TransferData message is received but not written to the non-volatile memory. To decrease the programming time, the ECU will start to receive the next TransferData message while in parallel the previous received Transfer Data message is written to the non-volatile memory. |
| REQPROD 53836<br>REQPROD 53856<br>REQPROD 53858<br>REQPROD 53909 | The SBL from Vector will only contain services (flash driver routines) for erasing and writing to non-volatile memory. |
| REQPROD 60102 | The N_As timeout is handled by the transport protocol implementation as required (the TP layer will be reset on timeout). However, in situations, where the transmission will be blocked until N_As has expired, the respective frame will appear on the bus nevertheless. This is because the request has already been passed to the data link layer and the request cannot be recalled in case of an upper layer timeout. |
| REQPROD 392111 | By default, the bootloader performs an ECU reset on detection of a bus-off situation. The behavior can be adapted by modifying user function fbl_ap.c::ApplFblBusOff(). |
| REQPROD 400961 | By default, the validity information is always stored as pattern in flash memory. Hardware specific security features are not used yet (but can be implemented by the user). |
| REQPROD 366329 | By default, the public key is always stored inside the protected PBL memory area (one-time programmable). Hardware specific security features are not used yet (but can be implemented by the user). |
| REQPROD 366327 | By default, the authenticity verification function is always stored inside the protected PBL memory area. Hardware specific security features are not used yet (but can be implemented by the user). |
| REQPROD 419731 | This requirement is not supported by default but can be implemented by the user. |

| Requirement | Deviation |
|---|---|
| REQPROD 56204 | The bootloader itself doesn't supervise the P4Server_max timeout.<br>Rationale: There's no appropriate action which the bootloader can take in that case. Abortion of the current operation would most likely result in more problems than finishing the operation even though P4Server_max has been exceeded.<br>The bootloader testing comprises a test case which checks the implementation by inspection if a RCRRP is allowed to be sent or not.<br>However, it has to be taken into account that the actual response behavior strongly depends on the hardware choice and the bootloader configuration.<br>That means, the fulfillment of this requirement has to be verified by the Tier 1 supplier after configuration/adaptation of the bootloader has been finished. |
| REQPROD 60236 | The first FC.Wait frame will appear immediately after the first frame, not right before the C_Br timeout.<br>The following FC.CTS will appear exactly after time which is set in the configuration tool, following the last FC.Wait (in contrast to "immediately after the final response of the preceding request has been sent"). |
| REQPROD 53838 | Might be violated depending on the bootloaders' memory configuration (e.g. compiled-in EEPROM or data flash driver). However, at least the program flash driver will usually fulfill the requirement. |

Table 11-1 List of deviations between VolvoCar specifications and the Vector bootloader

# 12 Terminology

| Abbreviation | Description |
|---|---|
| CAN | Controller Area Network |
| DID | Data Identifier |
| DTC | Diagnostic Trouble Code |
| ECU | Electronic Control Unit |
| ESS | ECU Software Structure (Volvo specific term for Logical Block Table) |
| FBL | Flash Bootloader |
| FBT | Flash Block Table |
| HIS | Hersteller Initiative Software<br>Refers to an effort made by a group of manufacturers to standardize aspects of software development. The FBL bases the low-level Memory I/O (e.g. the Flash Driver) APIs on this work. |
| ISO | International Organization for Standardization |
| LBT | Logical Block Table |
| LIN | Local Interconnect Network |
| MCU | Microcontroller Unit |
| NRC | Negative Response Code |
| OTP | One-time programmable |
| PBL | Primary Bootloader |
| RCRRP | Request Correctly Received, Response Pending |
| SBL | Secondary Bootloader |
| SWDL | Software Download Specification |
| TP | Transport Protocol |
| VBF | Versatile Binary Format (container format for download files) |

Table 12-1 Abbreviations

# 13 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com